

电子科技大学

UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

硕士学位论文

MASTER THESIS



论文题目

基于模型的多源异构数据自适应

可视化技术研究

学科专业

通信与信息系统

学 号

201821010633

作者姓名

彭玉婷

指导教师

杨 鍊 副教授

分类号 _____ 密级 _____

UDC ^{注1} _____

学 位 论 文

基于模型的多源异构数据自适应

可视化技术研究

(题名和副题名)

彭玉婷

(作者姓名)

指导教师

杨 鍊

副教授

电子科技大学

成 都

(姓名、职称、单位名称)

申请学位级别 硕士 学科专业 通信与信息系统

提交论文日期 2021.6 论文答辩日期 2021.6

学位授予单位和日期 电子科技大学 2021年 6 月

答辩委员会主席 文光俊

评阅人 李桓 潘晔

注 1: 注明《国际十进分类法 UDC》的类号。

Research on Adaptive Visualization Technology of Multi Source Heterogeneous Data Based on Model

A Master Thesis Submitted to

University of Electronic Science and Technology of China

Discipline: **Communication and Information System**

Author: **Yuting Peng**

Supervisor: **Prof. Lian Yang**

School: **School of Information and Communication**

Engineering

摘要

随着新兴信息技术的快速迭代,各应用领域产生了大量的多源异构数据。如何对这些数据统一整合和处理,挖掘出有价值的信息,引起了学术界的大量关注与研究。数据可视化将数据转化为不同类型的可视化元素直观展示,是数据处理与分析流程的重要环节。本文针对多源异构数据可视化存在的问题,研究了可视化流程中的组件生成技术和基于可视化组件的大屏构建技术。主要内容如下:

(1) 实现了一种基于模型的组件可重构生成技术。

根据可视化组件的生成原理,本文提出一种通过可视化界面配置快速生成组件的方法。首先,本文分析了传统组件库生成组件的流程与存在的缺点。接着,本文提出一种组件快速生成框架,并构建了基于 Echarts 的图表模板,以模型化的方式生成组件。最后,引入可重构的概念,实现组件可重构配置。实验结果证明,本框架集成的可重构的组件模板达 36 例,在集成组件的种类和数量上优于大部分同类工具,且框架具有可扩展的特点。此外,与主流组件库的对比实验表明,本方法构建的组件在性能上具有更短的初始化时间和更高的动画帧率。

(2) 实现了一种基于页面自适应的大屏敏捷构建技术。

针对可视化大屏构建流程复杂的问题,本文提出一种基于组件动态构建自适应大屏的方法。首先,本文对比了主流的大屏构建方式的优缺点,分析了基于商业智能工具构建大屏的灵活性。接着,在研究大屏动态构建的机制上,本文提出了一种基于可视化组件的大屏敏捷构建框架,并设计了三种大屏页面的自适应方案,以实现可视化结果的自适应展现。最后,开发了构建敏捷大屏的通用工具。测试结果显示,基于本工具构建的大屏,在功能上具有较好的易用性与交互性,在性能上也表现良好,满足实际应用的可视化需求。

(3) 实现了多源异构数据可视化的案例研究

基于上述的研究成果,本文面向多源异构数据可视化的需求,使用开发的工具,快速搭建联合作战电磁态势数据可视化分析平台。实现对复杂多源的电磁态势关键指标数据进行实时可视化展示与分析。案例实现结果表明,基于本文提出的组件快速生成方法和大屏敏捷构建方法可以面向多样的可视化需求,快速构建相应的数据分析平台。因此本文的研究成果对于不同领域的数据可视化分析具有较高的应用价值。此外,本文提出的大屏敏捷构建工具已在实际项目中获得了较好的反响。

关键词: 多源异构数据可视化, Echarts, 组件可重构, 大屏敏捷构建

ABSTRACT

With the rapid iteration of emerging information technologies, a large quantity of multi-source heterogeneous data has been generated in various application fields. How to integrate and process these data to mine valuable information has aroused a lot of attention and research in academia. Data visualization transforms data into different types of visual elements, which is a vital part of data processing and analysis process. Aiming at the problems of multi-source heterogeneous data visualization, this thesis studies the widget generation technology and the visualization dashboard construction technology based on widget in the visualization process. The main contents are stated as follows:

(1) A model-based widget reconfigurable generation method is proposed.

According to the generation principle of visual widgets, a method to quickly generate widgets through visual interface configuration is proposed. Firstly, the process of generating widgets from the traditional visualization library and the existing shortcomings are specifically analyzed. Then, a rapid widgets generation framework was proposed, and charts templates based on Echarts were constructed to generate widgets in a modeled manner. Finally, the concept of reconfigurable is introduced to realize the reconfigurable configuration of widgets. The experimental results prove that the framework integrates 36 cases of reconfigurable widget templates, which is superior to most similar tools in the types and quantity of integrated widgets, and the framework is extensible. In addition, comparison experiments with mainstream visualization libraries show that the widget constructed by this method has shorter initialization time and higher animation frame rate in performance.

(2) A visualization dashboard agile construction method based on page adaptation is proposed.

In view of the complex process of visual dashboard construction, a method for dynamically building adaptive dashboard based on widgets is proposed. First of all, the advantages and disadvantages of mainstream dashboard construction methods are compared, and the flexibility of building dashboards based on business intelligence tools is analyzed. Then, in the study of the mechanism of dynamic construction of dashboards, an agile construction framework for dashboards based on visualization widgets was proposed, and three adaptive solutions for dashboard pages were designed to realize the

adaptive display of visualization results. Finally, a general tool for building agile dashboards was developed. The test results show that the dashboard built based on this tool has high usability and interactivity in function, and also performs well in terms of performance, which meets the visualization requirements of practical applications.

(3) The case study of multi-source heterogeneous data visualization is realized.

Based on the above research results, this thesis aims at the requirements of multi-source heterogeneous data visualization, and uses the tool developed to quickly build a joint operation electromagnetic situation data visualization analysis platform. The key indicator data of complex and multi-source electromagnetic situation can be visualized and analyzed in real time. The case study results show that based on the proposed rapid widget generation method and visualization dashboard agile construction method can quickly build the corresponding data analysis platform for diverse visualization needs. Therefore, the research results have high application value for data visualization analysis in different fields. In addition, the visualization dashboard construction tool proposed has been well received in actual projects.

Keywords: Multi Source Heterogeneous Data Visualization, Echarts, Widget Re-configuration, Agile Construction of Dashboard

目录

第一章 绪 论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	3
1.3 本文研究内容与结构安排	6
第二章 多源异构数据可视化及其关键技术概述	8
2.1 多源异构数据可视化相关理论	8
2.2 数据采集技术	11
2.3 数据处理技术	14
2.5 组件生成技术	18
2.6 大屏展现技术	21
2.7 本章小结	23
第三章 基于模型的组件可重构生成技术研究	24
3.1 组件生成原理	24
3.2 传统组件库的研究	25
3.2.1 基于矢量绘制的组件库	25
3.2.2 基于像素渲染的组件库	27
3.2.3 不同组件库的对比分析	27
3.3 基于模型的组件可重构生成技术	28
3.3.1 总体流程	30
3.3.2 组件快速生成框架	30
3.3.3 模型构建	37
3.3.4 组件可重构实现	40
3.3.5 扩展组件库方案	43
3.4 实验及结果分析	44
3.4.1 扩展组件库实验结果	44
3.4.2 组件库的功能测试	46
3.4.3 组件库的性能测试	48
3.4.4 与同类组件库的比较	50
3.5 本章小结	52
第四章 基于页面自适应的大屏敏捷构建技术研究	53

4.1 传统大屏构建方式	53
4.1.1 基于前端技术构建大屏	53
4.1.2 基于 BI 报表动态构建大屏	54
4.1.3 分析小结	55
4.2 基于页面自适应的大屏敏捷构建技术	55
4.2.1 敏捷构建系统总体设计	55
4.2.2 大屏动态构建	58
4.2.3 自适应策略	63
4.2.4 大屏管理机制	65
4.2.5 大屏交互	66
4.3 实验及结果分析	67
4.3.1 功能测试	67
4.3.2 性能测试	72
4.4 本章小结	75
第五章 面向多源异构数据的可视化案例展示	76
5.1 案例需求分析	76
5.1.1 电磁设备可视化	76
5.1.2 电磁行动可视化	76
5.1.3 电磁态势四域分布可视化	77
5.2 电磁态势数据处理	77
5.3 电磁态势可视化分析场景实现	78
5.3.1 电磁数据查询集合	78
5.3.2 组件设计与生成	79
5.3.3 可视化分析场景实现	80
5.3.4 结果分析	81
5.4 本章小结	81
第六章 总结与展望	82
6.1 总结	82
6.2 未来展望	82
致谢	84
参考文献	85
攻读硕士学位期间研究成果	89

图目录

图 1-1 可视化大屏在实际应用的实现场景	2
图 1-2 历年来可视化文献发表数量	3
图 2-1 数据可视化的元素构成	9
图 2-2 数据与图形空间的映射	9
图 2-3 信息可视化流程	10
图 2-4 多源异构数据可视化流程	11
图 2-5 JDBC 访问数据库的接口规范	12
图 2-6 Python 访问数据库的接口规范	12
图 2-7 Python 查询数据库的流程	13
图 2-8 通用型爬虫的工作原理	14
图 2-9 物化式整合原理	17
图 2-10 虚拟式整合原理	17
图 2-11 组件生成技术架构组成	18
图 2-12 标记类型示例	19
图 2-13 视觉通道	19
图 2-14 大屏展现技术架构组成	21
图 3-1 组件库生成组件的通用流程	24
图 3-2 Highcharts 生成组件的流程	25
图 3-3 D3 生成组件的流程	26
图 3-4 基于不同组件库生成的折线图组件	27
图 3-5 基于模型的组件可重构技术架构	29
图 3-6 基于组件快速生成框架生成组件的流程	30
图 3-7 Angular 的生命周期流程	31
图 3-8 MVVM 架构	32
图 3-9 Angular 的事件循环机制原理	32
图 3-10 基于 Angular 的组件快速生成框架实现	33
图 3-11 接口配置页面实现效果图	34
图 3-12 两种浏览器事件传播机制示意图	35
图 3-13 画布实时更新流程	35
图 3-14 防抖算法的原理	36
图 3-15 画布页面渲染组件前后的效果图	36

图 3-16 模型内部流程图	37
图 3-17 Echarts 图表模板的组成结构	38
图 3-18 Echarts 图表模板各个模块之间的交互	38
图 3-19 基于饼图模板的页面渲染示意图	39
图 3-20 组件编辑页面实现效果图	40
图 3-21 可重构模块与其他模块的交互	41
图 3-22 组件可重构流程	42
图 3-23 Plotly 图表模板各个模块之间的交互	43
图 3-24 构建的 Echarts 图表模板	45
图 3-25 构建的 Plotly 图表模板	45
图 3-26 构建的表格组件模板	45
图 3-27 默认样式和属性配置测试结果	47
图 3-28 组件预览测试结果	47
图 3-29 大数据量情况的渲染测试结果	48
图 3-30 生成不同组件的性能测试结果	49
图 3-31 不同图表类型的初始化时间测试结果	51
图 3-32 不同图表类型的动画帧率测试结果	51
图 4-1 基于前端技术构建大屏的流程	54
图 4-2 基于 BI 工具构建大屏的流程	54
图 4-3 大屏敏捷构建系统的模块组成	55
图 4-4 系统的整体架构图	56
图 4-5 系统前端架构图	57
图 4-6 系统后端架构图	58
图 4-7 大屏动态构建功能模块组成	58
图 4-8 大屏编辑界面的原型图	59
图 4-9 Flex 布局结构	60
图 4-10 组件动态配置实现流程	61
图 4-11 拖拽实现原理	62
图 4-12 大屏样式编辑流程	63
图 4-13 自适应策略	63
图 4-14 大屏管理界面原型图	65
图 4-15 数据交互的实现流程	66
图 4-16 大屏共享解决方案	67

图 4-17 构建的完整大屏实现结果	68
图 4-18 在浏览器打开 URL 后的大屏共享结果	69
图 4-19 组件拖拽前、后自适应测试结果	70
图 4-20 全屏模式下和非全屏模式下自适应测试结果	70
图 4-21 浏览器占用内存测试结果	72
图 4-22 调优前后浏览器占用内存性能对比结果	73
图 4-23 实时大屏响应时延测试结果	74
图 4-24 引入“节流”前后的数据更新流程对比。	74
图 4-25 调优前后实时大屏响应时延对比结果	75
图 5-1 电磁数据查询集合的数据预览图	78
图 5-2 极坐标图和柱状图组件的实现效果	79
图 5-3 折线图和雷达图组件的实现效果	80
图 5-4 电磁态势可视化分析平台实现	80

表目录

表 1-1 国外部分可视化工具	4
表 1-2 国内部分可视化工具	6
表 2-1 SVG 与 Canvas 的对比	21
表 3-1 Echarts、Highcharts 与 D3 的对比	28
表 3-2 组件的构成元素	42
表 3-3 扩展的组件库包含的组件类型和组件数量	46
表 3-4 兼容性测试结果	48
表 3-5 测试数据集的格式	49
表 3-6 不同组件库的对比	50
表 4-1 大屏构建功能测试结果	68
表 4-2 大屏管理功能测试结果	69
表 4-3 在不同分辨率终端的自适应测试结果	71
表 4-4 不同方案的自适应功能测试结果总结	71
表 5-1 电磁态势四域显示内容	77

第一章 绪论

1.1 研究背景及意义

近年来,大数据等信息技术高速发展,信息技术与IT、金融、科学研究、政府管理和国家安全等领域大量交叉融合,产生了海量多领域、多空间、复杂关联的数据。其中由用户产生的交互信息是数据迅速增长的主要来源。这些数据不仅在数量和复杂度上呈现爆炸式增长,并且在数据来源上呈现多源性,在数据结构上呈现异构性^[1]。

如今,我们逐步走向了“数据即服务(Data-as-a-Service,DaaS)”的时代,复杂数据的采集技术不断更新,对复杂多源异构数据处理及可视化分析是推动社会发展的必要途径^[2]。海量的数据带来了多样的数据处理及分析需求。然而,传统的方法难以对这些原始数据直接处理与分析,信息通常淹没在浩瀚的“数据海洋”中,实现快速挖掘这些庞大数据中的信息,是信息技术领域亟待解决的热门问题^[3]。

在这种背景下,可视化技术飞速发展,可视化将数据映射成视觉可见的图像。研究发现,对比数字,人们更对图形化的结果记忆更深刻^[4]。人类主要通过视觉通道获取外界信息,可视化直观表达了数据代表的含义,促使人们能够快速理解复杂组成的数据,有效获取潜在的重要信息以及发展规律,从而为相关领域的科学研究和决策分析提供关键的参考依据^[5]。

图表是可视化展示的核心要素。不同种类的图表能直观明了展示各个领域的数据。随着浏览器端技术的发展,可视化图表不仅具有可视的功能,还增加了交流与互动的特性^[6]。由于可视化面向用户群体,因此可以在图表的基础上添加丰富的交互功能提升用户体验和简化操作流程,以此增加用户的可操作性和可交互性^[7]。

为了更有效地分析和挖掘复杂组成的数据,可进一步利用大屏对数据进行分析。作为一种综合形式的可视化工具,可视化大屏把来自不同数据集合的数据映射到多种类型的图表上,并将这些图表最终整合在一个界面整体展示。该界面可以在不同尺寸和不同分辨率的屏幕终端上呈现,比如电脑的普通显示屏和大型投影屏等。如今,可视化大屏表现出了超强的可视化能力,已成为各行业各领域不可缺少的核心工具。

通常这些大屏都承载着企业内部相关业务的关键数据^[8],在公共服务方面,如城市交通局可以通过大屏展示停车场的车位分布情况及车流量情况,并结合神经网络算法和人工调度决策,实时推荐附近空余的停车场位;在商业分析方面,电商可以通过大屏展示平台的交易量及各种关键数据,辅助决策分析。在政府管理方面,

电力部门通过大屏在线监控电网实时运行数据,分析潜在的安全隐患,提高态势感知能力,实现保障智能电网的网络安全^[9]。在军事应用方面,海量数据已成为未来战场的重要战略资源^[10],战场可视化是我军在战场信息优势获取必须具备的核心技术。如图 1-1 为可视化大屏在零售行业的实现场景。



图 1-1 可视化大屏在实际应用的实现场景

目前,数据可视化图表和大屏在商业决策分析、公共服务以及政府管理等领域都有非常好的应用前景^[11]。随着数据量的增加,对数据的呈现与分析提出了更迫切的需求^[12],引出了数据可视化的快速构建工具。通过这些工具可以方便快速地将数据转换成视觉敏感的图表在屏幕渲染,也可以进一步构建可视大屏为用户提供决策分析。虽然现在已经有十几种用于可视化设计和开发的创作平台和编程工具^[13],但对于用户来说,快速实现基于 Web 的交互式可视化仍然很麻烦^[14]。

目前,数据可视化在实践应用中还存在较多需要解决的问题,表现在以下方面:

(1) 数据源接口未统一。由于不同来源的数据都有可视化呈现的需求,对多源异构数据分别进行接口适配的方式存在数据管理冗杂和数据链接繁琐等问题,导致数据呈现周期长,极大地降低了数据的使用效率。

(2) 代码复用率不高。由于复杂业务场景的可视化需求多样,如果不利用通用的生成框架和组件库,不仅会增加工作的任务量,导致代码的重用率较低,维护性差。长远来看还会影响整体的开发进度,延长项目的开发周期。

(3) 可视化结果的渲染效果不佳。在大数据时代,基于浏览器端的可视化大

屏需要负载的数据总量可能达到几十万条，这对于系统各方面的性能实现提出了极大的挑战。

(4) 用户交互性待提高。用户交互性可以提升用户对数据的理解和分析效果。现有可视化工具重心在视觉呈现上，如何提升用户的交互性还值得深入研究。

综上所述，随着数据量的骤增，未来可视化的需求是庞大且多样的。传统的数据可视化平台往往针对于特定的业务场景构建大屏，不仅设计图表的流程复杂，并且展现的效果也不理想，具有通用性不足、扩展性较低和用户交互性差等问题，已不能满足未来人们日益增长的可视化需求。面向大屏实现操作流程简便，图表渲染效果好的可视化图表展示平台已成为目前可视化领域的重点研究方向。因此，本文面向异构数据的可视化进行研究，提出了通用的可视化图表生成方法和高效的敏捷构建大屏的方法，能够改进现有的可视化平台的不足，具有重要的研究意义和较大的实用价值。

1.2 国内外研究现状

可视化应用的发展源远流长，早在计算机出现之前，其在信息传递、数据分析和数字图像化等方面已有了可观的发展。在早期，一些地理学家们提出运用图形化的表达方式，将地理自然信息数据用等高线、等温线、等压线、等势线等系列的等值线图 and 图示符号表示，以此发现自然界中各种现象之间的关联。在 17 世纪之前就诞生了人类历史的第一幅城市交通图。而在 18 世纪则诞生了现代的第一张图表，这来自 W.Playfair 撰写的《商业和政治图解》的贡献^[15]。

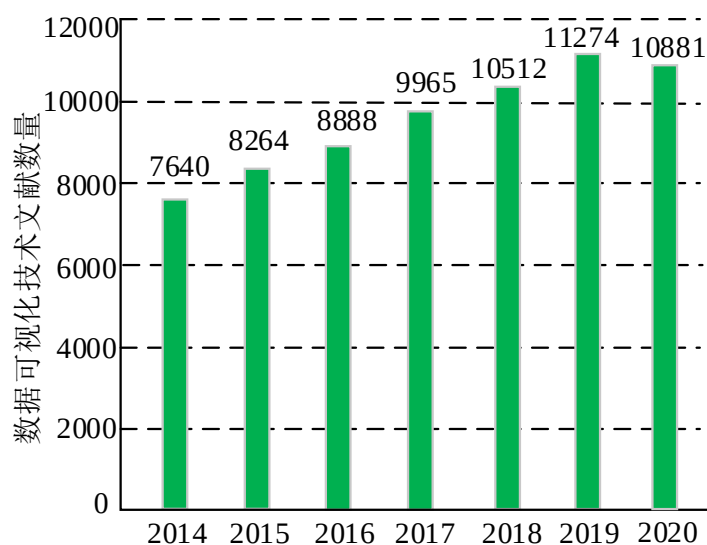


图 1-2 历年来可视化文献发表数量

可视化理念与技术的发展见证了科学与工程制图、统计学科等的不断进步。在 20 世纪,使用统计图、表格等可视化的方式在数据分析中已被广泛使用^[16]。随着几何统计学的兴起,人们开始广泛使用图表展示数据,这一方式不仅改变了科学发展的趋势,也使得人类的认知能力有了质的飞跃。计算机技术快速迭代带来了现代数据可视化的进步。进入 21 世纪以后,信息量与日俱增^[17]。从 2010 年以来,数据应用的广度与深度不断快速拓展,现有的技术已难以满足可视化发展的需求。人们开始深入探索与可视化相关领域的背景知识,融合各个学科的技术知识,寻求快速展示和挖掘海量异构数据的方式。几年来,可视化成果屡创新高,如图 1-2 为近年来数据可视化发表文献的数量(数据统计于 Web of Science)。

1. 国外研究现状

在国外,对现代可视化技术的启蒙研究较早。90 年代,IEEE 举办了第一届 IEEE Visualization Conference (IEEE 可视化会议),会议上召集了各个学科领域的高端研究人员,旨在共同从事可视化的科学研究^[18]。他们把可视化划分成三个方向^[19],一是 IEEE Scientific Visualization Conference (IEEE Sci Vis),即科学可视化,重点研究各个自然科学领域中的可视化^[20]。二是 IEEE Information Visualization Conference (IEEE Info Vis),即信息可视化,重点研究抽象数据的可视化^[21];三是 IEEE Conference on Visual Analytics Science and Technology (IEEE VAST),即可视化分析学,重点研究可视化分析在科研、工业和医疗等领域的应用;

在国外,如 Microsoft、Tableau 等著名企业推出了很多成熟的商用可视化工具,表 1-1 是国外部分可视化工具介绍。

表 1-1 国外部分可视化工具

名称	开发者/组织	介绍	面向领域
Power BI ^[22]	Microsoft	由 Power Query、Power Pivot、Power View 及 Power Map 构成的商业智能分析工具	商业智能
Tableau ^[23]	Tableau	交互式的可视化分析工具	商业智能
SPSS ^[24]	IBM	专业的自助式数据分析工具	商业智能
Openlayer ^[25]	MetaCarta	基于 WebGIS 的地图可视化工具	地理信息
Gephi ^[26]	Gephi	构建动态分层数据图表的可视化网络平台	图网络

除这些工具外,IBM 的 Cognos、SAP 的 BO 等面向商业智能分析领域,也推出了强大的可视化工具。但是,由于国外的可视化技术相对国内更加成熟,因此大多数平台工具需要收费购买。对于用户来说,还需要提前熟悉数据处理与可视化的基础知识,上手较难,花费成本较大。

2. 国内研究现状

国外可视化的研究较早，因此国内可视化技术的相比与欧美等国家还存在较大的距离^[27]。国内逐步引入可视化技术是从上世纪 90 年代开始，当时我国科研数据开始存在可视化的需要。后来，随着可视化需求的增加，数据可视化逐渐在市场扮演重要的角色，科研学者也慢慢将研究重点转移到可视化领域中。在现阶段国内存在两个顶尖的可视化研究团队：一是以北京大学袁晓如教授领头的可视化分析团队，另一个则是浙江大学的以陈为教授为代表的 CAD&CG 实验室^[28]。

特别是近几年，我国在可视化领域开展了行之有效的研究，取得了令人骄傲的科技成果。在 2020 年，袁晓如团队基于可视化研究的两项成果被 ACM CHI 国际学术会议收录，其中一项成果是面向可视化智能分析，提出了一种自动对可视化图表进行高亮和注释的方案^[29]。当使用者上传可视化图表及其对应的文字描述后，便可以自动对图表生成智能化分析后的高亮和注释。该方法运用深度学习神经网络模型和图像检测技术提取图表中的实体和对应的视觉属性，然后基于自然语言处理技术，实现语义信息与视觉的匹配，最后在图表上生成相应的高亮和注释的动画。另外一项成果是面向层次机构化的数据，提出了一种面向树可视化的描述性语言-GOTree^[30]。GOTree 将树的可视化构建划分为坐标系，视觉元素及布局，并将布局递归地解构为基本单元，生成相应视觉符号和属性，从而构建多种形式的树可视化结果。

陈为所在可视化团队面向可视化分析领域，发表了数十篇顶级期刊学术论文，其在文献^[31]中提出了一种新的图布局算法 DRGraph，该算法基于负采样等融合方案来实现非线性降维过程的加强。DRGraph 算法在生成布局过程中的计算复杂度和内存消耗量呈线性变化，即使在节点数很大的规模图也得到了良好的应用。与同类最先进的算法相比，DRGraph 具有更低的运行时间和更低的内存消耗。在另一篇文献^[32]中，该团队提出了一种分析文本与时空参照关联数据的方法，以了解文本语义在空间和时间上如何变化。

其他学者在多维复杂数据可视化方面也展开了富有成效的探索研究，文献^[33]提出了一面向海量超声数据的可视化方法，基于混合框架采集超声数据，应用多媒体和硬件资源处理这些超声数据，然后将结果存储在线传输的环境中实现数据共享，最后再基于核外算法实现超声数据的可视化。张杰等人提出了一种融合网络安全数据与可视化分析的方案^[34]，基于数据存在的相关性，先消除冗余的数据，再进行归一化处理，最后分析是否存在有威潜在威胁的入侵动作。

与此同时，我国在国际学术界的影响力也日益提升。香港科技大学的可视化实验室 VISLab 已成为全球前沿的可视化研究所，并在近年来发表了数量可观的可视

化领域顶级期刊。而领军人物屈华民教授的研究成果更是被全球知名企业如 IBM、腾讯微信等纳用^[35]。

此外,国内的跨国公司及民族企业也在大力调整在可视化领域的发展布局,取得了令人鼓舞的研究成果。如 IBM 中国研究院在新兴发展的文本可视化学科投入了大量研究。阿里巴巴建立了独立的可视化部门,并针对在线数据可视化开发了相应的组件库。

总体来说,近年来我国在可视化领域的研究不断发展进步,面对可视化分析学这一新学科,国内的学者和企业单位也在不断开展相关研究。一方面国内的研究基础在逐步扩大,另一方面在可视化领域的国际影响也不断提高。

目前,国内在数据可视化方面开发的工具部分介绍如下表 1-2 所示:

表 1-2 国内部分可视化工具

名称	开发者/组织	介绍	面向领域
DataV	阿里云	基于 Web 的数据可视化平台	商业智能
FineBI	帆软	自助式大数据分析工具	商业智能
腾讯云图	腾讯	一站式的数据可视化展示平台	商业智能
网易易数	网易	数据分析与可视化工具	商业智能
ZTMAP	上海臻图	空间信息三维可视化工具	地理信息

通过研究发现,国内的数据可视化平台如 DataV 等,多数基于 Web 端技术,可实现跨平台共享,且对硬件环境的性能要求不高,收费也低于国外的平台。但是与国外类似的平台相比较,国内这些平台的数据处理和分析能力还比较弱。并且由于基于 Web 端实现,在大数据渲染方面的性能不理想。

1.3 本文研究内容与结构安排

本文根据现有组件生成效率低和大屏构建不敏捷的问题,提出了基于模型的组件可重构技术和基于页面自适应的大屏敏捷构建技术,并在此基础上面向多源异构可视化的需求,使用本文开发的工具实现复杂案例验证。

本文分为六个章节,各章节的安排如下:

第一章首先介绍了异构数据可视化的研究背景及研究意义。其次介绍了数据可视化的发展历程及相关技术和工具平台的研究状况。最后说明本文的结构安排。

第二章中,介绍了数据可视化的元素组成、分类以及异构数据可视化的流程,然后分别对可视化流程中涉及的关键技术作了简明介绍。

第三章中,首先分析了传统组件库组件生成的流程。接着,提出了一种基于模

型的组件可重构生成技术，在构建组件生成框架上集成可视化模型，可以快速生成组件并且对组件进行重构配置。本文提出的框架可以集成多种类型的组件库，并且生成组件的方式更灵活。最后，对构建的组件库进行了功能测试与性能测试，并与其他主流的组件库进行了对比分析。

第四章中，首先分析了传统构建大屏方法的优缺点。接着，提出了一种基于页面自适应的大屏敏捷构建技术，在研究大屏动态构建机制上，以可视化组件作为大屏的组成元素，实现大屏页面的自适应展现，并且开发了构建大屏的通用工具。最后，对使用本工具构建的大屏分别进行了功能测试和性能测试实验，测试结果显示，构建的大屏具有较好的易用性与交互性，满足实际应用的可视化需求。

第五章是基于以上研究成果，面向多源异构数据可视化的需求，实现了多源异构数据的可视化案例研究。基于本文实现的工具可以快速搭建联合作战电磁态势数据可视化分析平台，对关键指标数据进行可视化展示与分析。案例结果证明本文的研究成果对于不同领域的数据可视化分析具有较高的实际应用价值。

最后一章是对全文工作的总结与展望。

第二章 多源异构数据可视化及其关键技术概述

本章主要介绍多源异构数据可视化的相关概念与涉及的关键技术。包括数据可视化的元素组成、分类以及多源异构数据可视化的流程。

2.1 多源异构数据可视化相关理论

根据数据可视化在不同领域的应用情况,不同的研究学者对其有不同的理解。Stacey 等人认为数据可视化是将数据值映射为图形的可量化特征,不仅要准确地传达数据,还要美观地展现数据^[36]。

Mitchell 等人提出数据可视化涉及统计学、数据科学、图形学等众多学科的技术,最终目标是实现对数据的交互式探索发现,以帮助人们更好地理解数据^[37]。陈为等人则认为数据可视化通过借助人的视觉感知能力,实现对外界数据的交互式可视展现,最终来增强大脑的认知能力^[38]。

2.1.1 数据可视化概述

根据不同角度,数据可视化有多种分类方式。按照学科来划分,分为以下三种^[39]: (1) 科学可视化,主要是对自然科学领域的数据进行可视化处理。(2) 信息可视化,主要面向非结构化的抽象数据进行处理。(3) 可视化分析学,融合可视化与分析形成的新兴方向。

按照应用领域划分,数据可视化又可分为战场数据可视化^[40]、地理数据可视化^[41]、生命科学数据可视化^[42]、网络与系统安全可视化^[43]、商业智能可视化^[44]、金融数据可视化^[45]等。

数据可视化是基于预先设定的框架,根据不同数据类型的特点,匹配合适的图表来简洁准确地表现数据的意义。如图 2-1 所示是数据可视化的基本元素组成。

数据: 数据是具有实际意义的数值,根据测量标度,数据可分为有序或无序数据等、连续或离散型数据等。

全局: 全局包括多个信息维度,每个信息维度含有相应的要素。图例通常位于图表一侧,以特定颜色的符号,解释图表上的对应颜色线条描述的内容。图表的文字包含主副标题,还包括图例和提示框的文字及坐标轴上的刻度指标等。视觉映射是数据的值到视觉表现属性的体现。

图形: 由标记(点、线、形状)和视觉特征(颜色)构成。数据通过映射转化为直观显示的图形元素。

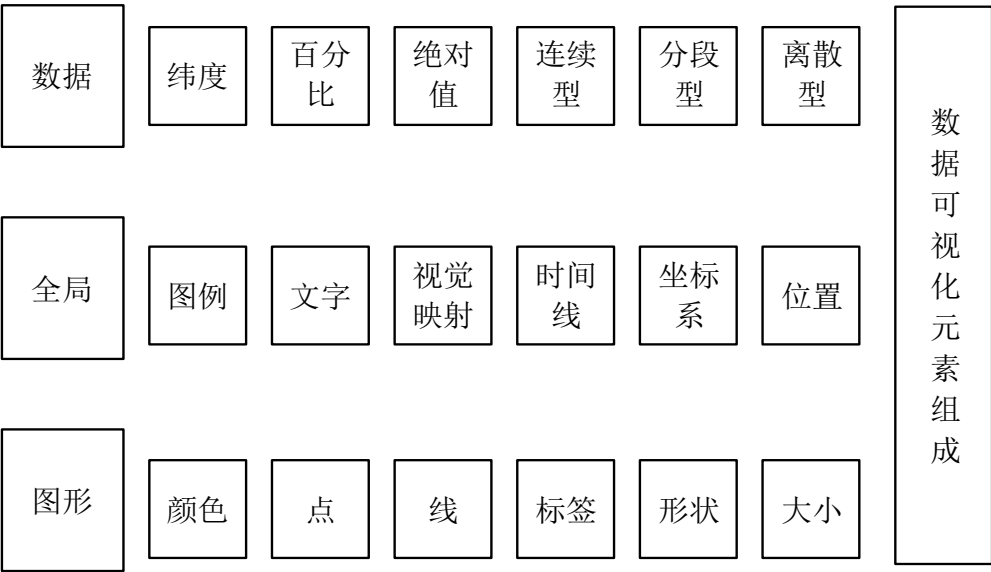


图 2-1 数据可视化的元素构成

2.1.2 数据可视化的流程

整体来看，数据可视化实现数据与图形的映射。原始数据首先经过数据处理模块，接着被转换成几何数据，最后渲染成视觉可见的图像数据，即图形。完整的可视化流程，不仅包括视觉映射，还包括以数据流为导向的数据处理过程，人们通过交互的方式从处理后的结果获取有价值的信息。

为此，如图 2-2 所示，科学可视化建立了数据可视化流程的参考模型^[46]，并提出了由数据处理和图形绘制融合构成的可视化流水线。

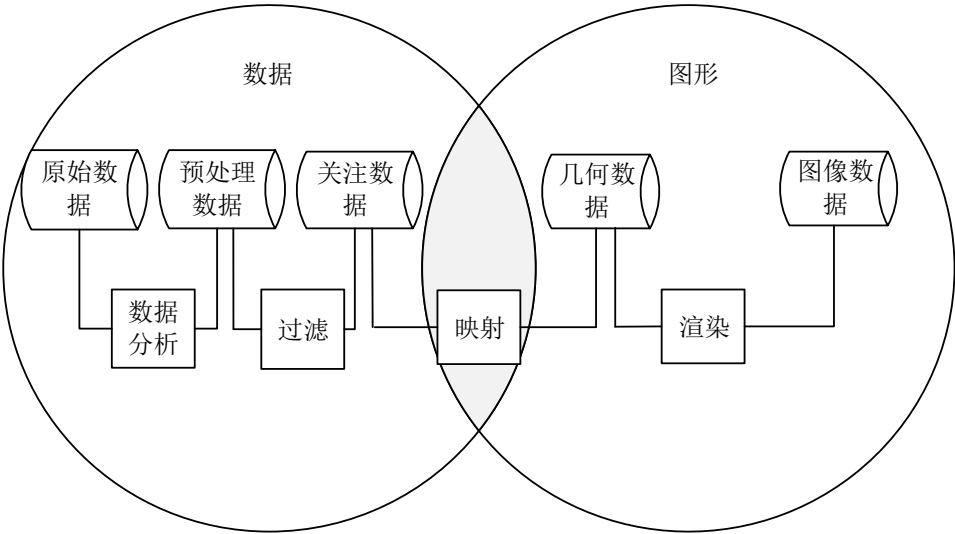


图 2-2 数据与图形空间的映射

图 2-2 展示了从抽象数据空间到直观图形空间的映射过程，主要包括了数据处理、数据映射和图形渲染。

类似的，信息可视化也提出了可视化流程的体系架构模型^[47]，如图 2-3 所示。但是该模型将上述流水线改成环路形式，并且用户可以在任何节点进行交互操作。流程的起始点是数据，采集到原始数据后，首先需要对数据进行清洗和精简。接着，过滤后的数据进入可视化项目库中等待转换，在此阶段中需要根据数据的特征选择合适的图表进行视觉可视化映射，并借助人的视觉认知能力使数据进行形状、颜色、大小和布局等形式上的视觉凸显，在引起用户视觉注意的同时避免让用户造成认知负荷。最后，绘制的可视化图表以交互的形式展示，用户既可以根据需求返回起始点添加、删减或修改数据，也可以通过可交互的组件来主观调整可视化图表的样式。

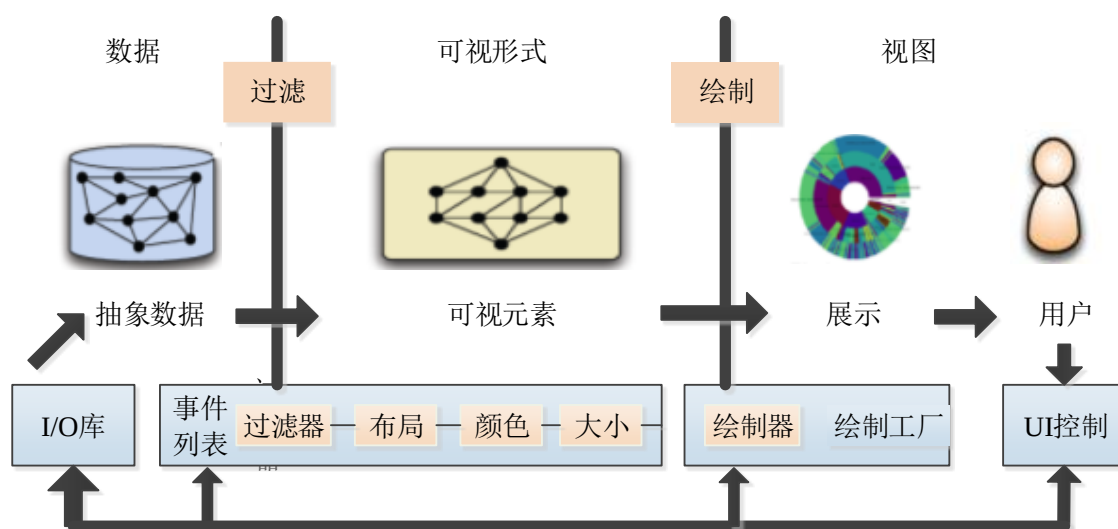


图 2-3 信息可视化流程

在大数据时代，随着信息系统的发展，各领域产生了许多多源异构数据。随着复杂异构数据的采集日益普及，对异构数据的大屏可视化需求也越来越多。经过分析，面向大屏的多源异构数据可视化涉及的关键技术包括了数据采集、数据处理、数据整合、组件生成、大屏展现等技术，各关键技术将在下面的小节分别介绍。

如图 2-4 所示为多源异构数据可视化的具体流程，首先获取存在的多源异构数据，并进行初步的数据预处理，之后将这些数据整合到同一数据集中的等待后续的可视化操作。当面临可视化需求时，再将这些数据从数据集中查询出来，作为组件的数据来源进行可视化组件的设计。最后基于生成的可视化组件，构造可视化大屏进行数据展现与分析。

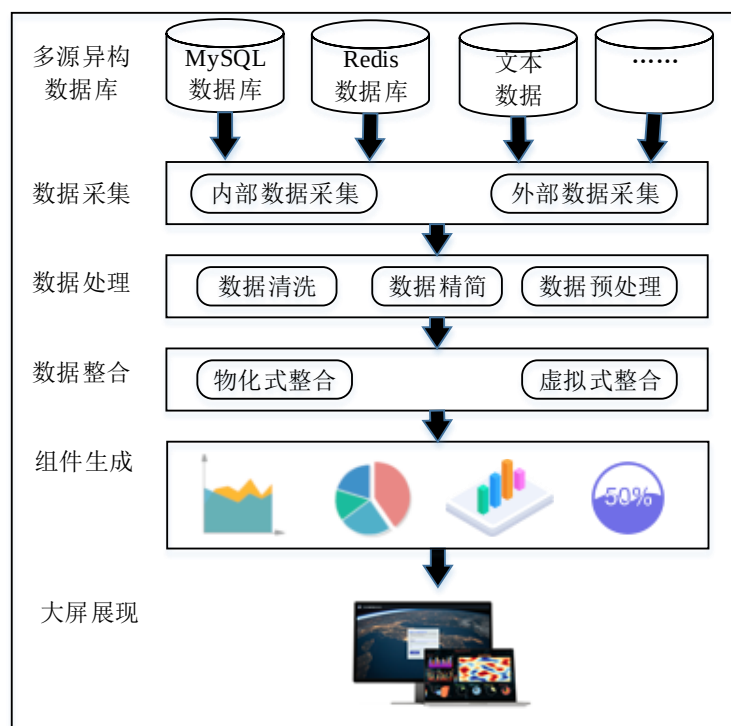


图 2-4 多源异构数据可视化流程

2.2 数据采集技术

数据采集是可视化的第一步，数据采集的方式和质量，对可视化的最终效果有着至关重要的作用。在大数据时代，数据变得廉价，采集数据的方式也层出不穷。通常来说，数据采集的方式有实验仿真、实际测量和在线数据传输等，并且传统的数据采集以文件输入/输出方式为主。在万物互联的时代，以网络为介质传输的多源数据占据主要采集方式，数据采集面临的难点相应有多源异构和数据格式变换等。从数据的来源划分，数据采集方式可分为内部数据采集和外部数据采集。

2.2.1 内部数据采集

企业通常将内部产生的业务数据存储在指定的数据库中，当需要对这些数据进行处理时，直接从这些数据库采集数据。如果要进一步分析用户的行为数据等，还需要采集部分行为日志数据。

（1）数据库实时数据采集

目前，Java 应用程序与数据库通信的方式在各应用中越来越广泛。在该框架中，一般用 JDBC 作为两者通信的中间接口。JDBC 是基于 Java 的 API（Application Programming Interface，应用程序接口），负责为不同数据库分别提供访问接口。如图 2-5 为 JDBC 访问数据库的接口规范。

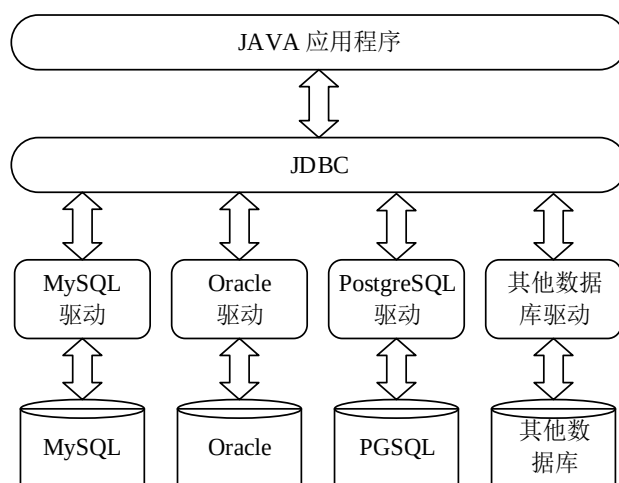


图 2-5 JDBC 访问数据库的接口规范

利用 JDBC 可以实现统一的接口规范并创建与不同数据库（例如 SQL Server、Oracle 等）的连接。区别于为不同数据库单独开发数据读取程序的方式，JDBC 使用统一的 SQL 规范连接并读取数据。

此外，在 Python 应用程序中，采用 Python DB-API 作为与数据库通信的中间接口。如图 2-6 为 DB-API 访问数据库的规范。

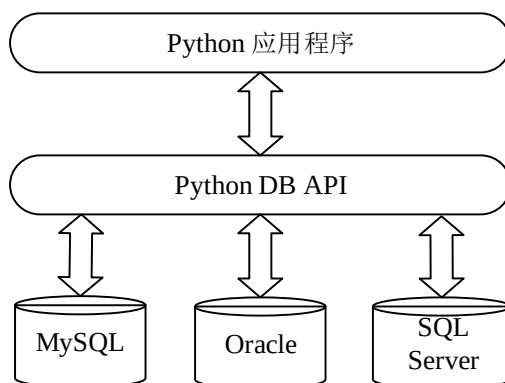


图 2-6 Python 访问数据库的接口规范

与 JDBC 类似，DB-API 通过标准的接口来处理关系类型的数据库。基于 DB-API 可以实现与不同数据库的交互。不同点在于，DB-API 在应用程序与驱动程序中间创建了单独的抽象层。

使用 DB-API 查询数据的具体流程如下图 2-7 所示。简单地说，访问数据库流程为：首选定义连接属性，接着建立服务器与数据库的连接，然后发起查询请求，最后数据库执行相应的动作后，返回处理结果。

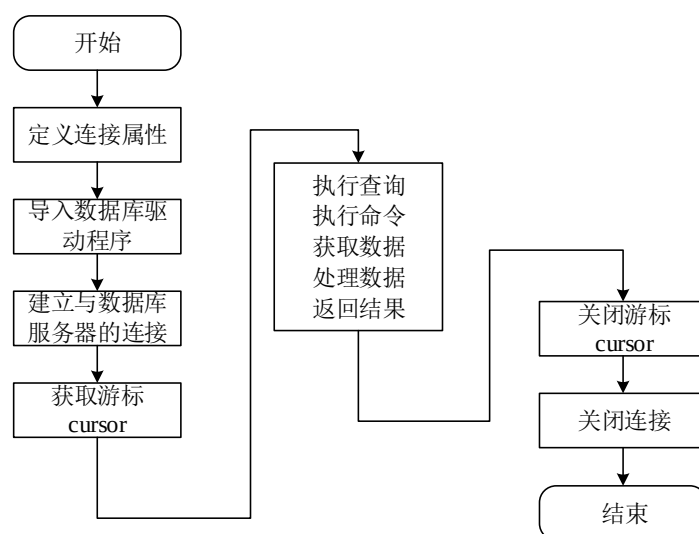


图 2-7 Python 查询数据库的流程

(2) 系统日志数据采集

系统日志存储着有关于系统的各种问题信息，通过它可以排查系统的错误故障，或追溯攻击者产生的痕迹。系统日志又划分为记录系统安全信息的日志和记录应用程序崩溃信息的日志。当前，大数据平台每天会生成海量的日志，采集这些日志信息已成为当前的热门研究领域。目前主流的日志采集系统有基于 Facebook 的 Scribe 和 Hadoop 的 Chukwa 等。以 Scribe 为例，此类系统的采集速度达到了数百 MB 每秒，已经满足目前对传输速度的需求。

2.2.2 外部数据采集

外部数据采集指通过一些方法获取企业外部的数据，主要包括网络的数据。网络数据又可以通过两种方式采集：一种是基于应用程序接口 API 采集，另一种基于网络爬虫技术采集。

(1) 通过 API 采集

API 是网站应用程序接口，通过该接口能直接实现对数据的查询服务。目前国外的 Instagram 和国内的新浪微博等社交平台都提供了 API 功能，但是使用 API 技术采集数据的缺点是每天调用接口的次数有限制。

(2) 通过网络爬虫采集

通过网络爬虫采集数据的方式是遵照制定的规范流程，自动化搜索获取网络在线信息的脚本。最常见的爬虫是人们每天使用的搜索引擎，如百度、谷歌搜索等。由于是无条件采集全部网页上的信息，因此被称作通用型爬虫。该类爬虫技术的具体工作原理如图 2-8 所示。

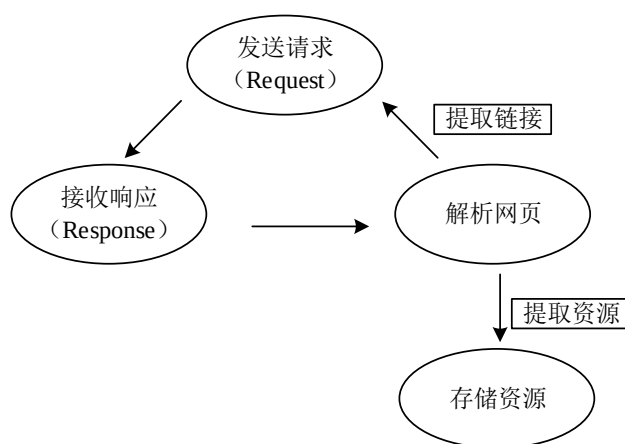


图 2-8 通用型爬虫的工作原理

通用型网络爬虫的工作流程是：首先给爬虫一个初始网站的 URL，爬虫会向网页发送请求，接收响应并解析该网页和提取需要的资源进行保存，同时还会提取出该网页中存在的其他 URL 链接。接着爬虫会向提取的 URL 发送请求，再通过响应和解析页面，提取出网页中的资源，并重复此过程。随着采集需求的增多，相继产生了主题爬虫和多线程爬虫。主题爬虫不再无条件搜索信息，而是按照一定的规则先过滤无关的信息，只留下目标采集的数据，因此可以大大降低无效数据带来的数据稀疏问题，提高数据采集质量。多线程爬虫是通过开启多个线程，每个线程同时执行采集任务来提升数据采集的效率。

2.3 数据处理技术

2.3.1 数据清洗

数据采集后，首先需要考虑数据的质量问题。常见的质量问题包括数据中存在噪声冲突、存在数值异常、存在数值缺失和存在数值重复等，通过数据清洗可以解决这些问题。按照数据质量问题的归类，常用的数据清洗方法有以下几种：

1) 噪声数据

面向噪声冲突的清洗原理是基于完整性约束，对原始数据集合生成依赖函数，再基于生成的函数清洗数据。按生成方法分类，依赖函数有条件函数依赖^[48]、差别依赖^[49]等。此外，还可以应用小波去噪，平滑滤波等方法对噪声数据进行清洗。

如定义一个关系模式 R 如下：

$$R = (ID, A_1, A_2, \dots, A_i, \dots, A_n) \quad (2-1)$$

其中，ID 为实体标识符， $A_1, A_2, \dots, A_i, \dots, A_n$ 代表 R 的属性。定义 $\text{attr}(R)$ 为 R

的属性集合。对属性 A ，假设 $A \subseteq \text{att}(R)$ ，对属性 B ，假设 $B \subseteq \text{att}(R)$ 。

如果 $\{A \rightarrow B\}$ 是一个标准依赖函数，则定义在 R 上的条件函数依赖 f 如下：

$$f : (A \rightarrow B, T_p) \quad (2-2)$$

其中 T_p 定义了属性 A 和属性 B 的约束条件。因此按照约束值，条件函数依赖又可分为常量条件函数依赖和变量条件函数依赖。

2) 异常数据

面向异常数据的检测方法有基于整体错误的检测^[50]、基于密度的异常检测^[51]和基于属性值组合的异常检测^[52]等。基于密度的异常检测实现原理如下：

通常异常数据点处于低密度区域，且相对远离近邻点。因此可以定义元素 x 的密度为 1 与至 k 个邻近点的平均距离的比值：

$$\text{density}(x, k) = \left(\frac{\sum_{y \in N(x, k)} \text{distance}(x, y)}{|N(x, k)|} \right)^{-1} \quad (2-3)$$

其中， N 代表包含 x 的 k 个邻近点的集合， $|N|$ 代表集合的大小， y 属于集合 N 。 $\text{distance}(x, y)$ 定义如下：

$$\text{distance}(x, y) = \max \{ \text{distance}(y), d(x, y) \} \quad (2-4)$$

3) 缺失数值

对缺失数值的检测一般较为简单，因此可以在判断有缺失值的条件下，自动填补缺失值^[53]。常用的填补方法有插补法和插值法。其中，插值法的原理是根据已有样本点构建插值函数多项式，通过求解该多项式的值来填补缺失值。

对于埃尔米特 (Hermite) 插值法，建立 n 个点 $(x_1, x_2, \dots, x_n)$ 的插值多项式：

$$H(x) = \sum_{i=1}^n h_i [(x_i - x)(2a_i y_i - y_i') + y_i] \quad (2-5)$$

其中，各项的值如下：

$$y_i = y(x_i) \quad (2-6)$$

$$y_i' = y'(x_i) \quad (2-7)$$

$$h_i = \prod_{\substack{j=1 \\ j \neq i}}^n \left(\frac{x - x_j}{x_i - x_j} \right)^2 \quad (2-8)$$

$$a_i = \sum_{\substack{j=1 \\ j \neq i}}^n \frac{1}{x_i - x_j} \quad (2-9)$$

3) 冗余数值

对冗余数值进行检测的步骤如下：首先对数据进行分组，如可以采用聚类的方法，提高检测的效率。接着将数据对比，计算数据间的相似度。最后进行冗余判断，检测是否存在冗余数据，这一步可以基于相似度函数进行判断。

2.3.2 数据精简

如果直接对高维复杂的海量数据集可视化处理，会产生参差不齐的结果，这种情况也被称为高维度导致的视觉混乱。因此需要用到数据精简降低这些复杂高维数据的复杂程度，以实现在有限的图形空间内表达。数据精简虽然减少了数据点的数量，但是保留了其本来特征，因此可以大大降低数据查询和处理的花费开销，降低响应时间。常用的数据精简方法有合并、采样、降维等，还有基于数据特征变换数据的方法，比如离散小波变换等。

1) 合并：将多个对象整合为一个对象，可以达到降低数据复杂度、改变数据尺度及减少数据的方差的作用。

2) 采样：在数据总体较大的情况下，随机选取整体对象的部分，以采样样本的研究规律作为整体的特征规律。采样分为回置采样和不回置采样，采样也可将数据集总体划分为多个集合，再在子集合中进行随机采样。

3) 降维：降维可以消除一些无关特征和噪声的影响，从而减少整体处理时间与资源消耗量，提高可视化数据的利用率。常规的降维方法有：奇异值分解、主元分析和非线性降维 ISOMAP 等方法。

2.3.3 其他预处理操作

其他预处理步骤还有属性变换、特征生成等。

1) 属性变换：按照一定的转换规则，将数据属性的值映射为响应维度的值，以得到新的属性值。常见的有绝对值变换、指数变换等。

2) 特征生成：基于原始数据集的数据属性经过算法产生新的属性，以补充额外的重要信息。常用的特征生成有特征构造和特征抽取等。

3) 离散化与二值化：将原始数据集离散化表达，重新分为多个子类。常见的方法有将数据值映射为到指定的区间内，如归一化映射到[0,1]区间。

2.4 异构数据整合技术

当需要对采集、清洗、精简和变换后的多源异构数据的集合进行处理时，通常需要借助数据整合方法。数据整合将这些多源数据集成于一个新的集合内，并为用户提供该数据集合的统一访问和数据预览视图。常用的数据整合方式有物化式整

合和虚拟式整合。在下面对两种整合方式作简单介绍。

2.4.1 基于物化式的数据整合

如图 2-9 是物化式整合的示意图。在数据查询前，涉及的数据块被存储到同一个物理位置（如数据仓库），这种整合方式需要将数据从源数据库物理移动到其他位置。

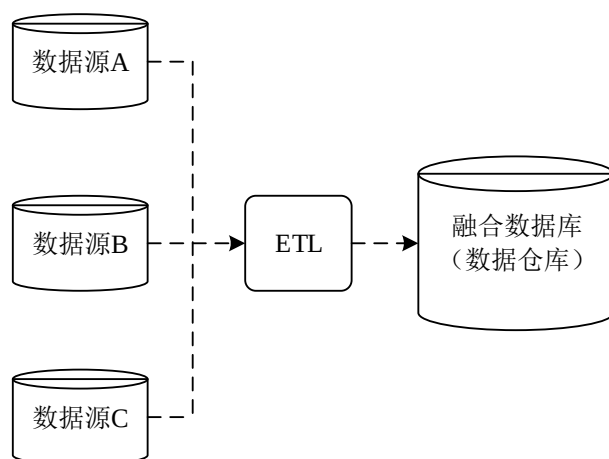


图 2-9 物化式整合原理

2.4.2 基于虚拟式的数据整合

如图 2-10 是虚拟式整合的示意图。与物化式整合不同，不需要移动数据，只需利用数据转换策略，并单独构建一个虚拟层，用以提供一致的数据访问接口。这种整合方式通常借助中间件技术，利用中间件提供的虚拟数据层，定义数据的映射关系。此外，虚拟层将不同来源的数据在语义层面进行融合，使得在数据查询时达到语义一致。

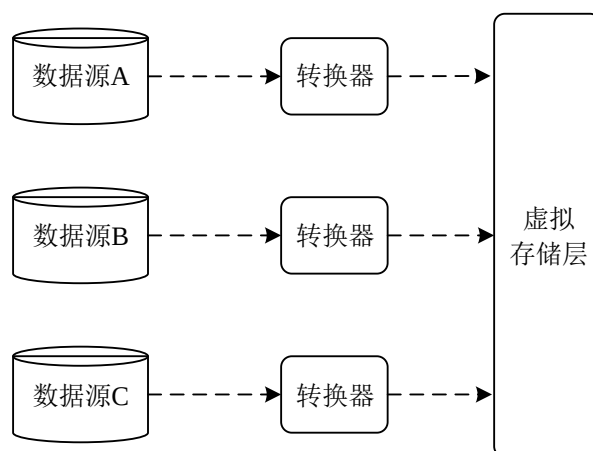
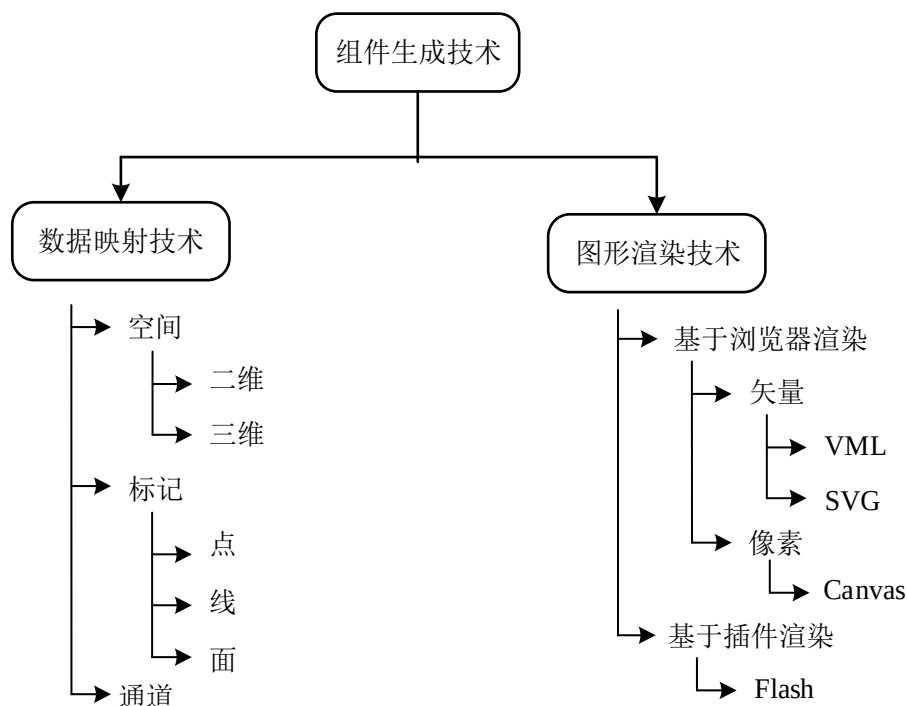


图 2-10 虚拟式整合原理

2.5 组件生成技术

组件是指在屏幕显示的可视化元素，基于浏览器端的组件生成的具体实现流程是：数据先通过一定的编码原则映射成可视化图形，然后在浏览器端通过图形渲染技术呈现，如图 2-11 为组件生成技术的架构组成。



2.5.1 数据映射技术

数据映射实现将上述处理后的数据信息，转化为可视化元素，因此也被称为可视化编码（Visual Encoding）。可视化编码元素包括空间、标记、视觉通道。可视化映射的实现，就是不同编码元素的组成并以可视化的形式准确地将数据信息表达的过程^[58]。

（1）可视化空间

数据可视化结果一般以二维为显示空间。基于图形渲染技术，可以解决三维可视化显示的问题。

（2）标记

标记，是数据属性映射到图形元素的具体表现形式，可以表征数据的性质归类。在可视化中，标记通常代表着几何图形元素，如点、线、面、体等，其空间自由度分别为零、一维、二维和三维。根据标记的分类性质，可用不同的标记编码不同的数据属性。如图 2-12 所示，基于点、线、面、体标记，可以分别映射为散点图、

线性图、面积图和三维立方体。

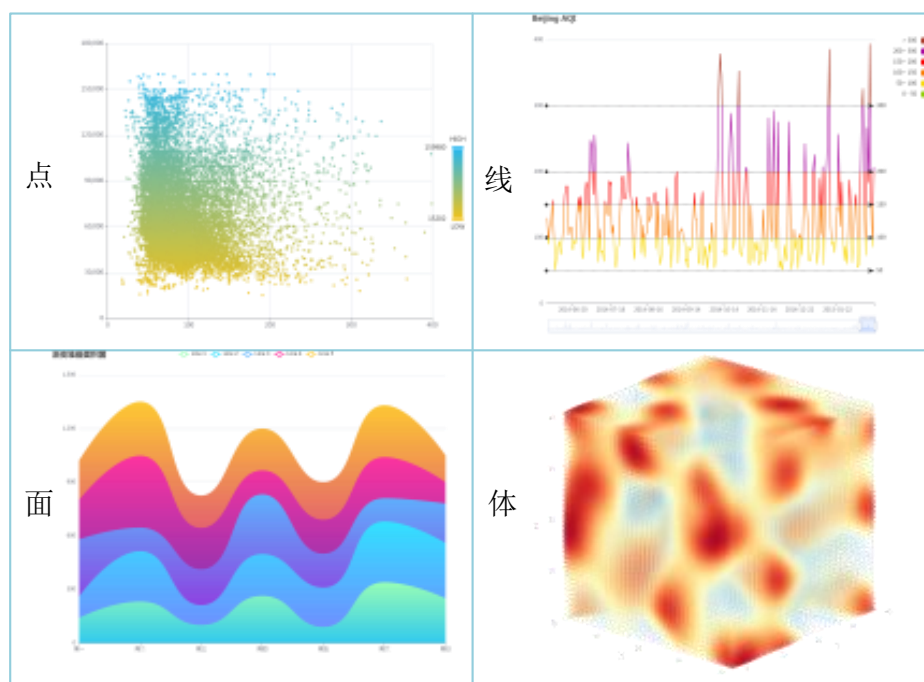


图 2-12 标记类型示例

(3) 视觉通道

视觉通道，则是属性值映射到视觉表现属性的具体量化。视觉通道用于控制标记的展现特征，从而可以定量地描述标记在图形中的呈现状态。基于视觉通道的分类特征，可以将数据编码为多种数据属性（如形状）。基于视觉通道的定量特征，可以编码一个属性的不同值（如长度）。如图 2-13 所示，视觉通道具体分为：位置（一维位置、平面位置）、大小、形状、颜色等。

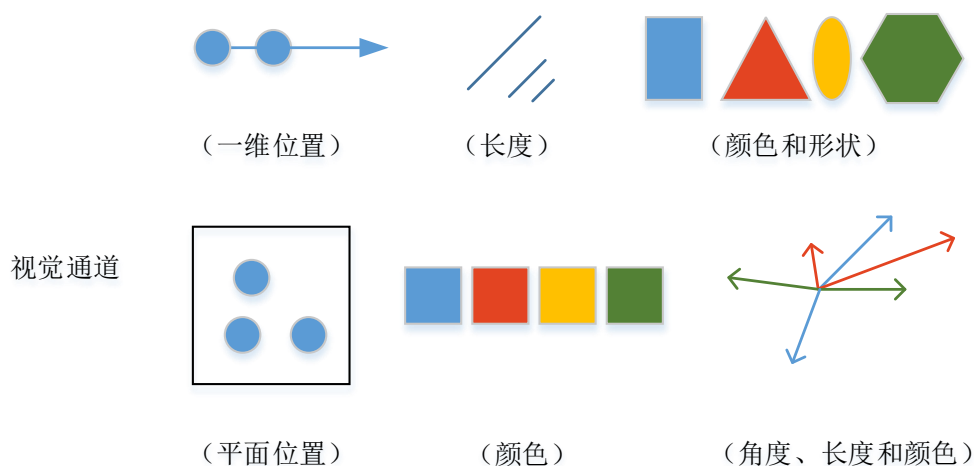


图 2-13 视觉通道

2.5.2 图形渲染技术

当数据映射成可视化元素后,通常需要借助浏览器端的图表渲染技术,将生成的组件展示出来。随着浏览器端的可视化需求增多,相应的渲染技术也快速发展。

从底层实现机制来看,这些绘图技术大体可分为两种:一种是基于插件形式,如 Microsoft SilverLight、Applet 和 Flash 等绘图技术,这类技术通过安装插件的形式运行,优点是兼容性好,缺点是需要依赖执行环境,因此面临承担浏览器禁用插件的风险。另一种是基于浏览器原生支持,如早期的 VML (Vector Markup Language, 矢量可标记语言) 和现在流行的 Canvas 和 SVG (Scalable Vector Graphics, 可扩展矢量图) 等绘图技术,这种方式的优点是不需要安装插件执行,但缺点是各浏览器的支持程度不同。

由于目前基于 Web 端的可视化,主要应用了基于浏览器端原生的渲染技术,下面重点对其进行介绍。基于浏览器端的图形渲染技术从种类上可继续分为两类:一类是基于矢量图形绘制,另外一类是基于像素绘制。

(1) 基于矢量图形绘制

矢量图像,也被称为面向对象的图像,其原理是基于图像存在几何信息实现绘制。矢量图像中的元素被看做独立存在的对象,具有特定的属性。在改变矢量图像的大小,可以维持每个对象本来的清晰度和弯曲度,因此具有高度保真的特点。基于矢量图形的主流绘图技术是 VML 和 SVG。

VML 是微软提出的矢量图形规范,采用 XML (Extensible Markup Language, 可扩展标记语言) 语法,能在浏览器端页面的绘制矢量图形。虽然 VML 支持高质量的图形显示,但是其仅在 IE 浏览器中兼容性较好,因此后来出现了 SVG 技术。

SVG 是 W3C 组织制定的矢量绘图语言。同样基于 XML, SVG 支持嵌入式外部对象(如 JPEG)和矢量图形对象(如曲线)以及文本对象等。此外, SVG 还具有文件体积小、支持无损放大缩小图像等优点,但是与 IE 浏览器的兼容性较差。

(2) 基于像素的图形绘制

基于像素的绘制原理是,把每个数据属性值一一映射到空间内的某个像素点上,尽可能展示更多的可视化数据对象。通过像素点的排列,表现数据中存在的模式。近年来,基于像素的可视化技术在许多领域中得到了广泛的应用,在这其中, Canvas 是最出名的图形绘制技术。Canvas 由 W3C 推出,已得到主流浏览器的支持,它通过 JavaScript 绘制图形,具有可直接保存为图像、节省资源的优点,缺点是放大缩小很麻烦,不支持单独的文字绘制。随着现在开发工具的支持,目前 SVG 和 Canvas 已成为主流的图形渲染技术,其对比如表 2-1 所示。

表 2-1 SVG 与 Canvas 的对比

SVG	Canvas
矢量图	位图
不依赖分辨率	依赖分辨率
逐元素（DOM）进行渲染	逐像素进行渲染
基于 XML，可以用文本格式描述图像	基于 JavaScript 绘制图像
支持事件处理器	不支持事件处理器
适合大型渲染区域	文本渲染能力弱
复杂度高	复杂度低
被绘制的图形被视为对象，如果对象的属性改变，浏览器能够自动重绘图形	当图形被绘制完成后，如果发生变化，需要重新绘制场景。

通过分析，Canvas 与 SVG 绘图方式的区别，是 Canvas 采用不断调用 API 的方式渲染绘图，而后者是基于 XML 描述进行绘图。

2.6 大屏展现技术

可视化平台将不同数据库接口的数据，经过一系列可视化模块的处理，最终形成直观明了的可视化大屏。其中，大屏展现技术又包括了大屏构建技术和大屏交互技术，其架构组成如图 2-14 所示。

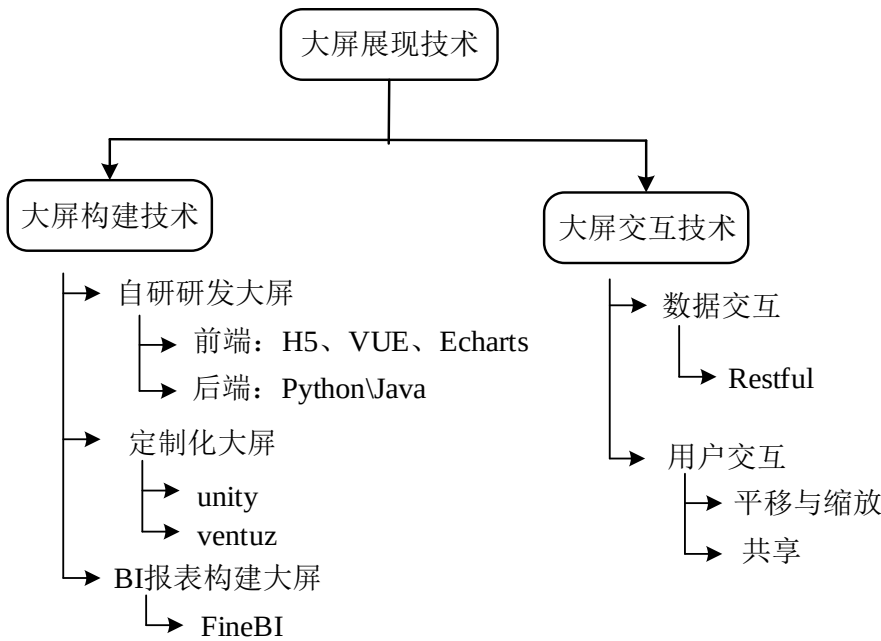


图 2-14 大屏展现技术架构组成

2.6.1 大屏构建技术

现在通常有三类构建大屏的方法，一类是自主研发前后端框架平台来构建大屏，但是受开发人员的技术水平限制，构建的大屏通常只能展示二维、简单的数据和基础类型的图表，因此易用性不足。

第二类是找寻求软件开发团队进行定制开发，这类开发的平台一般是根据特定的业务需求（如 3D 建模可视化）高度定制的，价格极其昂贵。

第三类是借助 BI 商业智能工具便捷地构建大屏，用户通过系统内置的图表组件库，可自主地根据业务需求设计组件，动态地组装可视化大屏。但是这些智能工具一般由开发公司维护，具有扩展性不高的缺点。

2.6.2 大屏交互技术

大屏交互具体又包括了基于大屏内数据更新的数据交互技术和基于大屏操作的用户交互技术。

（1）数据交互

数据交互，指的数据在大屏上实时动态变化。动态可视化能展现数据变化的过程，在基于交互可视化的基础上，动态展示数据信息的变化趋势，以加深对数据间关联的理解。因此，为满足互联网信息快速迭代的趋势，可视化大屏应实时动态展示不断更新的数据信息。目前基于事件的数据交互方式主流为使用 Restful 方式。

Restful 是基于 Rest 理念的交互风格，REST 的核心思想是资源，以 URL 方式进行资源的传递。服务端定义资源形式、资源类型以及获取资源的方法紧密相关。REST 是多入口的，每个资源对应一个 URL 标识，通过指定的 URL 访问该资源。作为用户交互的入口，Restful 有四种请求方式，分别是 GET，POST，PUT，DELETE，如用户通过向指定 URL 发送 GET 请求来获取资源，或发送 POST 请求来创建或操作资源。Restful 风格只返回 JSON 或 XML 数据，不关心页面如何展现。

（2）用户交互

用户交互指用户在浏览图表的时候对图表做一些交互操作，便于更好理解和分析隐含的数据信息。交互能直接推动可视化流程的顺利运行，其中用户交互的核心在于人机交互。目前，人机交互有以下几种方式：动态查询（dynamic queries）、平移与缩放（panning+zooming）、整体与细节（overview+detail）等。交互需求有重配图表、过滤图表、共享图表等，交互手段主要为键盘和鼠标。通过大屏可视化可以展示复杂数据和混合型图表，还可以过滤用户需求，只展示关注的的数据。

交互可视化不仅能加深人与数据的交流，还能提高用户的交互性，更好地进行数据分析。然而，适用于海量数据的智能交互方法仍然存在瓶颈，这些方法需要克

服硬件平台和显示环境不同以及目标任务多样的难点。

2.7 本章小结

本章首先介绍了数据可视化的基本概念，并对其分类和流程进行简要说明。接着介绍了多源异构数据可视化的流程中涉及的几种关键技术，并对每种关键技术的现有研究方法进行了说明。

第三章 基于模型的组件可重构生成技术研究

本章根据可视化组件的生成原理，提出一种基于模型生成组件的方法。区别于传统对组件库基于代码配置生成可视化组件，本章提出的方法通过构建组件快速生成框架，集成不同类型的组件模板，通过可视化界面简单配置的方式生成组件。本方法实现了将组件生成流程模型化，模型化的方式可以简化生成组件的流程并提高生成组件的效率。

因此，在本章中探索基于模型的组件生成方法对快速生成组件技术的发展具有很大的研究意义。在介绍本文提出的方法之前，首先介绍基于浏览器端生成组件的实现原理以及本文对各组件库的研究情况。

3.1 组件生成原理

由第二章研究可知，传统生成组件的流程是：原始数据映射成相应的图形元素，再通过矢量或者像素渲染的形式在浏览器端生成视觉可见的元素。随着可视化的发展，对组件生成的需求越来越高，不仅要求映射后的图形元素覆盖的图表种类数量众多，还对图表在屏幕的渲染性能提出了极高的要求。

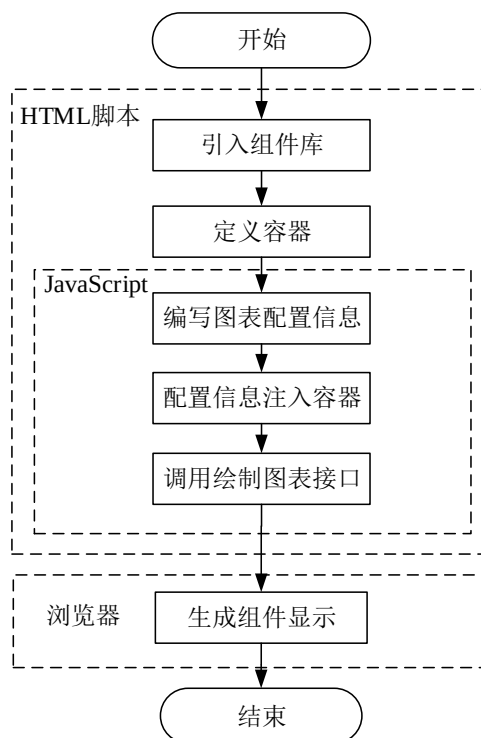


图 3-1 组件库生成组件的通用流程

在这种情况下,可视化组件库应运而生,这些组件库内置了一定数量的图表模板^[54],用户需要按照一定的规范编写代码,定义图表的配置信息(如图表标题、数据、图表样式等),即可在屏幕端渲染出图表,而无需关注底层浏览器端的渲染原理。这些组件库一般基于 JS (JavaScript, 解释型编程语言) 实现,JS 根据其即时编译的特点,被广泛应用于开发 Web 页面。

国外主流的组件库以 Highcharts、D3 等图形组件库为代表,是基于 SVG 矢量渲染。国内以 Echarts 组件库为代表,是基于 Canvas 像素渲染。如上图 3-1 所示,为使用不同组件库生成组件的通用流程。首先在 HTML 脚本中引入相关组件库的依赖,接着定义一个图表渲染容器,然后在 JS 中定义图表的配置数据,并执行数据装载和图形绘制的操作,最后在浏览器端渲染出对应的组件。

3.2 传统组件库的研究

3.2.1 基于矢量绘制的组件库

(1) Highcharts 组件库

Highcharts 是一套基于 JS 编写的组件库,可以方便的在可视化页面中实现交互性的图表^[55]。Highcharts 组件的核心是 SVG 绘图语言,同时使用 JSON 格式来定义项图表的配置信息。原始数据在 Highcharts 底层被分析转化成在坐标轴可见的元素,最终形成合理的可视图表。如图 3-2 所示,为使用 Highcharts 在浏览器端生成组件的流程。

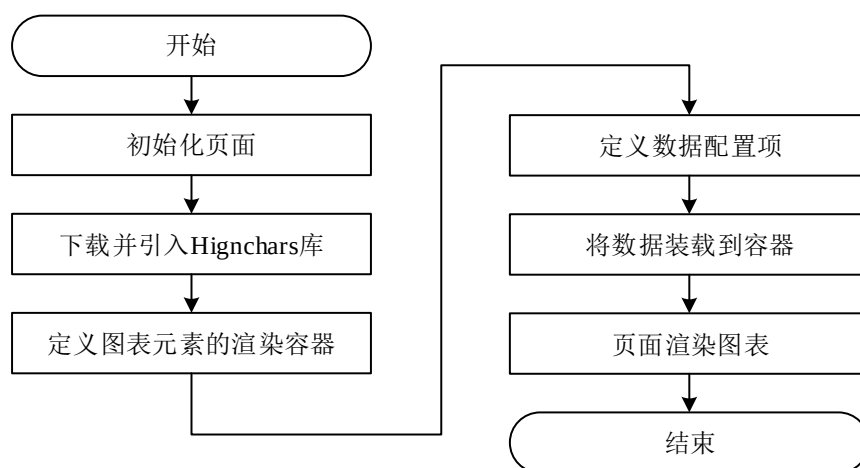


图 3-2 Highcharts 生成组件的流程

根据图 3-2 可知,首先初始化阶段会创建一个空的页面模板,接着需要在模板中引入相关库并建立一个具有一定宽高的图表渲染容器,之后定义图表的配置数

据，并将数据装载到容器，即可实现在页面渲染 Highchart 图表。通过分析，使用 Highcharts 生成组件的优点有：1) 兼容性好；2) 使用方便；3) 图表类型丰富；4) 应用性好；5) 具有动态交互功能。

(2) D3 组件库

D3 (Data-Driven Documents, D3.js) 也是基于 JS 的可视化组件库，它允许在数据集的上下文来操作文档页面元素。D3 遵循当前的 Web 标准，基于可视化组件来驱动 DOM 操作。如图 3-3 为 D3 在前端页面生成组件的流程。

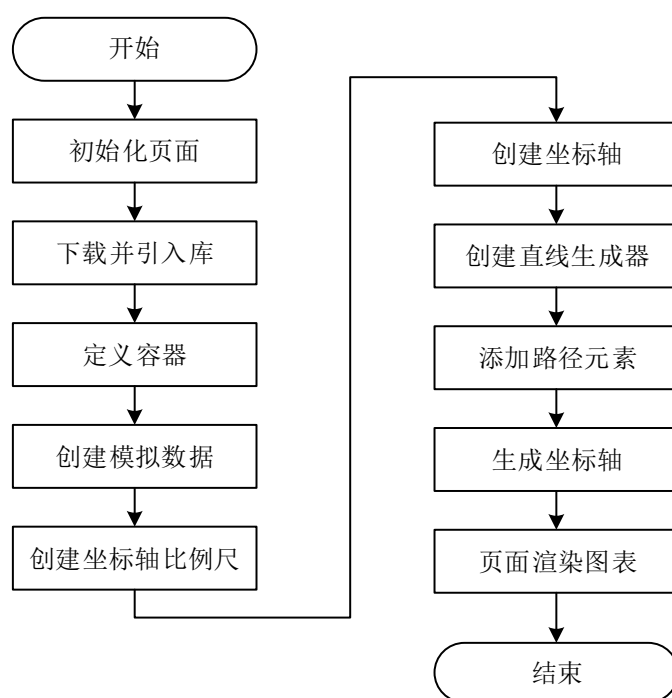


图 3-3 D3 生成组件的流程

由于 D3 可以实现对任意对象的定义，在定义了图表容器后，首先要设定坐标轴的比例尺，接着创建坐标轴元素。然后创建一个直线生成器，将生成的值赋值给添加的路径元素，最后再进行坐标轴的生成，即可绘制出完整的折线图。也就是说，D3 的数据映射与绘制渲染相互独立。使用 D3 生成组件的优点有：1) 可以绑定任意数据到文档 DOM；2) 封装了大量图表的数据转换算法；3) 图像精度高。

(3) Plotly 组件库

虽然 D3.js 拥有强大的功能，可以根据需求进行定制化图表的设计，但是在使用流程上较复杂。因此基于 D3.js，后续出现了一些轻便的组件库，如 C3.js 和 Plotly.js。其中，Plotly.js 是基于 D3.js 和 Stack.gl 的高层次的 JS 组件库。基于 Plotly 生成组件的流程，除了将数据装载到容器的方法不同，其余均和 Highchart 类似。

3.2.2 基于像素渲染的组件库

Echarts，基于 Canvas 渲染，是百度研发的可视化组件库^[56]。Echarts 底层基于 Zrender 渲染，可以流畅的运行在各种终端。Echarts 在前端页面生成组件的流程也与 Highchart 类似，只是通常在定义的渲染容器上先初始化一个 Echarts 实例，然后再进行图表渲染的操作。使用 Echarts 生成组件的优点有：1) 兼容性好；2) 使用便捷；3) 交互性好；4) 可跨平台运行；5) 定制灵活。

3.2.3 不同组件库的对比分析

为了探究不同组件库生成组件的区别，本文分别对研究的组件库进行了生成折线图的实验。其中，实验数据来源于 2014 年到 2020 年电子科技大学的本科生和研究生（硕士、博士）毕业人数统计 (<https://eportal.uestc.edu.cn/new/index.html>)。实验过程是首先分别搭建不同组件库的渲染页面，并引入相关库的依赖，然后编写相应的图表配置信息，最后调用组件库的绘图接口，渲染生成折线图组件。我们在 Google 浏览器（版本号 90.0.4430.212）打开渲染后的结果。如图 3-4 为分别基于 Highcharts、D3、Plotly、和 Echarts 组件库生成折线图的实验结果。

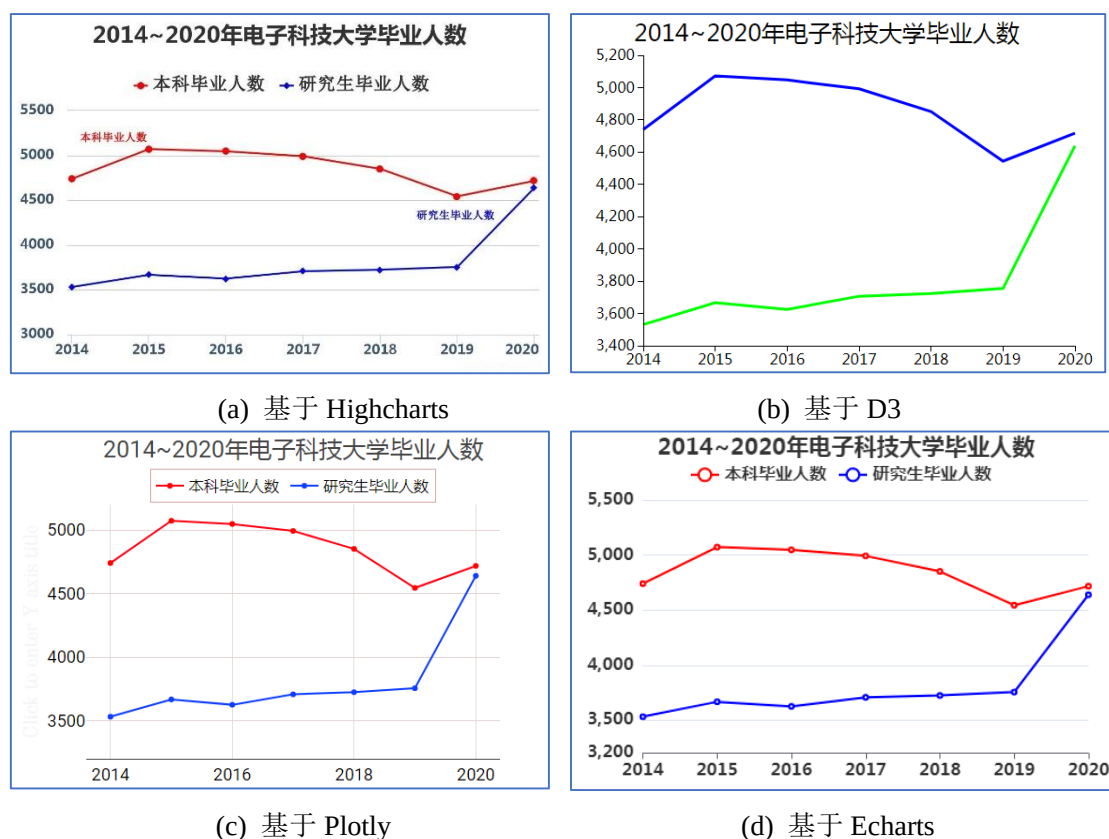


图 3-4 基于不同组件库生成的折线图组件

本文对不同组件库生成组件的流程进行了探索，并完成了基于不同组件库生成折线图组件的对比实验。实验过程和结果表明，Echarts 组件库的使用流程更简便，且图形渲染效果较好。此外，本文还详细对比了 Echarts、Highcharts 与 D3 组件库的表现差异，对比结果如表 3-1 所示。

表 3-1 Echarts、Highcharts 与 D3 的对比

特点	Echarts	Highcharts	D3	总结
兼容性	兼容 IE9 及以上的主流浏览器	兼容 IE6 及以上的主流浏览器	兼容 IE9 及以上的主流浏览器	Highcharts>Echarts =D3
是否免费/开源	完全免费，开源	非商业免费，商业需授权，开源	完全免费，开源	Echarts=D3> Highcharts
支持定制能力	基于 Canvas，定制能力一般	基于 SVG，可方便定制开发	基于 SVG，灵活度高，定制自由度高	D3>Highcharts> Echarts
支持图表种类	图表类型丰富，包括酷炫的 3D 效果	图表类型有限	图表类型有限	Echarts>Highcharts =D3
大数据表现力	适合万条以上的数据	适合超大数据量	适合超大数据量	Highcharts=D3> Echarts
学习难易程度	国内开发，有详细的配置文档说明，易二次开发	国外开发，有中文档说明，学习难度一般	学习难度较大，适合高度定制项目	Echarts>Highcharts> D3 (从简单到困难)

根据表 3-1 可知，Echarts、Highcharts 和 D3 各有优势与劣势，Echarts 学习难度最小，现已有丰富的图表示例及强大的图表展示效果，且完全开源易于二次开发。Highcharts 和 Echarts 类似，但是图表美观程度不如 Echarts，且非商业免费。D3 灵活性最高，但是生成图表流程最复杂，学习难度也较大，适用于高度定制化的项目。

基于以上讨论结果，本文提出一种基于模型的组件可重构生成技术。该技术在前期组件库的研究基础上，构建了组件快速生成框架，并在框架内集成了 Echarts 类型的图表模板，实现模型化的方式生成可重构的组件。同时在扩展实验中还集成了基于 D3.js 的 Plotly 类型的组件模板以及非图表类型的组件模板（如文本组件、视频组件等），以验证本方法的可扩展性。

3.3 基于模型的组件可重构生成技术

在本节中，提出一种基于模型的组件可重构生成技术，希望通过模型化的流程高效地生成组件。在此之前，首先说明基于现有组件库生成组件存在的问题。

虽然现有的组件库通常已经集成了大量图表模板，用户无需关注底层图形渲染原理，即可根据配置信息生成相应的组件，但这种方式的缺点是在用户在使用这些组件库之前，需要具备通用的 JS 和 HTML 等前端开发技术，并且还需要熟悉不同组件库的编码规范以编写相应的图表配置信息，因此对无技术背景的用户使用体验非常不友好。

此外，通过前面分析组件库生成组件的流程可知，基于组件库生成组件的方式都是按照固定的流程自上而下地配置代码以呈现图表。不仅生成组件的流程繁琐，且生成的组件不具有通用性，即当已有组件不满足预期需求时，需要重新修改配置项部分的代码。进一步的，如果可视化的需求频繁变动会导致需要已有代码的丢弃和重新配置，从而造成代码复用低、生成组件效率低的情况。

综上，考虑到这些问题，本文在研究主流的组件库的基础上，基于 Echarts 和 Plotly 组件库构建并实现了相应的图表模板。并在此研究成果上，提出了基于模型的组件可重构方法。本文提出的可视化模型，将组件的生成流程模型化、简便化。应用此模型，用户可以根据需求自由选择图表模板，通过可视的配置界面快速生成相应的组件。这种方式生成的组件类型不仅包括主流的组件库图表，而且包含文本组件、视频组件等其他非图表类型的组件，可满足用户多方面的可视化应用需求。最后，本文引入了组件可重构的概念，在基于模型生成的组件不满足预期要求时，实现对组件的各部分组成单元部分更新或对组件整体快速重组，提高了组件的通用性和提升了组件生成的效率，进而可以高效响应不断变化的需求。

如图 3-5 所示为本文提出的基于模型的组件可重构生成技术架构，首先构建了组件快速生成框架，实现在可视化的配置界面生成组件。然后在组件库的研究基础上，构建了相应的图表模型，形成模型化的组件快速生成流程。最后在组件管理中引入可重构和删除等功能对生成的组件进行统一管理。

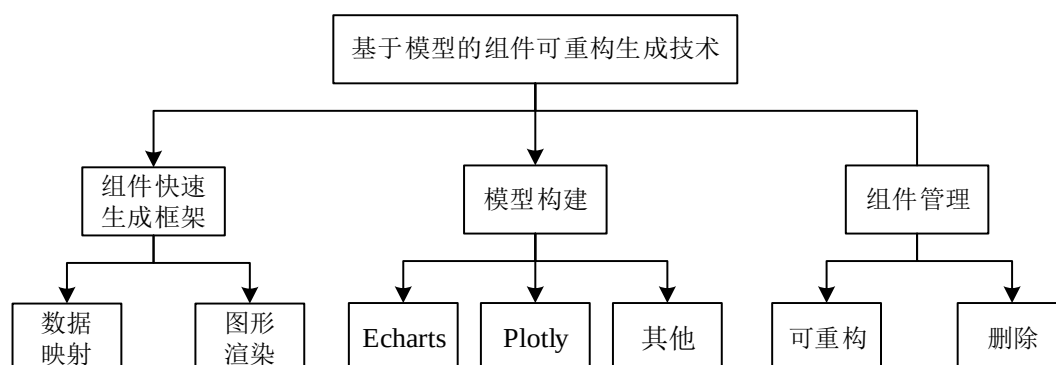


图 3-5 基于模型的组件可重构技术架构

3.3.1 总体流程

在图 3-6 给出了本章实现的基于组件快速框架中生成组件的流程。框架内部构建了 Echarts 类型的组件模型，因此需要熟悉 Echarts 组件库生成组件的原理。同时，默认可视化的数据类型为多源异构数据，且已经经过采集、处理和整合后统一存储在数据查询集合中。

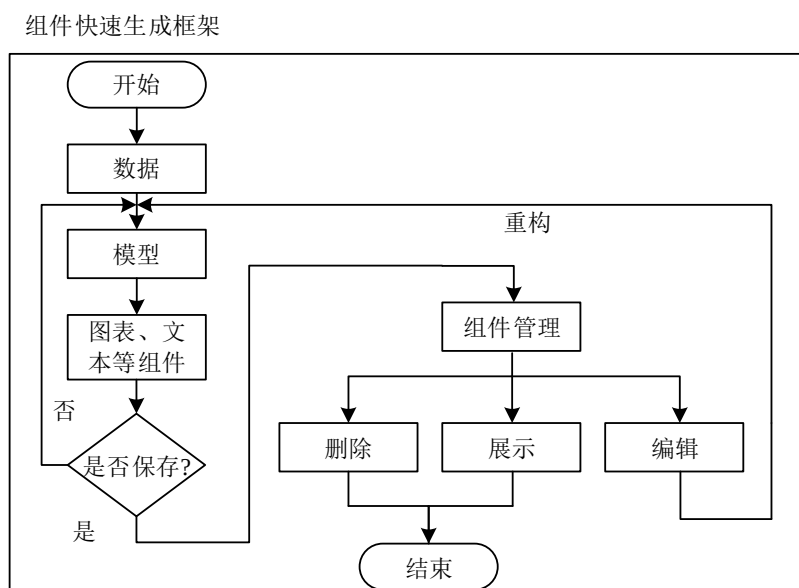


图 3-6 基于组件快速生成框架生成组件的流程

根据图 3-6 生成组件的流程，本节提出的基于模型的组件可重构生成方法实现大致分为五步：

第一步，搭建组件快速生成框架，并在底层框架中引入组件库接口，实现通过图形化的配置界面生成图表组件。

第二步，基于 Echarts 组件库构建不同的图表模型，并集成挂载到组件快速生成框架中，实现模型化的方式快速生成组件。

第三步，构建组件管理模块实现组件可重构配置。

第四步，在模型中扩展集成其他类型的组件库（Plotly）和其他类型的组件（文本组件等）。

第五步，验证本文方法的有效性并对构建的组件库进行功能验证和性能评估。

3.3.2 组件快速生成框架

搭建组件快速生成框架是在研究前端开发框架的基础上，在框架内部集成组件库接口，并构建统一的图形设计页面，用可视化的图形配置界面代替传统代码生

成组件的流程。通过简单的可视化配置操作,即可实现数据映射与图形渲染并实时在浏览器端的界面快速展示可视化组件,构成用户友好型的可视化组件生成技术。

传统的 Web 开发框架缺乏完善的业务逻辑能力,因此在复杂数据的可视化应用实践效果不佳。为此,我们研究了目前主流的前端开发框架 Vue.js、Angular.js、React,以下是对这三种框架的研究分析。Vue 是一个由国内开发的前端开发框架,VUE 的思想是数据驱动,具有轻量化和双向数据绑定的特点,但是成熟度不高。

Angular 是一款完善的 Web 界面框架,也具有双向数据绑定的特点,因为其强大的功能,已经被应用于 Google 的产品中。如图 3-7 所示为 Angular 的生命周期,首先进入构建阶段进行生命周期的初始化,接着在初始化阶段向服务端发起请求。当绑定的数据状态发生变化时候,在状态更新阶段,接收更新的值。之后进入检查阶段,如果是第一次加载将对内容和视图进行初始化。如果是数据变化导致状态变化,则分别检测内容和视图的变更,进行页面的更新与渲染,最后进入销毁阶段。

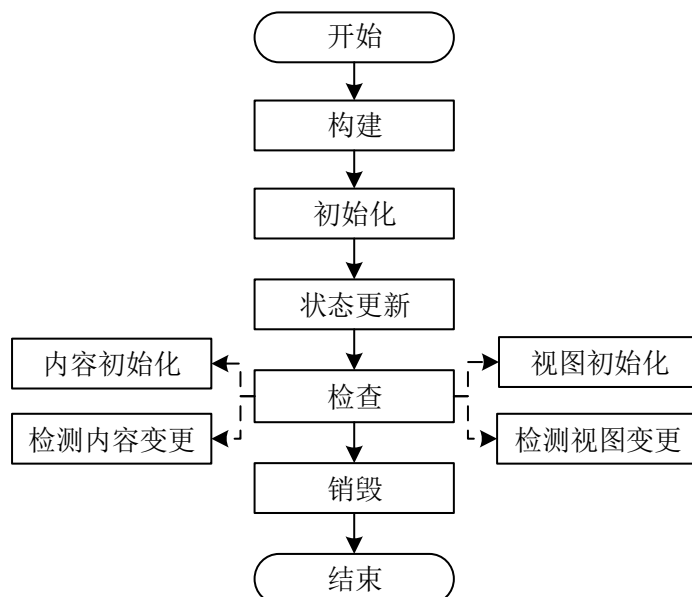


图 3-7 Angular 的生命周期流程

React 是 Facebook 推出的 JS 库,具有高效灵活的特点,因此可以方便地构建简洁美观的用户界面。但是 React 内部的数据流是单向数据流,因此通常需要结合 ReactRouter 和 Flux 使用。React 组件的生命周期有组件挂载、组件渲染和组件卸载。在初始化阶段,获取默认的属性 and 数据状态,进行页面的初始化渲染,随即进入挂载完毕状态。在运行状态中,发生属性改变则会更新属性并判断状态是否更新,如果更新则会以更新后的数据重新渲染组件。在组件卸载阶段,会将内存进行回收。

通过对比分析,由于构建统一的设计页面涉及用户与前端的交互操作。而基于

Angular 的双向绑定原理，能实现页面的输入改变数据模型的值，数据的变更驱动页面的更新，适合组件设计页面的接口配置。因此，本文采用 Angular 与 Echarts 构建组件快速生成框架，实现图表组件的快速生成。

Angular 是一种基于 M-V-VM 架构的框架，如图 3-8 所示为 MVVM 架构组成。其中 M 指 Model（数据模型），V 指 View（视图页面），VM 指的是 ViewModel（Model 与 View 之间的核心枢纽）。Angular 双向数据绑定的核心原理是构建视图与数据模型之间的数据绑定与视图映射。如图 3-8 所示，Angular 基于 Data Binding 将数据模型与视图的值绑定，当数据模型的值发生改变，框架内部会自动将视图进行更新。同时，基于 DOM Listener，当视图发生更改时，框架还会自动侦听视图操作并更改数据，从而形成双向的数据绑定。

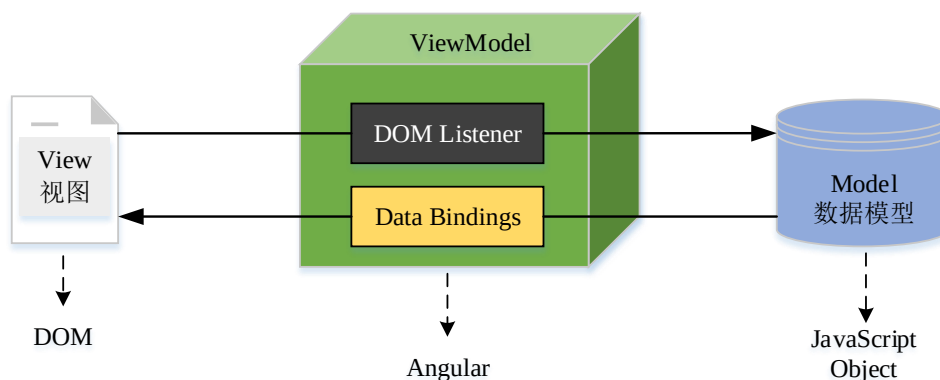


图 3-8 MVVM 架构

当绑定的数据模型发生变化时，JS 环境与 Angular 内部的环境会发生事件交互，其事件循环机制如图 3-9 所示。

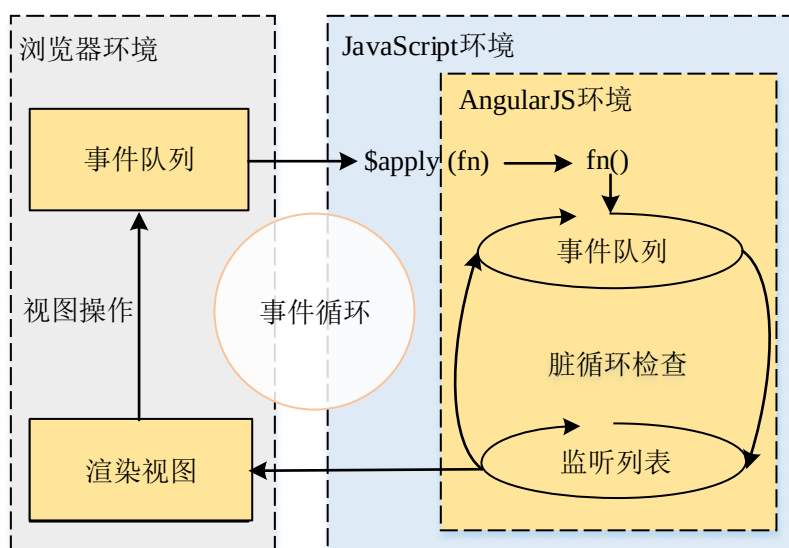


图 3-9 Angular 的事件循环机制原理

首先,每当绑定一个数据到视图上时,监听列表会插入相应的监听函数。当视图触发指定操作后,浏览器接收到相关事件,事件先进入事件队列中等待。接着进入 JS 环境并在环境内部执行 `apply` 方法,再进入 Angular 环境内部,将回调函数的处理事件放入异步队列等待被执行。这些事件可能会导致绑定的数据模型的值发生改变,此时会触发“脏循环检查机制”(digest-loop),它由 2 个小循环组成,1 个处理异步队列,另一个处理监听队列。

“脏循环”进行循环时将遍历监听的所有队列,查看是否有数据更新,这种遍历称为“脏检查”(dirty-checkin)。如果脏检查发现有监听列表下的数据发生更新,则会触发新的脏检查,直到所有的监听列表都没有更新,这样就能保证每个数据模型都不会变化。此外,当检查到作用域上的数据模型发生变化时,还会进行视图的重新渲染。通过不断执行“脏循环检查机制”,可以确保整个作用域的数据模型处于稳定不再更新的状态。

基于 Angular 的双向数据绑定机制,本节构建的组件快速生成框架如图 3-10 所示,该框架由三个模块构成,分别是组件挂载渲染模块 A、逻辑控制模块 B 以及接口配置模块 C。

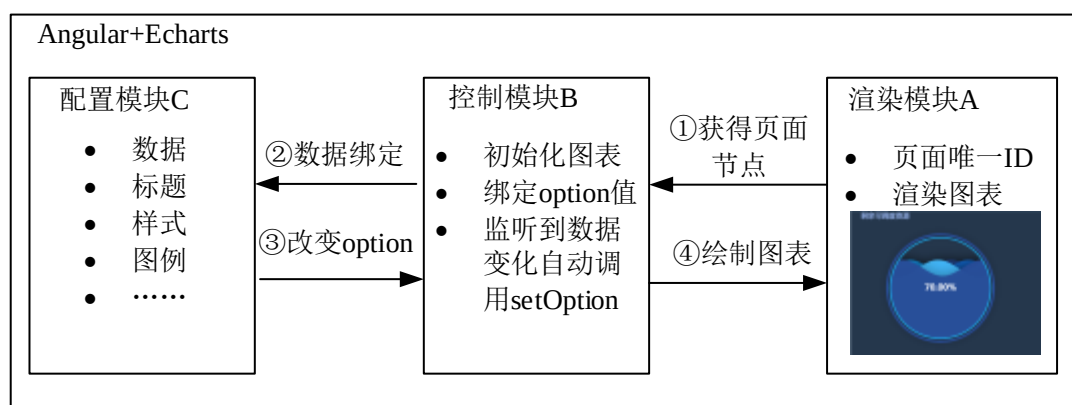


图 3-10 基于 Angular 的组件快速生成框架实现

渲染模块 A, 包括一个 HTML 的画布页面, 用于在浏览器端实时渲染展示组件, 该页面拥有一个字符类型的标志符 ID。在初始化时状态时, 画布默认为空。

控制模块 B, 是整体的业务逻辑处理部分, 用于控制数据到组件的渲染。在页面初始化加载时, 控制模块 B 通过页面的节点 ID 获取到页面元素, 进而初始化一个 Echarts 实例。同时, 控制器的作用域内包含关键的全局对象 option, 存储着所有 Echarts 图表配置项的数据。

配置模块 C, 提供了更改 Echarts 组件配置项的可视操作, 如图 3-11 所示为本文构建的组件配置接口页面。



图 3-11 接口配置页面实现效果图

配置模块的核心是实现画布的实时更新，即在可视化界面区域通过选项框、输入框等元素对组件的参数如数据、标题和样式等进行配置后，画布中可以实时呈现对应的效果。要实现组件的实时更新，需要理解浏览器的事件调度机制。浏览器运行机制通过事件来管理数据状态的变化，实现 HTML 页面和 JS 程序通信。

当用户点击了某个按钮或用户在某个输入框进行了数据输入的操作，在浏览器内部会产生相应的事件。此时，需要提前对这些可能发生的事件进行监听，并且编写相应的事件回调函数。事件回调函数指的是监听到用户进行了某个操作后，下一步程序需要执行的方法。通过这些事件回调函数，可以实现事件监听与响应。此外，HTML 页面内的所有元素对应成 DOM（Document Object Model，文档对象模型）的表现形式是层级的树状节点，其中根节点（Document）是整个文档的顶层对象，在 DOM 中各元素节点具有严格的事件传播机制。

如图 3-12 是两种事件传播机制的描述，按事件的传播方向可分为冒泡过程和捕获过程。对于冒泡过程，是指事件从 DOM 的子节点层层往上传播到根节点的过程。对于捕获过程，则是事件从根节点逐层往下传播到子节点的过程。

调研发现，冒泡型的事件传播机制在浏览器的兼容性能表现更好，因此本节采用冒泡型的事件传播机制实现对画布实时更新。首先对接口配置界面的父级元素添加监听事件，基于冒泡机制，对其所有子元素的触发事件会向上传播到父级元素，这些事件会被父级元素统一代理，这种方式称为“事件委托”。基于 Angular 的双向绑定机制，采用数据绑定指令将配置页面 C 中的输入元素与控制器中的配置项数据的字段一一绑定。

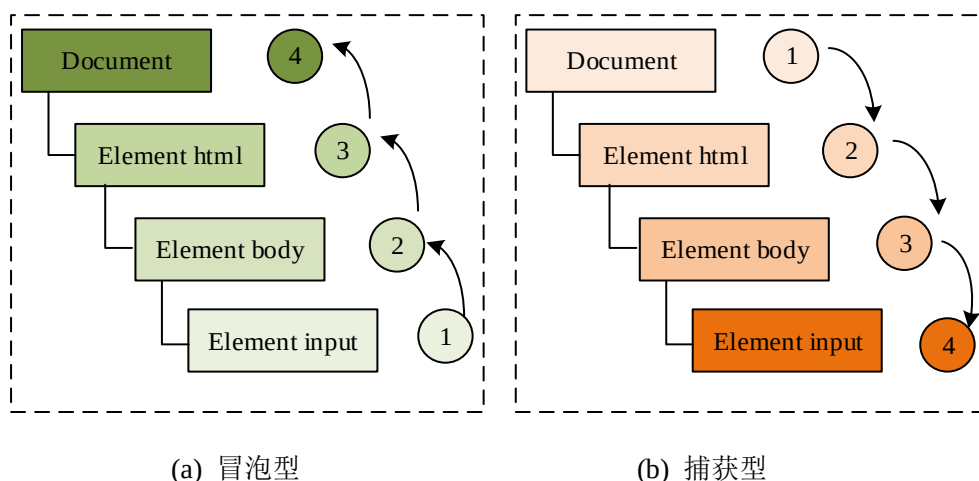


图 3-12 两种浏览器事件传播机制示意图

如图 3-13 是画布页面实现渲染更新的流程图。在第一次加载渲染页面时，会进行画布页面的初始化过程，此时页面将渲染出空白的画布。当用户操作配置界面的元素时，绑定到配置项对应的值也会发生变化，此时会同步触发监听事件并被父级元素代理。接着会在回调函数的处理中调用控制模块的绘图方法，加载更新的配置数据进行页面渲染。如果此时连续操作配置界面的元素（如连续点击某个按钮）会多次触发事件并不断执行事件回调过程，导致系统性能消耗过大。

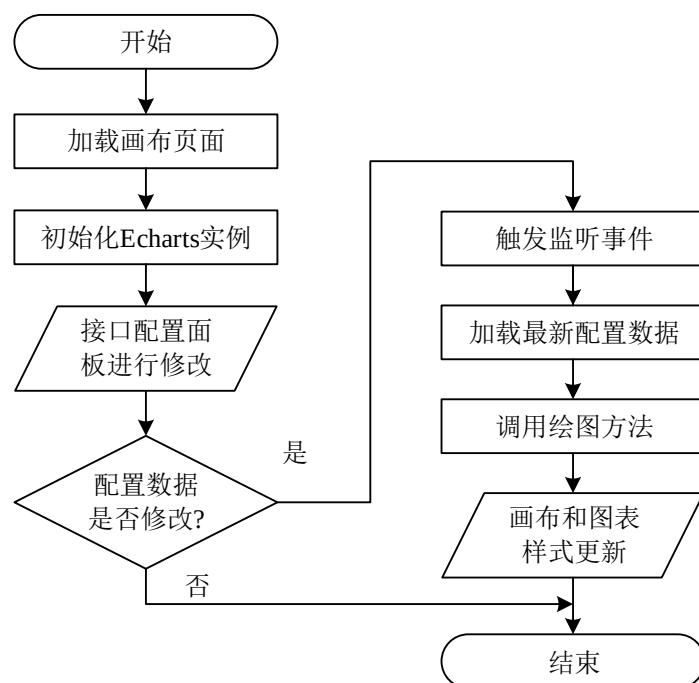


图 3-13 画布实时更新流程

因此本节还引入了“防抖”算法，如图 3-14 为“防抖”的核心思想。首先设定

一个定时周期 T ，当触发事件时，在定时周期后延迟执行相应的操作。如果在此周期内又触发了事件，则重新计算定时时长，直至在定时周期内无事件再次被触发，则执行动作。该算法可以实现当事件在短时间内被多次触发时，只对最后一次触发操作进行处理。应用到本节构建的框架中，当用户在短时间内连续操作配置界面的元素时，只会在最后一次操作时执行动作，更新配置项的数据。这样的方式可以减少画布不断更新带来的资源损耗，同时也提高了界面操作的流畅性。

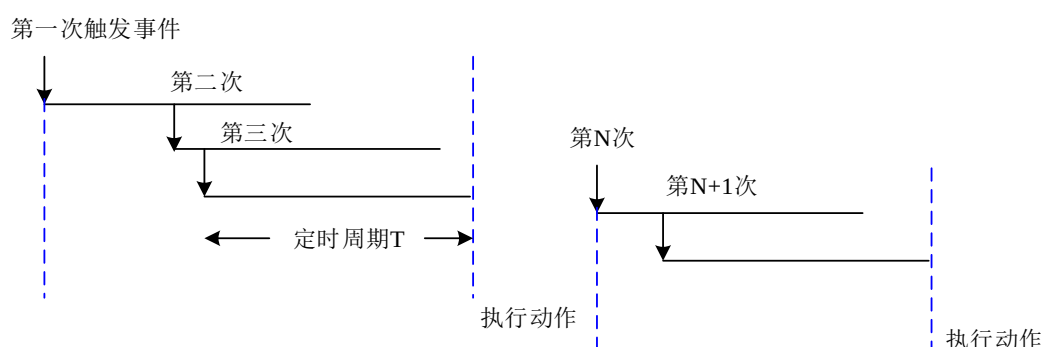


图 3-14 防抖算法的原理

本文按照上述方法实现了画布页面的实时更新。如图 3-15 所示，展示了在基于可视化的配置界面配置组件参数前后，画布页面的渲染情况。

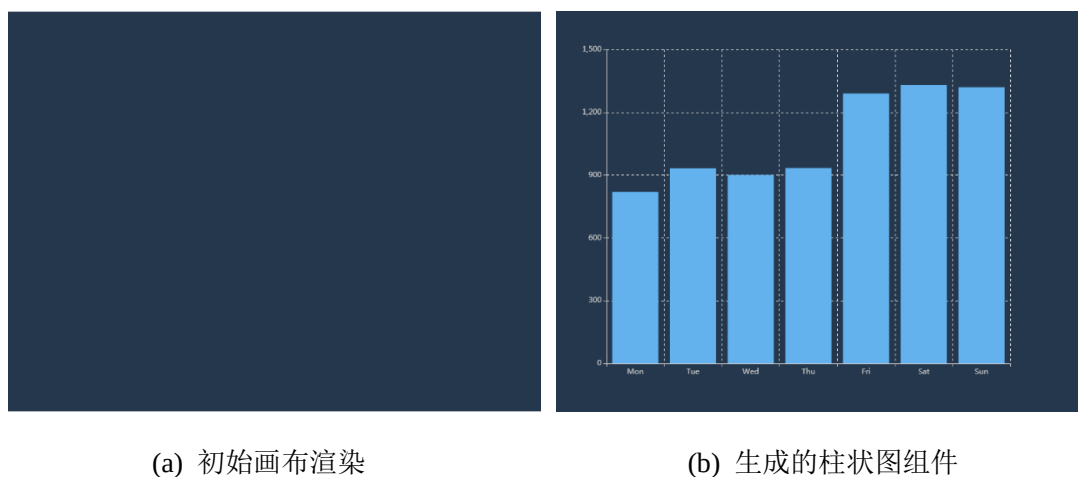


图 3-15 画布页面渲染组件前后的效果图

通过渲染结果可知，基于本文构建的组件快速生成框架，实现了用可视化的图形配置界面代替传统代码生成组件的流程，以简单的配置操作即可实现组件的敏捷生成。对于使用者来说，不需要具备开发技术和提前熟知不同组件库的编码规范，通过简化生成组件的流程提升了用户体验。

3.3.3 模型构建

在搭建好组件快速生成框架后，我们面临的问题是如何实现基于组件模板快速生成指定类型的组件而互不影响。因此在本节中引入模型的概念，基于组件库构建图表模板，将生成组件的流程模型化。在模型入口接入原始数据，在模型内部进行图形配置，即可在模型出口快速生成对应模板的组件，因此构建组件模板是模型实现的核心。

3.3.3.1 模型内部流程

本节在研究组件生成的基础上，基于 Echarts 组件库构建图表模板，并集成挂载到组件生成框架中，形成模型化的组件生成流程。如图 3-16 所示，为模型内部的流程图，模型的入口是选择的查询数据集，出口是浏览器端渲染的可视化组件。

模型内部是基于组件库构建的组件模板，为每个组件模板提供了对应的组件配置项。使用者无需关注模型内部的具体实现机制，选择组件模板后，通过可视化界面的简单配置即可快速生成指定模板的组件，并且丰富的组件配置项为组件提供了强大的定制化功能。

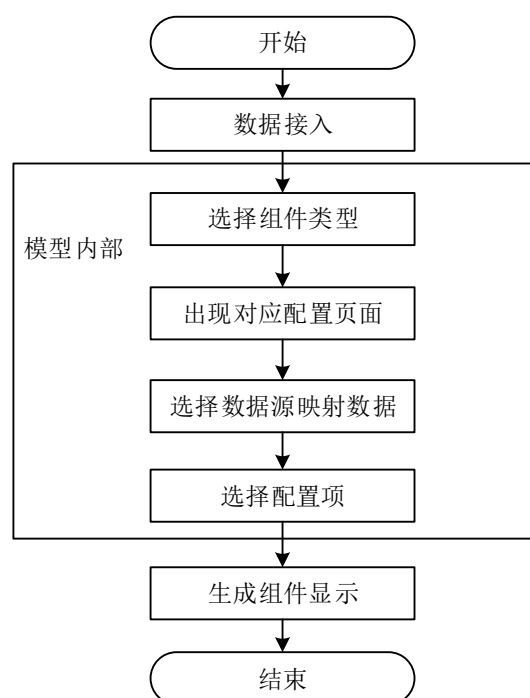


图 3-16 模型内部流程图

3.3.3.2 构建图表模板

这一步基于组件库构建图表模板，在快速生成框架的基础上，如图 3-17 定义

了一个 Echarts 图表模板的组成结构，主要由四个部分构成：组件默认配置模块、组件挂载渲染模块、组件接口配置模块以及组件逻辑控制模块。

其中，组件默认配置模块，用于初始化组件配置。组件挂载渲染模块，包含组件生成的渲染页面，该页面拥有唯一标识 ID，控制模块通过获取该节点 ID 进行页面的初始化。组件接口配置模块是用户交互中最重要的部分，以可视化界面的形式提供组件的配置项给用户操作。由于不同组件的配置项不同，因此需要根据组件的类型构建相应的配置界面。组件逻辑控制模块控制整个交互流程的数据流向。

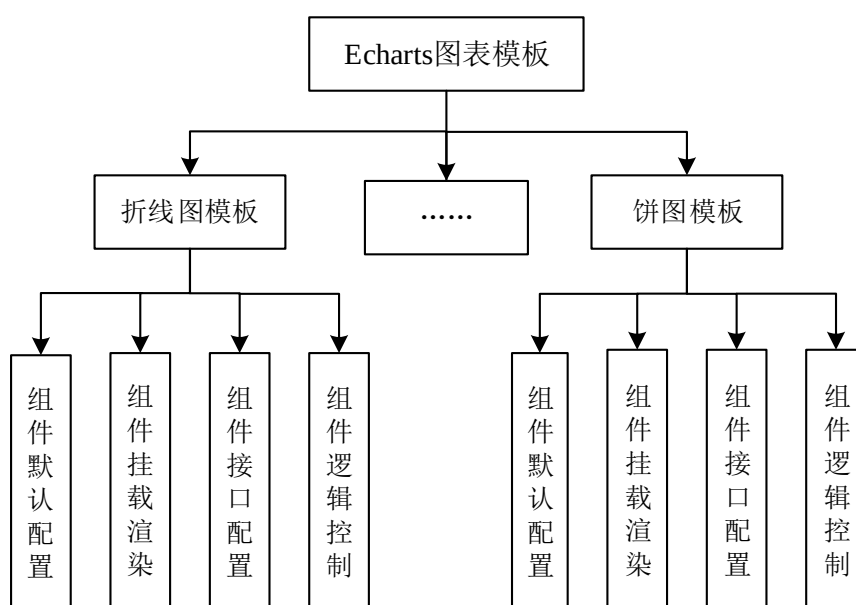


图 3-17 Echarts 图表模板的组成结构

如图 3-18 为各模块之间的交互图。首先，将接口配置的视图与控制模块的数据模型进行绑定。在初始化阶段，控制模块加载图表默认配置，绘制初始图表。当配置数据更新后，控制模块清空原有画布，并重新绘制图表。

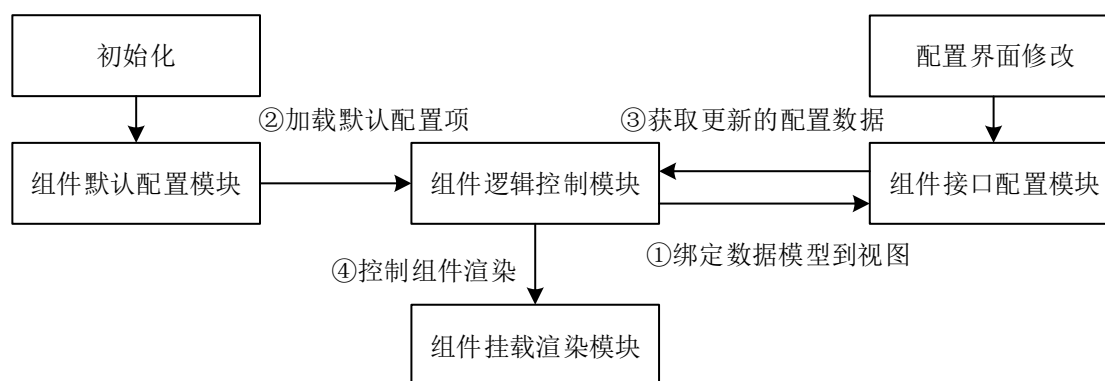


图 3-18 Echarts 图表模板各个模块之间的交互

本文基于上述方法构建了 Echarts 饼图模板。在基于模板生成组件的过程中,渲染页面的更新过程如图 3-19 所示。首先,在可视化之前将数据存储在数据集中,然后选择合适的组件模板。在渲染页面初始化时,逻辑控制模块先获取渲染页面的节点,再加载组件的默认配置,并将这些配置放入图表配置对象(option)中。最后初始化 Echarts 实例,并在渲染页面生成默认展示的组件样式。

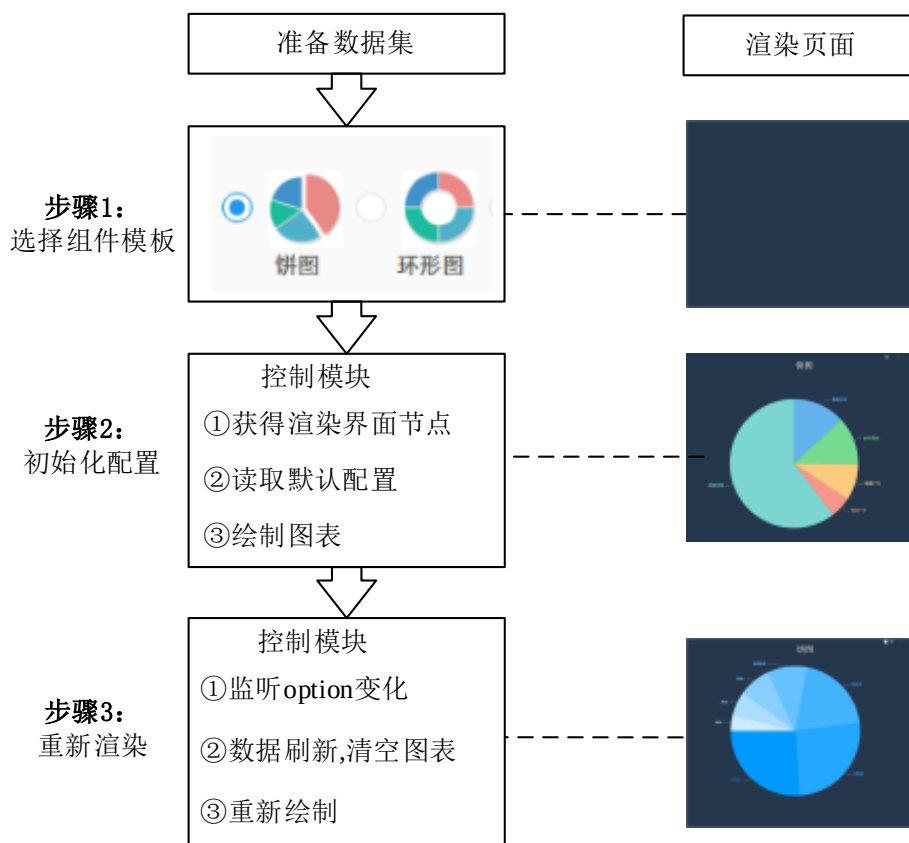


图 3-19 基于饼图模板的页面渲染示意图

当在组件接口配置页面进行操作时,绑定到 option 的数据模型相应发生了变化,此时控制模块监听到 option 发生了变化,会执行数据刷新操作,先清空当前画布绘制的组件,再重新调用绘图方法,在渲染页面绘制出更新后的组件。

3.3.3.3 组件编辑界面

此外,为了便于对不同模板的组件进行可视化配置,形成统一预览的渲染视图,本文还设计并实现了基于不同模板的组件编辑页面。

如图 3-20 所示,为构建的组件编辑页面的实际效果图,将整体界面分为三部分:左侧为组件模板选择区域,中间为提供了不同图表模板配置项的配置区域,右侧则是画布渲染区域。在组件编辑页面,基于模型生成组件的操作流程为:首先接

入数据源，接着选择组件模板，然后在对应模板的接口配置区域对组件的配置项进行修改，最后会在右侧画布区域渲染生成的组件。

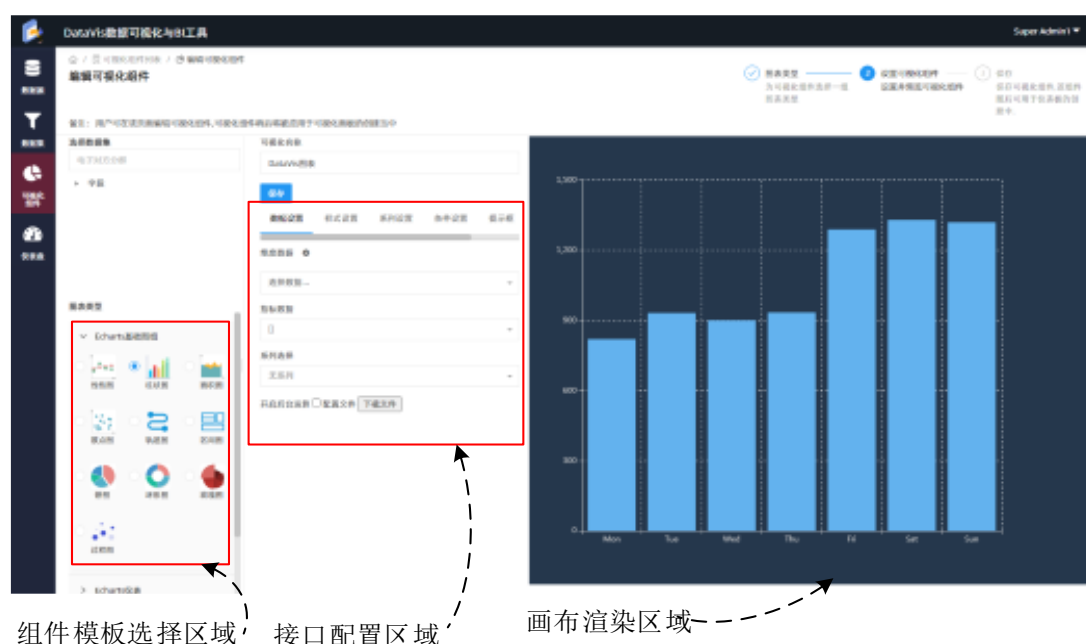


图 3-20 组件编辑页面实现效果图

3.3.4 组件可重构实现

传统生成组件的方式，如果生成的组件不满足预期要求或需要大量更改组件的样式时，需要频繁更改配置代码甚至重新开发组件。目前本文构建的基于模型的组件快速生成框架，以图形化的界面代替了代码开发方式来生成组件，在组件未保存前，如果需要对组件进行更改（如重新选择图表模板、修改标题、修改样式等）可直接在模型内部对配置界面的配置项进行操作。

面临对已保存的组件进行重新配置的需求，本文引入组件可重构的概念。组件可重构是指在组件快速生成框架中，当基于模型生成的组件不满足预期要求时，将组件重新解构设计或更新部分配置项的数据，以达到及时调整组件的功能，迅速响应可视化需求。其核心是组件的可重构实现，即对组件模块的各部分构成单元进行部分更新或对组件整体进行快速重组，以此改变模型输出的结果。基于可重构性，大大提高了组件功能的灵活性，可以不断调整组件生成的各个阶段，实时、高效响应不断变化的可视化需求，提高了组件复用率。因此可重构组件具有可动态变化、可持续扩展等重要特性。对于可重构组件技术的研究、开发和应用是未来可视化组件生成的重要关键技术。

本文对组件可重构的研究，按照以下方案实现：

- (1) 定义生成组件的构成单元，将生成组件的信息保存至服务端；
- (2) 当需要对生成的组件重构时，通过索引该组件的 ID 向服务端发起请求，获取该组件的具体配置信息；
- (3) 链接到该组件的可视化编辑界面，将组件配置信息分别渲染到编辑页面的配置模块和渲染模块，并在画布页面生成已构建的组件；
- (4) 更改组件的配置项对组件进行重构，并将修改后的组件配置信息覆盖原来的组件配置信息。

3.3.5.1 模块组成

如图 3-21 所示为组件可重构功能模块与其他模块之间的交互示意图，下面详细介绍每个模块的具体实现功能。

其中，数据模型定义了图表模板的基础类，包含基于模型的组件生成的方法，每个基础类包含了数据源、图表种类以及组件的配置项等属性。组件类是所有组件的父类，是基于模型生成的组件的对象原型。每个组件包含图表的私有属性。组件控制模块包括一系列基于组件的相关操作，当新建组件时候，会初始化一个新的组件。该模块具体实现功能包括查询已创建的组件列表，对组件进行预览展示，编辑组件的基本信息，永久删除不需要的组件等。组件可重构封装了一系列基于组件可重构的方法。

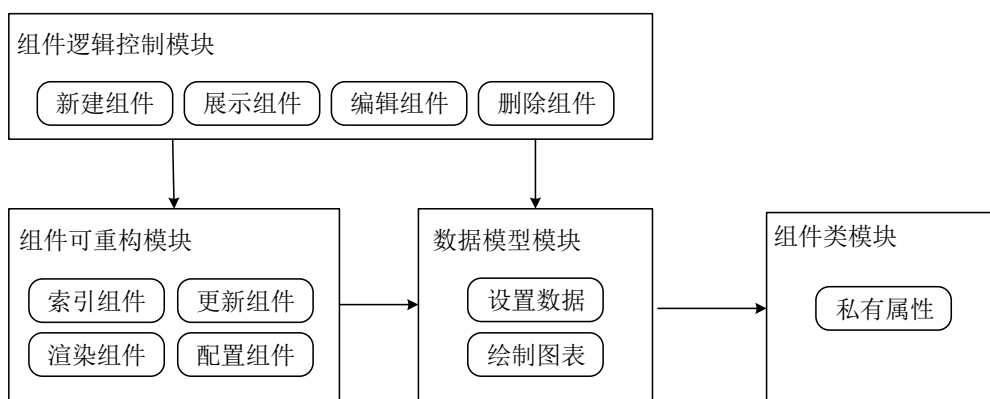


图 3-21 可重构模块与其他模块的交互

3.3.5.2 定义组件构成元素

如表 3-2 所示，定义了组件的构成元素。其中，id 代表组件的唯一序列号，是基于 UUID 标准随机生成的，便于与其他组件进行区分。name 是生成可视化组件的名称，option 是以 JSON 对象保存的该组件的配置数据。type 是指图表组件的类型，如折线图命名格式为“line”，饼图命名格式为“pie”。

表 3-2 组件的构成元素

组成元素	含义
id	组件的唯一标志
name	组件的名称
option	组件配置项，存储的组件信息
type	组件的类型

3.3.5.3 索引组件

当创建完一个组件后，会生成组件的描述信息。并在组件管理界面以目录树的形式展示生成的组件名称列表。本文采用“事件委托”的机制，将子元素的所有点击事件统一由父级元素代理。当点击生成的组件时，会通过点击事件获取当前组件的 id，并在父级的回调事件处理中向服务端发起请求。服务端根据接收到的 id 索引该组件的信息并在管理界面渲染出该组件的预览视图。

3.3.5.3 重构组件

实现可重构的完整流程如图 3-22 所示，首先在组件编辑页面生成一个组件，此时该组件的信息将被保存至服务端。当需要对组件进行重构时，可重新进入模型内部，并渲染出生成的组件视图。之后可以在配置界面操作图表的配置项，当视图绑定的数据项发生变化时，画布页面将重新渲染生成的组件。如果保存新生成的组件，则会将最新的组件配置信息保存至服务端，并替换原来的组件配置信息。

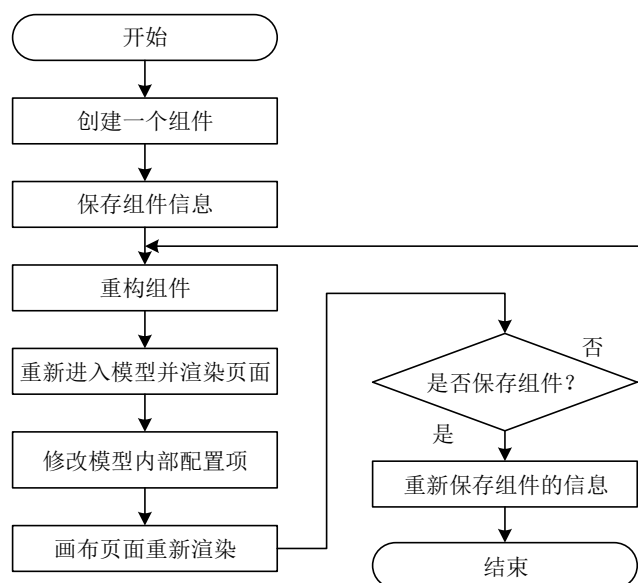


图 3-22 组件可重构流程

具体实现过程是当需要对该组件的进行重构时，同样先索引该组件的 id 获取组件的详细配置信息，并将当前页面重定向到组件编辑界面。在服务端返回组件的相关信息后，会调用可重构模块的配置组件方法和渲染组件方法，将该组件的配置信息放入模型中，即可在画布页面快速生成该组件。

当渲染完已生成的组件后，可以在可视化的配置界面修改组件的配置信息。当对生成的新组件保存后，会将新生成的组件信息将覆盖原来组件信息，完成对组件的重构。

3.3.5 扩展组件库方案

为了验证提出方法的可扩展性，本节在构建的模型中继续扩展集成其他组件。扩展的组件类型包括 Echarts、Plotly 及其他类型（如文本、视频等）的组件。

（1）扩展 Echarts 组件库

扩展其他类型的 Echarts 组件与上述构建 Echarts 折线图模板的方法相同，本文根据 Echarts 不同图表的特性，为每种图表模板构建了相应的可配置界面。

（2）扩展 Plotly 组件库

由于 Plotly 是基于 D3 的组件库，其生成组件的流程与 Echarts 有所不同。对应的，定义一个 Plotly 组件模板主要由四个部分构成：组件控制模块、组件挂载渲染模块、组件接口配置模块以及数据更新模块。如图 3-23，为各个模块之间的交互示意图。首先绑定数据模型到视图，当配置界面对配置参数发生修改时，接口配置模块绑定的数据模型会发生相应改变。此时数据更新模块监听到数据模型变化，会执行对应的数据更新方法，获取更新的配置数据（series、option）。

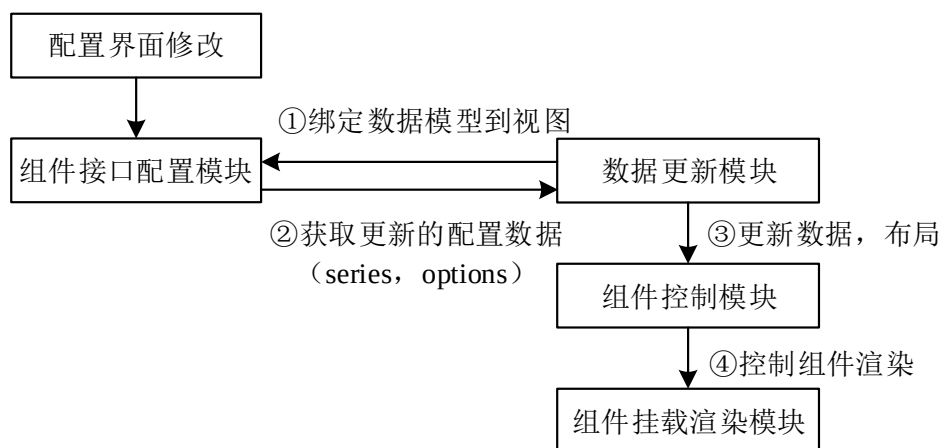


图 3-23 Plotly 图表模板各个模块之间的交互

组件控制模块加载更新后的数据，作为最新的 Plotly 图表的配置数据和布局

数据，并将更新后的数据作为参数，传入 Plotly 的绘图方法中，最后在页面重新渲染生成 Plotly 组件。

（3）扩展其他类型组件

由于文本、图片、视频是 HTML 原生的标签元素，因此构建这些组件的图表模板不需要引入额外的组件库。与 Echarts 组件模板类似，这些组件模板统一由四个模块构成：组件默认配置模块、组件挂载渲染模块、组件接口配置模块以及组件逻辑控制模块。这些组件的渲染流程与 Echarts 组件渲染流程类似，但是 Echarts 图表组件的渲染主要依靠底层调用绘图接口的指令实现，而视频组件或图片组件的渲染原理是基于上传视频或图片文件至服务器端后，向页面的标签元素（如 video 标签、div 标签）写入获取的该资源所在服务器端的 URL 值。

3.4 实验及结果分析

本章基于对组件库的研究，提出了一种基于模型的组件可重构生成方法。在本节为了验证提出方法和框架的实用性，首先，进行了扩展组件库的实验测试，并总结了本文基于模型的方法构建的组件类型和数量。其次，对组件生成和重构等功能进行了功能测试。接着，对组件的兼容性和渲染性进行了性能测试。最后，将本文构建的组件库与其他主流组件库进行了对比分析，分别测试了不同组件库渲染相同图表的初始化时间与动画帧率的表现情况，并对实验结果进行了总结分析。

在进行功能和性能测试时，所使用的测试机配置为：Intel(R)Core(TM) i3-2120 CPU@ 3.30GHz，内存为 12G，操作系统为 Windows 7（64 位），浏览器版本为 Chrome 9.0.4430.212。

3.4.1 扩展组件库实验结果

（1）扩展 Echarts 组件实验结果

目前，本文共构建了 24 种 Echarts 图表模板，如图 3-24 为构建的部分组件模板，分别代表了折线图、甘特图、3D 柱状图、饼图、极坐标扇形图和雷达图。

（2）扩展 Plotly 组件实验结果

目前，本文共构建了 6 种 Plotly 图表模板，包括折线图、散点图、柱状图、面积图、饼图、热度图。如图 3-25 为构建的 Plotly 部分组件模板展示。

（3）扩展其他类型组件实验结果

目前，本文一共构建了 6 种其他类型的组件模板，包括图片模板、视频模板、文本标题模板、图标模板、交叉表格模板、明细表格模板。如图 3-26 为本文构建的明细表和交叉表的表格模板。

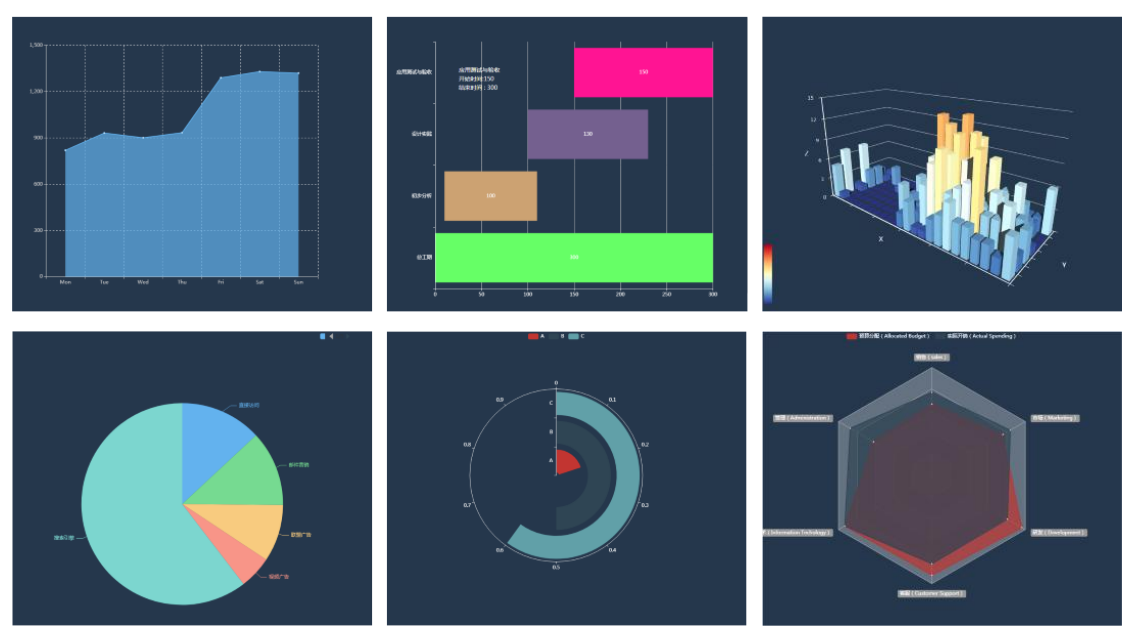


图 3-24 构建的 Echarts 图表模板

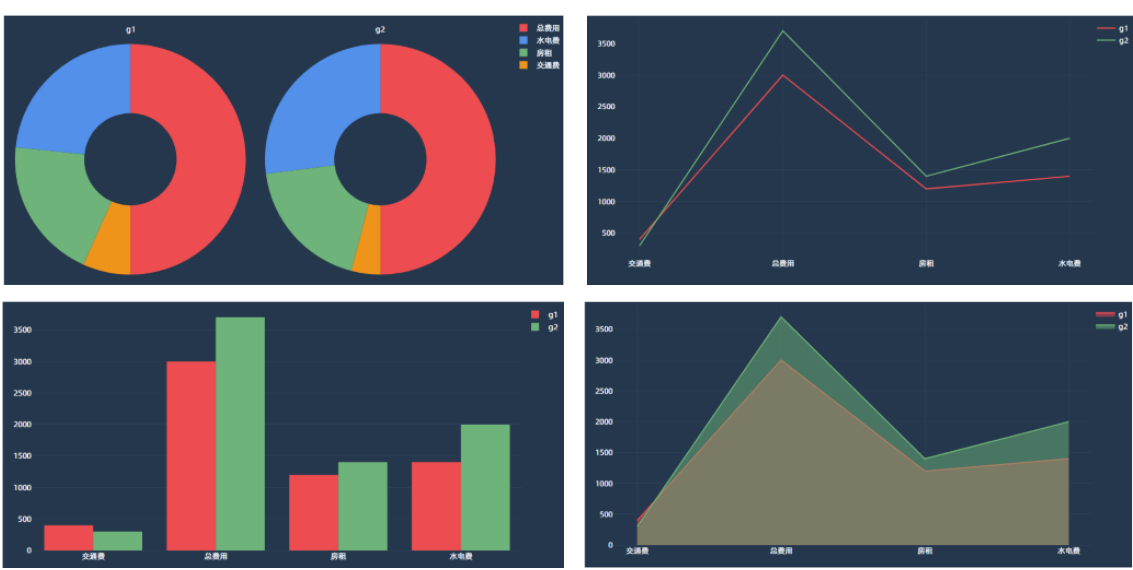


图 3-25 构建的 Plotly 图表模板

维度	max	min	组
总费用	3,000	1,000	g1
房租	1,200	200	g1
水电费	1,400	600	g1
交通费	400	200	g1
总费用	3,700	1,600	g2
房租	1,400	300	g2

维度 ▾

max ▾

min ▾

组 ▾

维度	交通费			总费用			房租			水电费			Totals
max	300	400	670	3000	3700	4000	1200	1400	2600	1400	2000		
min	100	200	330	1000	1600	500	200	300	1100	600	600	1200	
组	g2	g1	g3	g1	g2	g3	g1	g2	g3	g1	g3	g2	
Totals	1	1	1	1	1	1	1	1	1	1	1	1	12

图 3-26 构建的表格组件模板

总结本文扩展的组件库包含的组件类型和组件数量如下表 3-3 所示：

表 3-3 扩展的组件库包含的组件类型和组件数量

集成组件库	Echarts			Plotly	其他
集成组件类型	折线图	环形图	环比图	折线图	文本
	柱状图	玫瑰图	雷达图	散点图	图片
	面积图	过程图	饼图	饼柱状图	视频
	散点图	仪表图	扇形图	面积图	图标
	轨迹图	指标球	拓扑图	饼图	明细表
	区间图	试管图	甘特图	热度图	交叉表
	极坐标图	阶段仪表	3D 地图		
	多系列区间图	极坐标柱状图	3D 柱状图		
集成组件数量	24			6	6

根据实验结果可知，本文集成了不止一种类型的组件库，且集成的组件类型包括图表组件、文本组件、视频组件和图标组件等。因此，应用本文提出的组件生成框架，可以基于模型快速生成多种类型的可视化组件，能满足多方面的可视化需求。此外，通过扩展实验可知，本文构建的模型还具备可扩展的特点，对于其他组件库也可按照本文的思想在现有的技术上扩展和集成。

3.4.2 组件库的功能测试

（1）图表默认样式测试

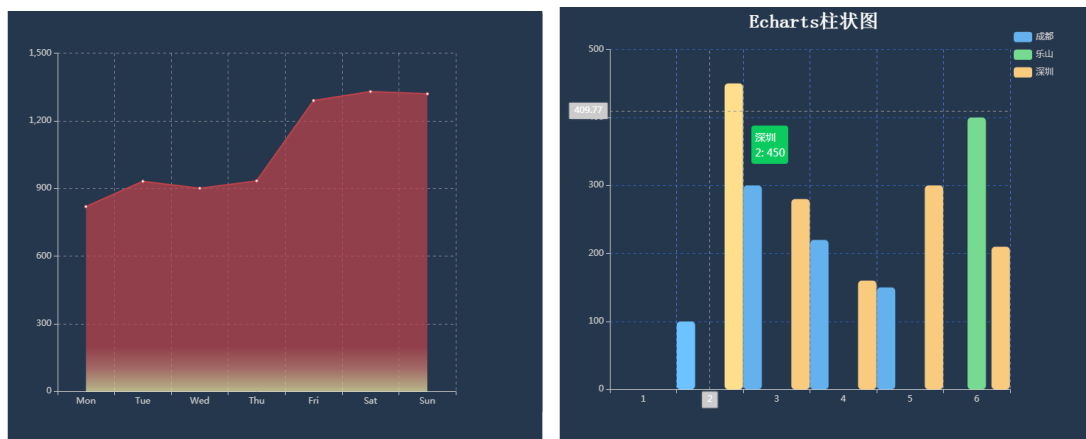
本文为构建 24 种 Echarts 图表模板，分别定义了默认的图表样式。由于在系统中做了代码封装处理，使用时不需要再引入 Echarts 相关库。在初始化情况下，如果选择模型后没有定义图表的配置数据，则会显示预设的主题数据和样式。以绘制基础的面积图为例，选择面积图模板，即可在画布页面渲染默认样式下的面积图，如图 3-27(a)所示为测试结果。

（2）图表属性配置测试

此外，为常用图表模板封装了 50 余种图表配置项，在基于模板构建组件时，能够根据可视化需求选择和配置相应的图表属性，进而快速渲染组件。图 3-27(b)为对 Echarts 柱状图组件同时配置了多种属性后的测试结果。配置项包括常用的：标题、网格线、提示框、图例、颜色、数据、系列 7 种属性。

(3) 图表预览测试

此外,为了对生成的组件进行统一管理,本文还构建了组件管理页面,将整体界面分为三个区域,左侧为组件列表展示区域,中间是选择对应组件的详细信息展示区域,右侧则是该组件的管理区域及预览区域。在组件管理界面中,可展示已创建的组件名称列表,并能选择查询指定组件的详细信息。



(a) 默认样式测试结果

(b) 属性配置测试结果

图 3-27 默认样式和属性配置测试结果

如图 3-28 为测试在组件管理界面查询创建的指标球组件,显示的该组件的详细描述信息和指标球组件的预览视图。



图 3-28 组件预览测试结果

3.4.3 组件库的性能测试

(1) 兼容性测试

本节通过集成的组件模板，构建了各类组件的示例，并在不同的浏览器上进行渲染测试，以验证图表组件的渲染效果是否与预期一致。如表 3-4 所示为本方法构建的组件库在各个浏览器的兼容性能表现。

表 3-4 兼容性测试结果

支持浏览器	本文构建的组件库
Chrome (90.0.x)	√
Firefox (88.0.x)	√
Safari (14.0.x)	√
Microsoft Edge (90.0.x)	√
Internet Explorer (9+)	√
QQ (10.7)	√
360 安全 (12.2.x)	√

根据测试结果可知，本方法所构建的组件库在主流浏览器上表现了良好的兼容性。使用 Chrome、Safari、Edge 及 IE9 及以上的浏览器打开包含生成组件的网页，测试结果表明网页布局和组件渲染的效果一致。

(2) 大数量渲染性能测试

在大数据时代，组件面临大数据量渲染的需求，因此还需测试在大数据量情况的渲染性能。在测试前，本文构建了包含 10000 条数据的数据集合。如图 3-29 为使用散点图组件渲染 10000 个数据点的效果。使用浏览器的开发者工具测试，图表的渲染时间是 0.7s，且无卡顿的情况，因此能提供良好的使用体验。

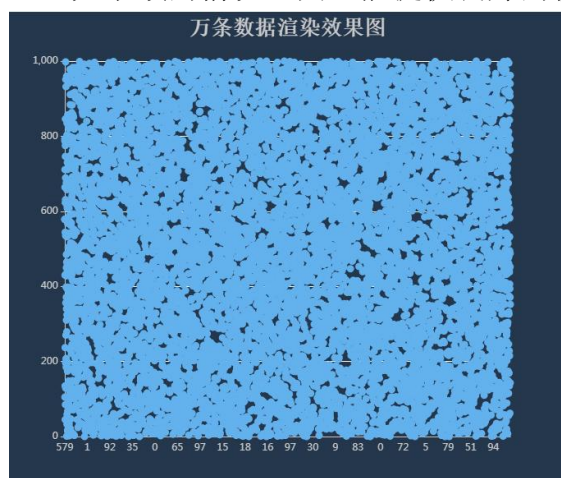


图 3-29 大数据量情况的渲染测试结果

（3）稳定性测试

在实际应用时，不仅要求生成的组件具有一定的美观性，也要求在构建组件时要保证一定的稳定性。由于本文集成了不同组件的模板，因此需要测试在基于不同图表模板生成可视化组件时，浏览器内存的占用情况与渲染时间。

测试前，本文制定了统一的数据源格式，部分字段如下表 3-5 所示，包括类别数据和数字类数据，数据条数为 20 条。我们将数据源以 Excel 形式存储在电脑端，并预先通过数据查询接口对数据源查询解析后形成数据查询集合，以供测试生成可视化组件时统一使用。

表 3-5 测试数据集的格式

序号	指标	值 1	值 2	值 3	分类
1	雷达防空	30	150	200	我方
2	远距离支援	20	170	250	我方
3	伴随掩护	44	180	360	我方
4	反辐射打击	50	200	180	我方
5	精导防护	60	250	230	我方

测试的操作流程是，首先接入数据集合，再选择构建的组件模板，然后在配置区域进行数据配置，完成数据配置后，画布页面将自动渲染出对应的图表。

在测试过程中，使用 Chrome 浏览器的开发者工具检测页面性能，记录完成数据配置和画布页面完全渲染组件的时间戳，以两者之差作为图表的渲染时间。同时，记录组件完全绘制时，当前浏览器的内存占用情况。如图 3-30 所示，为测试生成不同种类的组件，图表的渲染时间和当前浏览器的内存占用情况。

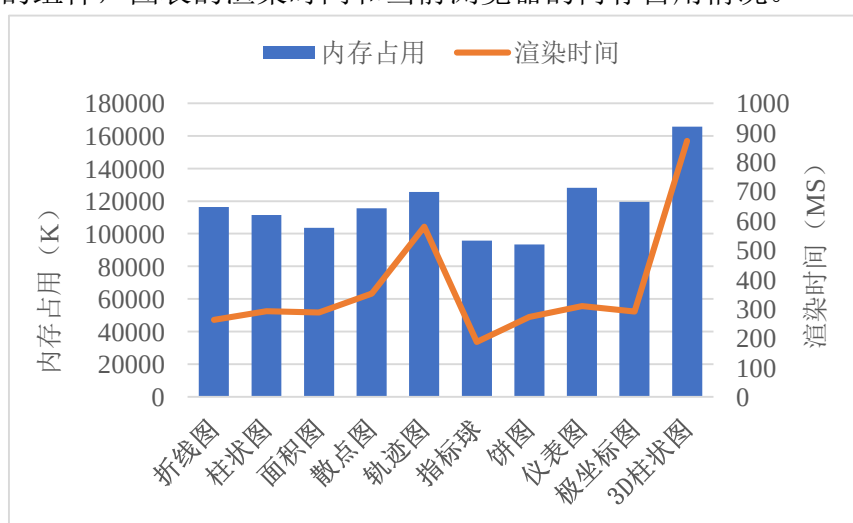


图 3-30 生成不同组件的性能测试结果

根据测试结果可知,生成不同图表组件的占用内存和渲染时间均不同。其中,3D 柱状图占用内存最多,渲染时间也最长,这是因为基于 Echarts-gl 进行三维渲染呈现 3D 效果的影响。轨迹图的渲染时间也相对较长,则是因为定制化实现的原因,轨迹图由折线和散点构成,因此需要先渲染出折线轨迹,再渲染出散点。

3.4.4 与同类组件库的比较

在本节中,我们将本文基于 Echarts 构建的组件库与其他 Highcharts、Chart.js 和 C3.js 组件库进行对比。这几个组件库都支持折线图、饼图及散点图等常用图表。其中,Highcharts 也使用声明性选项和数据系列,但是受到高度的限制。相比之下,本文提供了一种更灵活的方法,基于构建的图表模型,通过组装需要的组件配置来自定义设计和生成组件,允许用户以简单的方式创建可视化图表,并专注于可视化的设计。此外,丰富的内置模板以及可配置的选项提供了灵活的交互规范。

在扩展性方面,Highcharts 也受到一定限制,如 Highcharts 无法与在线地图库协同工作,而本文集成的组件库可以实现与百度地图等在线地图无缝集成。此外,Chart.js 和 C3.js 均不提供定制化的视觉配置,因而无法生成复杂的视觉效果。

从渲染机制方面分析,由于 Echarts 基于 ZRender 引擎渲染,使用 HTML5 画布呈现图表,因此可以实现高性能的渲染效果。此外,Chart 也提供了使用 Canvas 的方式渲染图表。而 Highcharts 和 C3 只能处理 SVG 的元素,基于 SVG 渲染的组件库,会在绘制 SVG 元素上消耗大量时间和内存。

根据以上讨论与分析,表 3-6 对比分析了本方法构建的 Echarts 组件库与其他组件库的区别。

表 3-6 不同组件库的对比

序号	组件库	可扩展图表	用户交互性	支持 Canvas
1	Highcharts	×	√	×
2	Chart.js	×	×	√
3	C3	×	×	×
4	本方法构建的组件库	√	√	√

根据表 3-6 结果可知,本文构建的组件库,提供了一种灵活的方法来支持图表设计和用户交互的定制,而其他类型的组件库不支持这些功能。

此外,本文还对这几种组件库渲染组件的初始化时间和动画帧率进行了实验测试分析。其中,初始化时间指的是图表从开始创建到渲染完成的持续时间。图表组件完全呈现意味着所有图形元素都已绘制在屏幕上,用户可以与页面进行交互。

在测试中,本文在 Chrome 浏览器中进行了实验,对于每个性能指标分别在不

同数据集的情况下生成了不同类型的图表，包括折线图、散点图和饼图图表，使用浏览器的开发者工具测试这四种组件库的性能。

如图 3-31 所示，为 Highcharts，Chart.js，C3 和采用本方法构建的组件库生成不同类型图表的初始化时间比较。

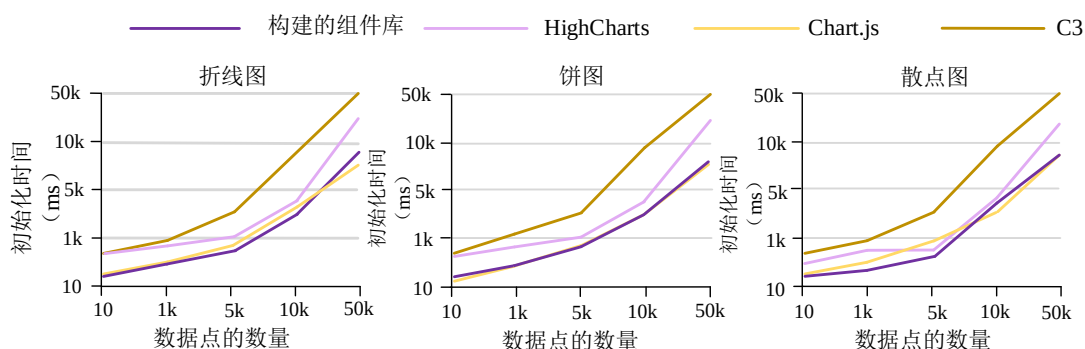


图 3-31 不同图表类型的初始化时间测试结果

图 3-31 显示了四种组件库的性能，其中本方法构建的组件库和 Highcharts、Chart.js、和 C3 相比，具有更短的初始化时间。分析原因可能是由于创建 SVG 元素导致 DOM 树的工作量过大，造成时间消耗更多。

动画帧速率（FPS）表示的是每秒钟画面更新次数。动画帧率指示着重绘图表的性能，这种性能对于交互过程中流畅的用户体验至关重要。本文启用图表的过渡动画，设置动画过渡的持续时间为 5 秒，并且每 5 秒钟更新一次图表配置数据，使用开发者工具记录平均的 FPS。

如图 3-32 所示，是 Highcharts，Chart.js，C3 和本方法构建的组件库生成不同类型图表的动画帧率比较。

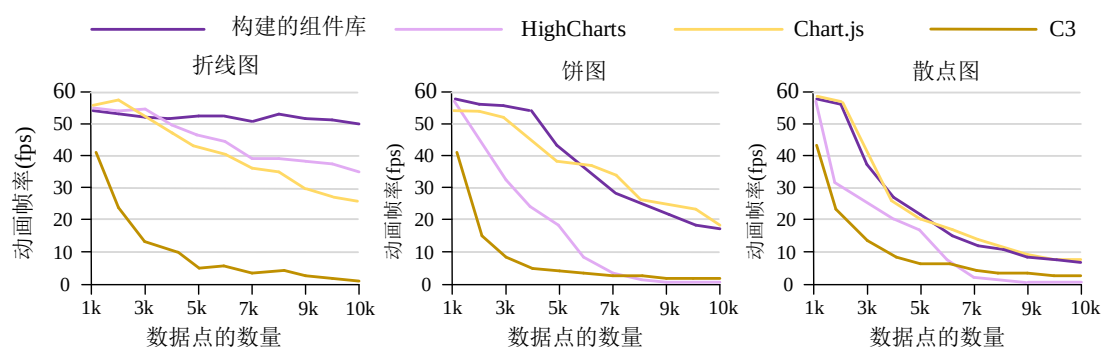


图 3-32 不同图表类型的动画帧率测试结果

根据图 3-32 的帧率测试结果可知，本方法构建的组件库与其他三种类型的图表库相比表现良好。分析这是因为 Echarts 基于 Canvas 画布进行绘制的影响，此

外 Chart.js 也具有相对较高的性能，而 C3 具有最低的帧速率。

性能测试实验表明，本方法构建的组件库在综合性能上优于其他三种工具。

3.5 本章小结

本章针对可视化流程中的组件生成流程复杂的问题，提出一种通过可视化界面简单配置生成组件的技术。

首先，本文分析了传统应用组件库生成组件的流程，并分析了此方式的缺点。接着，基于对已有组件库的研究，本文提出一种组件生成框架，以实现可视化组件的快速生成。接着，本文针对 Echarts 中各类型的图表，分别构建了相应的组件模板，并将图表模型集成到快速生成框架中，实现生成可视化组件的流程模型化。最后，引入可重构的概念，并构建了相应的组件配置与管理界面。

实验显示，本文提出的方法可以通过可视化配置实现快速生成可重构的组件，且生成的组件类型达 30 例，在集成组件的种类和数量上优于同类工具。此外，对比实验表明，本方法构建的组件库在性能上具有更短的初始化时间和更高的动画帧率。

第四章 基于页面自适应的大屏敏捷构建技术研究

本章针对可视化流程中的大屏构建流程复杂、大屏展现效果不理想的问题,提出一种基于页面自适应的大屏敏捷构建技术。此方法可以实现基于可视化组件动态构建自适应大屏,提高了构建大屏的灵活性与用户的交互性。本章的研究对于可视化大屏未来在各领域的应用实践提供了良好的思路。

4.1 传统大屏构建方式

目前,构建可视化大屏的途径一般分为以下三种:

一种是用户自己应用 JavaScript、VUE 和 Echarts 等前端开发技术搭建平台框架,这类方式开发大屏的周期一般较长,且构建的大屏功能单一,易用性和维护性较差;

第二种是寻求专业的集成商定制开发,这类开发的平台是根据特殊业务需求如(3D 建模)高度定制的,且通常基于 ventuz 或者 unity 等三维引擎技术。这类方式构建的大屏展示效果一般是最好的,但是成本开销极高;

第三种是购买或租用专业的商业智能(Business Intelligence,简称 BI)工具制作大屏,此类工具功能完备,已经搭建好完整的前后端数据连接平台,对外直接提供构建大屏可视化的“画布”,并内置了丰富的 BI 报表模板,只需简单的拖拽操作即可实现高效便捷的大屏设计模式。这种方式构建的大屏效果展示效果强大,上手较容易,可以大幅度节约构建大屏的时间成本,但是根据其功能的不同,此类工具的价格浮动较大,且存在扩展性不足的问题。

通过分析,前面两种构建大屏的方式都是基于代码开发,人工搭建平台实现大屏展现,且存在很多不足。而最后一种基于 BI 报表自主设计大屏的方式,以动态的设计模式构建大屏,更具有灵活性。下面介绍其中两种构建大屏的具体方法。

4.1.1 基于前端技术构建大屏

可视化大屏通常基于前端技术构建,如图 4-1 是基于 Echarts、VUE 等前端技术构建大屏的流程。这种基于技术开发代码来构建大屏的方式,优点是能较好地贴合业务需求,简洁方便地传递数据信息,适用于组件功能简单,页面布局要求不高的场景。但缺点是这类大屏不仅展示的图表种类和图表数量有限制,并且通常展示的是二维类型的简单数据,对于深层次、关系复杂的数据展示需求,具有易用性不足的问题。

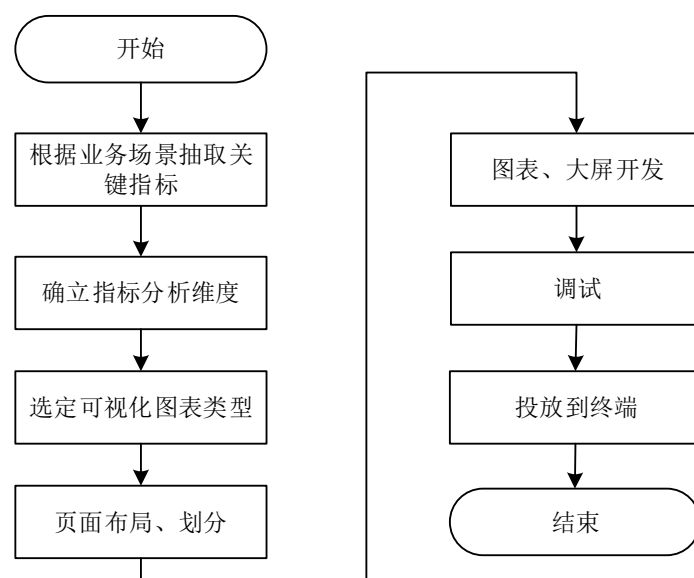


图 4-1 基于前端技术构建大屏的流程

4.1.2 基于 BI 报表动态构建大屏

基于 BI 报表动态构建大屏是借助已有的 BI 智能工具，在可视化界面按照流程设计和构建大屏。目前主流的 BI 工具国外有 Microsoft Power BI，国内有 Fine BI 等。以 FineBI 为例，构建大屏的流程如图 4-2 所示。

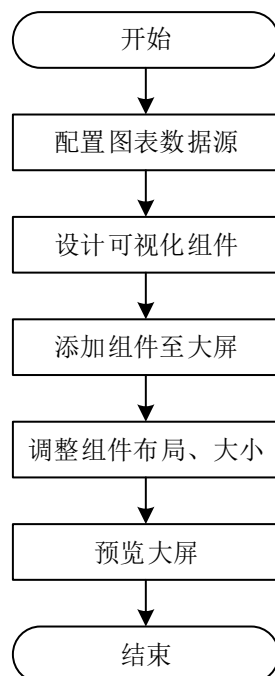


图 4-2 基于 BI 工具构建大屏的流程

借助 BI 工具构建大屏的方式，可以支持对多种数据源的数据进行数据分析处

理,且构建大屏的流程较灵活简便。但缺点是不支持在内置的组件基础上二次开发,扩展性不足。

4.1.3 分析小结

经过前期分析,通过 BI 工具构建大屏的方式更具有灵活性,因此本文拟采用这种动态构建大屏的思路,并基于第三章可视化组件生成技术的研究,提出基于页面自适应的大屏敏捷构建技术。

4.2 基于页面自适应的大屏敏捷构建技术

在本节中,提出一种基于页面自适应的大屏敏捷构建技术,希望通过该方法,基于生成的可视化组件,以动态的方式构建大屏,实现大屏的敏捷展现。

4.2.1 敏捷构建系统总体设计

4.2.1.1 系统模块划分

本文提出的基于页面自适应的大屏敏捷构建系统的模块组成如下图 4-3 所示。

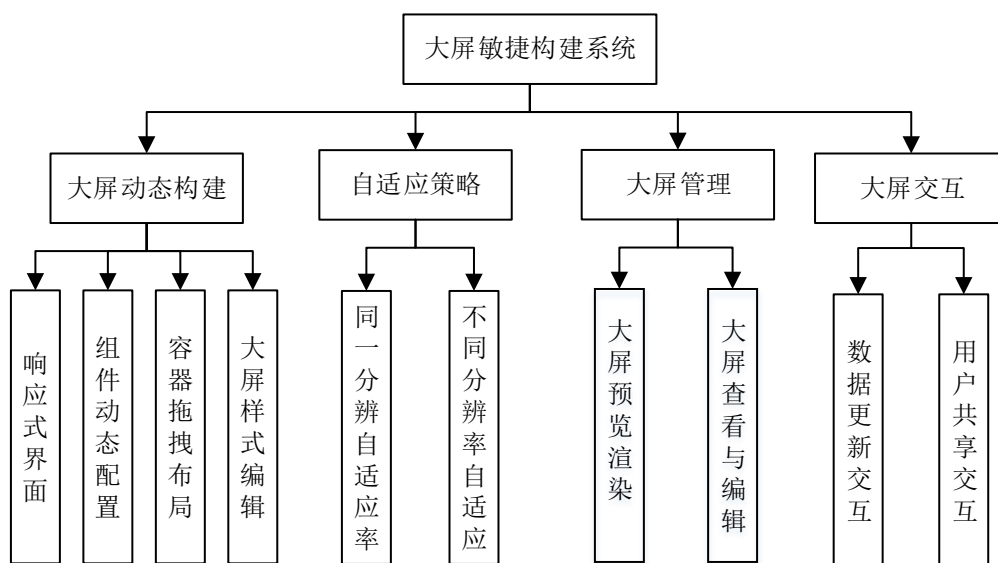


图 4-3 大屏敏捷构建系统的模块组成

其中,大屏动态构建功能划分为响应式界面模块、组件动态配置模块、容器拖拽布局模块和大屏样式编辑模块。自适应策略模块分为不同分辨率页面的大屏自适应和同一分辨率页面的大屏自适应实现。大屏管理模块划分大屏预览展示与编辑模块。大屏交互模块划分为数据更新交互和用户共享交互模块。

4.2.1.2 整体架构

如图 4-4 所示为构建的大屏系统的架构图，系统由数据库实体、数据服务层、逻辑控制层及视图层构成。

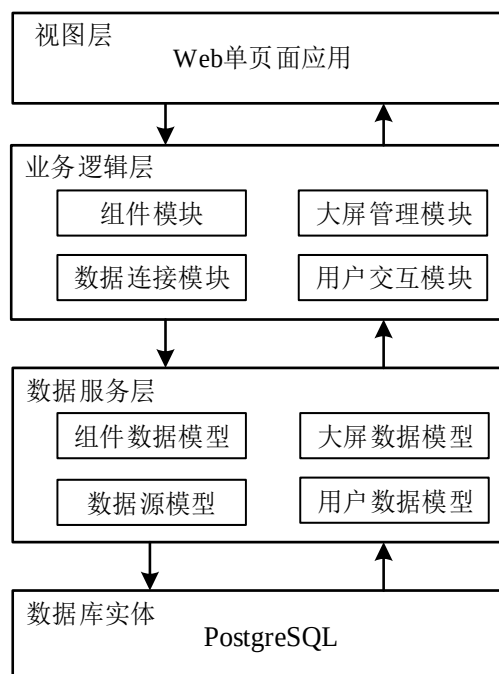


图 4-4 系统的整体架构图

视图层：设计系统前端界面以 SPA（Single Page Application, 单页面应用）形式呈现。基于 SPA 实现的应用程序只有一个主界面，通过用户交互不断向服务端获取数据，以动态渲染该页面呈现的效果。SPA 基于浏览器运行，加载速度相比于桌面应用程序更快，且无需安装，因此具有较好的用户操作体验。在本系统架构中，视图层直接与用户交互，并以 HTTP 的形式与逻辑控制层通信。

业务逻辑层：集中处理接收到的 HTTP 请求。当视图层发起新建、编辑、删除大屏等请求时，逻辑控制层接收这些请求并集中处理。

数据服务层：为逻辑控制层提供数据支持，这些封装好的数据模型为系统提供了方便的接口服务。

数据库实体：使用 PostgreSQL 数据库存储所有数据。

4.2.1.3 前端设计

本系统的前端架构设计如图 4-5 所示，系统的前端架构主要依托于 Angular 框架技术栈，其中 Angular 组件负责构建大屏系统的视图层。

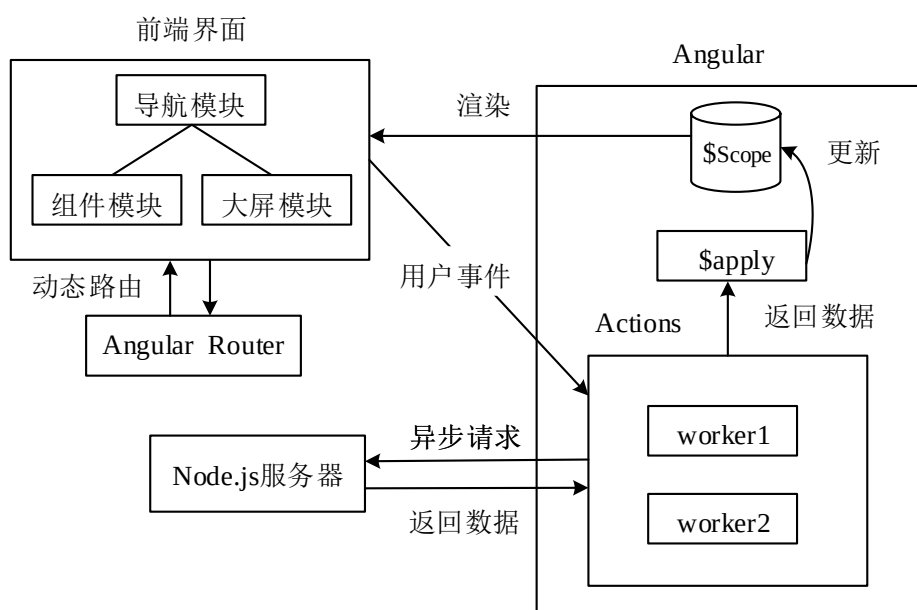


图 4-5 系统前端架构图

在架构中，系统中的数据流由 **Scope** 作用域进行统一管理。前端界面发起用户事件，**Actions** 是对用户事件的响应，负责向服务器端发送异步请求，获得查询的数据。当数据更新时候，会通过 **\$apply** 启动“脏循环”检查机制，“脏检查”机制启动后会检查每个 **Scope** 内的数据模型是否发生了变化。**Scope** 包含了全部的数据信息，**Scope** 绑定的数据更新时会导致界面的更新渲染。

此外，由 **Angular-Router** 实现在单页面的路由跳转。

4.2.1.4 后端架构

系统服务端采用 **Python** 开发，以 **Flask** 框架为 **Web** 框架基础。其中，**Flask** 是一个轻量化的程序框架，具有易于扩展等优点。如图 4-6 为系统的后端架构图。

如图 4-6 所示，前端通过 **Flask-RESTful** 与服务端进行通讯。其中，**Producer** 负责生成任务并传递给任务队列。**Beat** 是负责读取任务，并周期性传递任务给消息中间件。**Broker** 作为消息代理，负责把接收的任务按一定的顺序分发给任务消费方。**Worker** 负责执行任务，当多个 **worker** 共同运行时可以提高任务的处理效率。

数据查询引擎通过 **Flask-SQLAlchemy** 与各个数据源进行交互，并将任务处理后的查询结果存储至 **PostgreSQL**，便于下次前端查询时直接将结果返回给前端而无需再次从数据源进行数据查询。

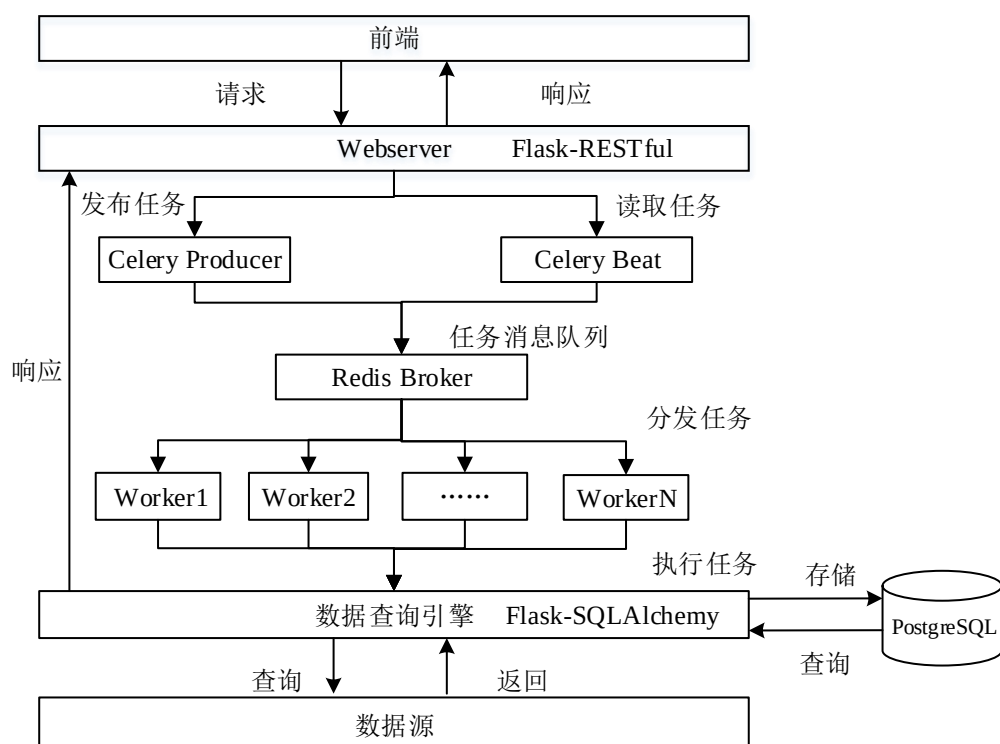


图 4-6 系统后端架构图

4.2.2 大屏动态构建

4.2.2.1 模块构成

如图 4-7 所示为本系统的大屏动态构建功能的模块组成，将详细介绍每个模块的实现功能。

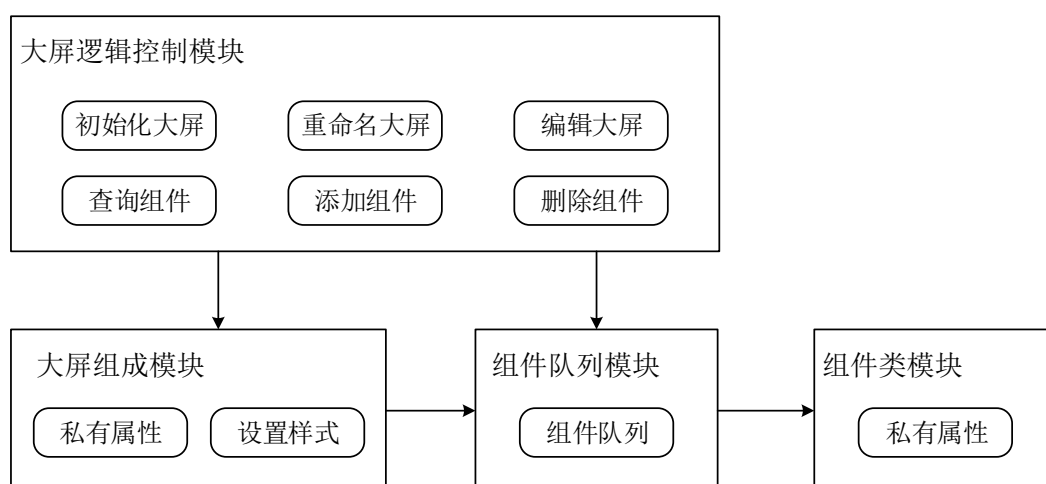


图 4-7 大屏动态构建功能模块组成

大屏控制模块包含了基于组件动态配置大屏的方法，如初始化大屏的基本信息，重命名大屏名称，设置大屏的描述信息。也可以查询创建的全部组件列表，添加指定的组件至大屏或移除大屏中指定的组件。

大屏组成模块定义了大屏的数据模型，每个构建的大屏都包括 ID、名称等私有属性。当建立新的大屏时，都会实例化该模块，并调用大屏控制模块的初始化大屏方法，设置大屏默认名称。此外，还可以设置大屏样式，如自定义上传背景图片，背景图片设置方式有填充、平铺、以及拉伸三种。

组件队列包含了添加至大屏中组件。组件队列模块依赖于组件类，其队列中存储着添加的每个组件的信息。

组件类是组件的基础类，对应生成的组件的对象原型。与第三章组件的定义结构一致。

大屏编辑界面是大屏动态构建方法的可视化实现，因此需要提前对界面进行设计和布局。如图 4-8 为大屏编辑界面的设计原型图，下面具体介绍该原型图的布局。界面划分为四个组成，分布是左侧的导航栏区域、上方的大屏管理区域、下方左侧的控制面板区域以及下方右侧可视化大屏编辑区域。

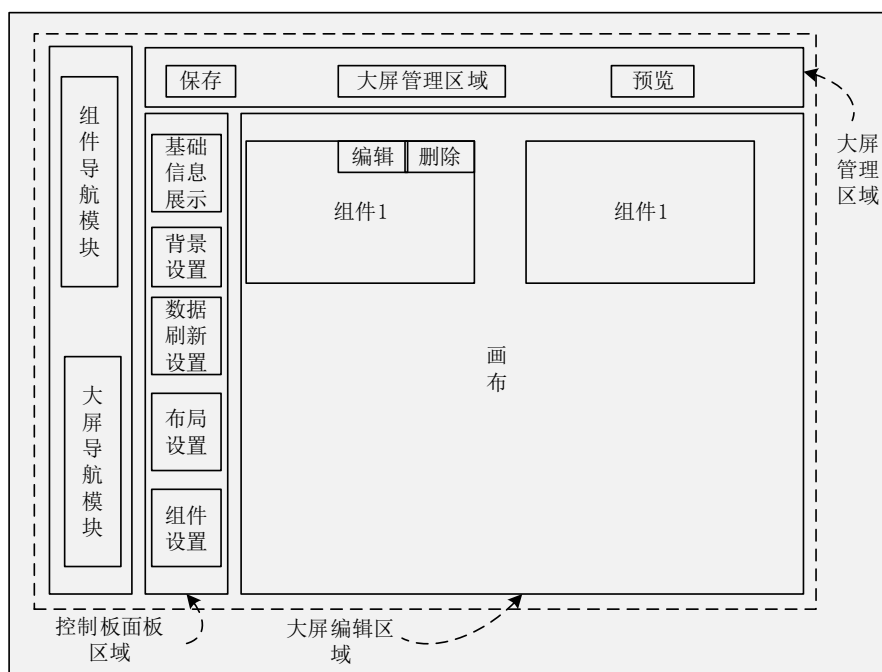


图 4-8 大屏编辑界面的原型图

在导航栏部分中，按照常规的界面设计模式，可选择导航到组件模块或大屏模块。如果导航到大屏模块，则会出现控制面板区域、大屏管理区域和大屏编辑区域。控制面板区域又分为五个模块。其中基础信息展示模块用来展示大屏的名称，主题

设置模块用来设置大屏的背景图片及图片填充样式，数据刷新设置模块用来设置大屏内数据刷新的周期。布局设置模块用来设置组件的布局排列样式。组件设置模块是整个核心模块之一，用来查询和添加可视化组件至画布区域。

右侧的画布区域模块除了展示已添加的组件样式，还可以编辑与删除每个组件。编辑组件时，会自动链接到该组件的编辑界面，删除组件可将该组件从画布中移除。

顶部是大屏管理区域，用于对构建好的大屏进行保存和预览。

4.2.2.2 模块具体实现

本节基于对于大屏动态构建模块的划分，介绍动态构建功能的具体实现过程。

(1) 响应式布局实现

根据对大屏动态构建界面的原型设计，整个界面被划分成为四个部分，界面左侧是系统的导航栏，右侧又划分为三个部分，上方的大屏管理区域，以及下方左侧的控制面板区域和下方右侧的大屏编辑区域。通过分析可知，该界面可能应用在不同屏幕分辨率的终端，因此可以采用响应式布局方案。

对于本界面所需要的响应式布局效果，本文采用 HTML+CSS 布局中的 Flex 布局。如图 4-9 所示，为 Flex 布局的结构，Flex（Flexible Box，弹性布局）通过一维的排列方式，使页面的盒子模型灵活在容器内灵活布局。该布局的原理是当标签元素应用 Flex 布局时，把元素当作容器（container）。把标签内的容器成员当作项目（flex item）。由于所有子元素均在父级容器内，因此可以灵活对齐排布。

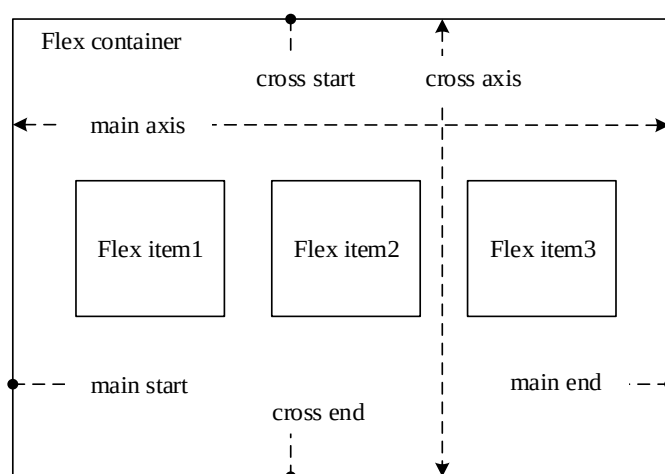


图 4-9 Flex 布局结构

(2) 组件动态配置

在组件动态配置功能中，要实现在编辑界面点击添加组件后，在画布页面实时

渲染出动态配置的组件。本文集成的 Echarts 类型的组件采用 canvas 渲染, 扩展的 Poltly 组件基于 svg 进行渲染, 其他组件则基于 html 的属性直接在容器内渲染。

第三章中完成了对可视化组件的封装, 这一部分的关键是动态配置组件的具体实现。如图 4-10 为基于组件动态配置大屏的流程图, 展示了添加组件至渲染组件的完整流程。其中, 组件队列存储了该大屏已添加的所有的组件 id, 当添加一个组件至大屏之前, 会先判断该组件 id 是否存在, 如果已经存在则不再添加。

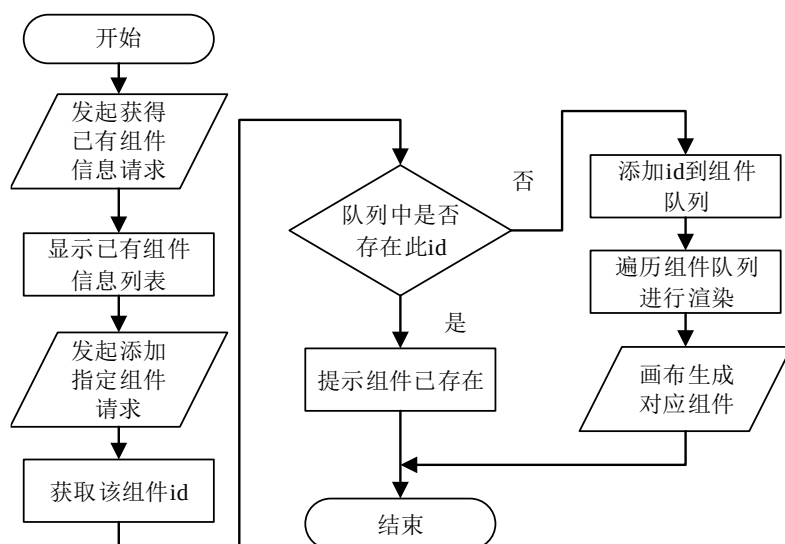


图 4-10 组件动态配置实现流程

根据流程图可知, 该流程的核心是画布渲染显示对应组件的过程, 当发起添加组件到大屏的请求时, 组件队列的数组会更新保存该组件的 ID。此时由于数据更新, 大屏将重新渲染, 首先会遍历组件队列每个组件的 ID, 获取相应的组件信息。对于每个封装好的组件, 已经生成相应的渲染模板。因此只要把渲染模板注入到渲染容器中, 即可基于 canvas 渲染相应的 Echarts 图表组件。

(3) 组件的拖拽布局实现

要实现大屏动态构建功能, 还需要实现在画布页面拖拽组件进行布局。在可视化大屏的展示过程中, 拖拽是一种提高用户交互性基础方式。基于构建的大屏系统, 用户可以面向实际的可视化需求, 通过拖拽布局组件实现个性化的大屏设计。

实现拖拽布局容器的关键是应用浏览器对象模型 (BOM) 和文档对象模型 (DOM)。主流的组件拖拽布局有两种解决途径, 下面分别进行介绍。

方案一: 在 HTML5 标准出现之前, 利用事件对象中的调度机制, 监听到鼠标点击事件后, 在回调函数中记录拖拽的状态。

该方式通过对鼠标操作提前添加监听事件, 当鼠标在拖拽过程中进行“按下”、“移动”、“抬起”操作时, 基于浏览器端的事件机制, 会分别触发相应的“拖拽开

始”、“拖拽中”和“拖拽结束”事件，并进一步的在事件处理函数中，更改拖拽元素的偏移量，实现拖拽元素的移动效果。

方案二：在出现 HTML5 规范后，对元素添加了拖动属性

区别于对鼠标添加监听事件，该方式主要是赋予对被拖拽的元素一个拖动属性，当该属性值设置为 false 的情况下代表此元素禁止被拖动。当该属性值为 true 时，该元素自动转为可拖拽的状态。同时，在拖拽中也会有相应的回调事件被触发。

通过分析两种解决方案，由于 HTML5 新增的属性在不同浏览器实践效果不同，为了提高展示结果在浏览器的兼容性，实现统一稳定的布局效果，本文采取第一种方式。在 HTML 标准的事件对象中，当触发鼠标点击事件后，在回调函数内部会传入关于鼠标信息的属性，如鼠标当前点击位置的坐标等。

实现拖拽布局功能主要利用了 client X, clientY 和 offsetX, offsetY 等属性值，各属性值的含义如下：

1) clientX, clientY：指当前鼠标位置相对浏览器可视窗口区域的坐标。client X 代表鼠标相对于窗口左边界偏移量，clientY 则是相对可视窗口上边界的偏移量。

2) offsetX, offsetY：包含鼠标移动的重要信息。指的是鼠标当前位置相对于被拖拽元素的 X, Y 偏移量。

实现原理如图 4-11 所示，当鼠标拖拽元素开始时，触发“鼠标按下”事件，并在事件回调处理中，记录 offsetX 和 offsetY 属性的值。当拖拽元素移动时，会触发“鼠标移动”事件，并在回调函数中记录当前 clientX 和 clientY 属性值。最后计算 clientX 减去 offsetX 的值，将结果赋值给拖拽元素的 top 属性，即可实现元素在 x 轴方向的移动。同样的计算 clientY 与 offsetY 的差，赋值给元素的 left 属性，实现元素在 y 轴方向的移动。

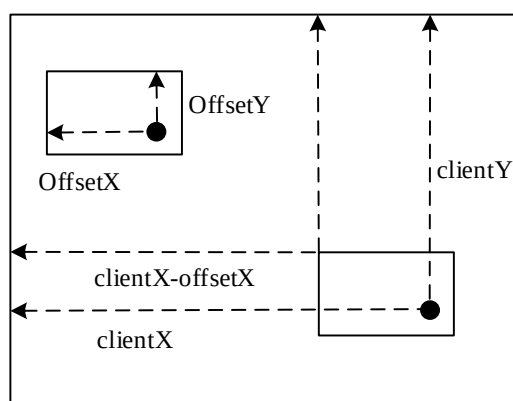


图 4-11 拖拽实现原理

(4) 大屏样式编辑

大屏背景样式影响着大屏直观展示的效果，因此有必要对大屏增加样式编辑

功能，具体实现流程如图 4-12 所示。其中，图片填充方式有三种，分别是填充、平铺和拉伸。设置图片填充方式是基于 CSS 的背景相关属性进行设置。

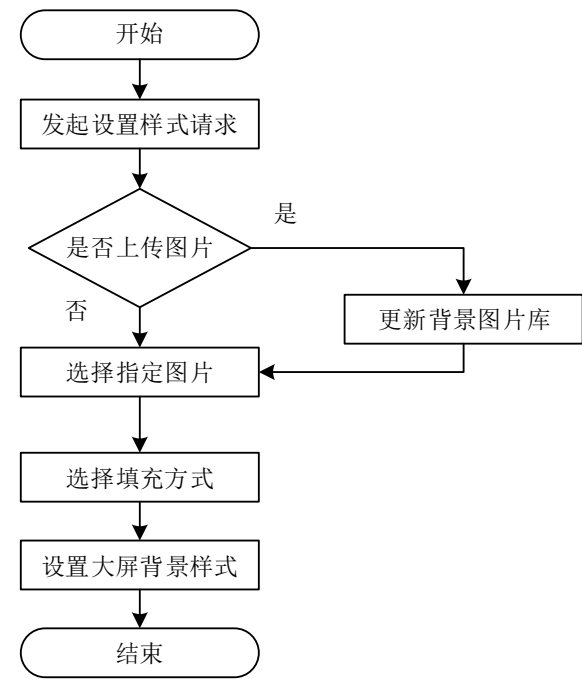


图 4-12 大屏样式编辑流程

4.2.3 自适应策略

在实际应用中，可视化大屏面临在复杂多样的环境中展现的需求。因此实现可视化结果在不同终端的自适应展示也是渲染页面时必须考虑的因素。由于不同终端的屏幕分辨率不同，因此需要从同一分辨率的结果自适应和不同分辨率的结果自适应去考虑。如图 4-13 所示，本文基于不同场景设计了三种自适应策略。

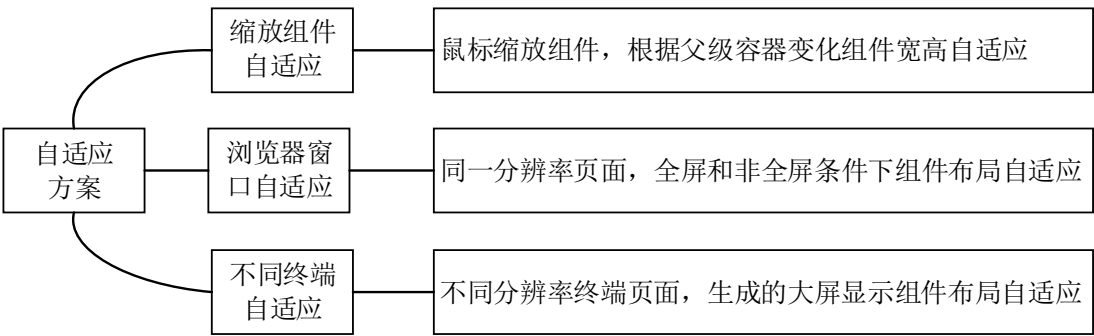


图 4-13 自适应策略

(1) 缩放组件自适应

缩放组件自适应预期实现在组件编辑页面，用鼠标手动缩放组件时，组件展示

的区域能随着容器拖拽自适应调整宽高。

此部分的实现思路是,在渲染组件区域的外层包裹一个容器,并设置容器属性为可调整(resizable),当在编辑界面拖拽容器时,容器的宽高属性发生变化。此时组件渲染区域监听到父级元素的宽高发生变化,自动与父级元素宽高 100%匹配,即可实现缩放组件时的自适应变化。

(2) 浏览器窗口自适应

浏览器窗口自适应预期实现在浏览器窗口全屏和非全屏模式下,打开大屏预览页面,页面内的所有组件能自动布局,并铺满整个可视化窗口。其中,实现思路分为从上到下的布局自适应和从左到右的布局自适应。

在从左到右的渲染中,采用“栅格”的思想,将浏览器可视页面按列划分为 12 份(即 12 个 column),每个组件的父级容器至少占据 1 份 column,至多占据 12 份 column。构建大屏时,只保存每个容器占据的 column 数量。因此,在全屏和非全屏模式下,打开大屏预览页面,渲染引擎会读取服务端每个容器保存的 column 数量,并将当前可视窗口区域划分成 12 份,按各自的占比设定父级容器的宽度渲染。此时组件通过监听父级容器宽高,自动与父级元素的宽度 100%匹配。

“栅格”是按列划分的思想,可以实现宽度自适应,因此需要单独对组件渲染区域的高度重新计算。当得到组件渲染区域的宽度时候,设定渲染区域的高度随宽度动态变化。由于矩形的黄金比例是高度与宽度之比为 0.618:1。按照这种思想,也可实现组件高度的动态变化。

从上到下的渲染,主要考虑当组件在大屏内上下布局时,鼠标放大上方区域组件的容器高度时,下方组件的位置应自动往画布下方平移。反之当缩小上方区域组件的高度时,下方组件的位置应自动往画布上方平移,以实现整体布局的自适应,

至此,通过组件渲染区域宽高的自适应变化,即可实现组件随浏览器窗口调整的自适应变化。

(3) 不同终端自适应

不同终端页面自适应预期实现在不同分辨率的终端页面预览大屏时,大屏内的组件布局能自适应铺满当前窗口页面,并能自适应调整容器宽高。此部分的实现思路与第二种方案类似,区别是在不同终端预览大屏时,需设置当前浏览器的可视窗口为全屏模式,即可继续按照第二种方案进行自适应渲染。

本文分别对三种自适应策略进行了实现。在构建时,需要将各组件的最外层容器作为父级容器,当父级容器随着页面屏幕分辨率变化时,内部组件按照上述自适应方案自动调整组件的宽高,实现结果自适应呈现。为验证大屏页面的自适应实现效果,进行了自适应功能的测试实验,将在 4.3 节详细说明。

4.2.4 大屏管理机制

在大屏管理模块，新建大屏时通过控制模块的初始化方法进行大屏页面的初始化流程。编辑大屏时通过重命名方法对大屏进行编辑，或通过组件的相关方法对大屏进行管理。同时，为了区分大屏管理与大屏编辑，单独构建了大屏管理界面，其界面原型图如图 4-14 所示。

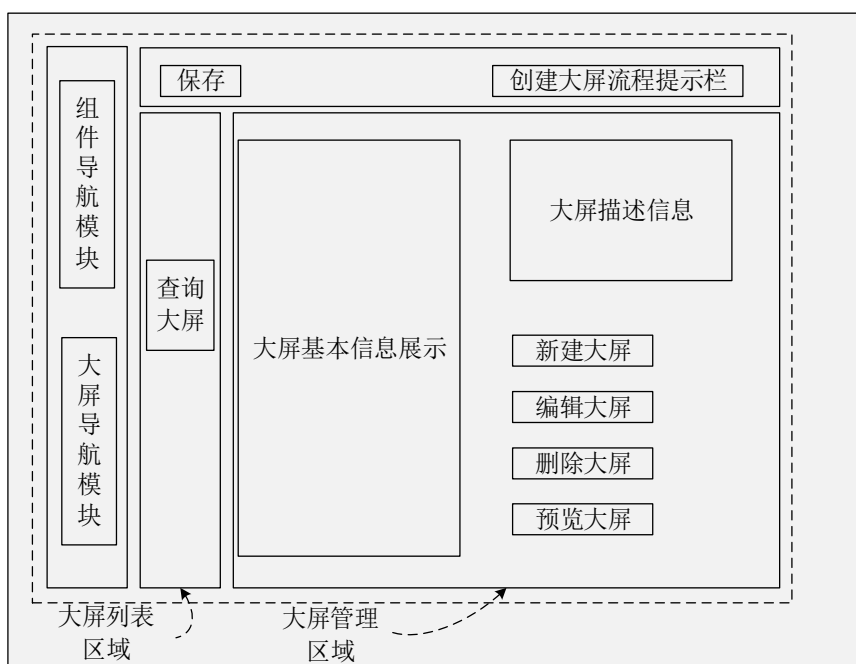


图 4-14 大屏管理界面原型图

大屏管理界面的布局划分与大屏编辑界面类似。点击大屏导航模块时，首先渲染右侧的大屏列表区域和空的大屏管理区域。当查看指定大屏的详细信息时，在管理区域生成对应的大屏信息。大屏管理模块需要提供对大屏的管理操作，如新建大屏、查询大屏、编辑大屏、删除大屏以及预览大屏等功能。

（1）新建大屏

为对大屏操作进行统一的管理，将新建大屏划分到大屏管理中。用户在新建大屏时，必须填入该大屏的名称。

（2）查看大屏

用户通过大屏管理查看已构建的大屏，以大屏列表的提供给用户，用户可在权限范围内选择列表中的大屏查看详细的信息。

（3）编辑大屏

用户通过编辑大屏对大屏的名称和描述等基本信息进行修改，或是在大屏编辑界面实现对大屏内组件的操作。

（4）删除大屏

提供删除大屏的功能，以回收不满足需求的大屏，并且在删除时通过二次确认降低误删大屏操作带来的危害。

4.2.5 大屏交互

大屏交互涉及大屏内数据交互和用户交互，本节将说明数据更新和用户共享的具体实现。由于大屏内的数据源通常来自数据库，当数据库数据发生更改时，需要在大屏更新变化后的数据。本节数据交互的实现是设置数据刷新周期后，大屏内的组件数据可以实时更新，表现为组件在大屏内的实时动态变化。

大屏框架基于 Angular JS 实现，因此本文采用 \$resource 服务与服务端进行数据更新交互。\$resource 服务通过建立资源对象，与使用 RESTful 的服务端通信。其中，RESTful 服务是采用 HTTP 并基于 REST（Representational State Transfer，表征状态转移）规范的 Web 服务。本文构建的数据交互实现流程图如图 4-15 所示。

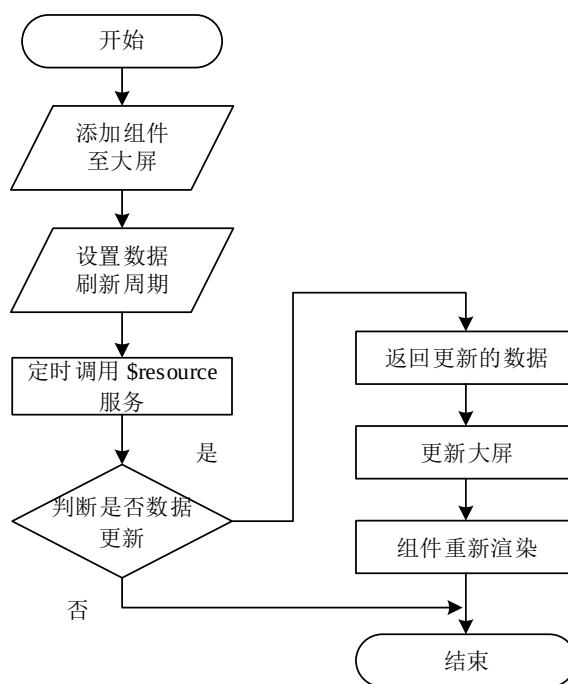


图 4-15 数据交互的实现流程

首先添加可视化组件至大屏，并设置数据刷新周期。此时大屏内管理模块根据数据刷新周期，定时调用 \$resource 服务与后端进行数据交互。如果此时数据发生变化，则服务端返回更新后的数据，并更新大屏内组件的数据配置，最后重新渲染大屏内的组件，实现大屏内的数据更新交互。

对于构建的大屏，用户面临资源共享的交互需求。实现大屏共享的核心思想是

为每个构建的大屏生成专属的共享 URL，其他用户可以通过该 URL 直接访问查看大屏资源信息。该 URL 的组成为前缀加后缀的结构，前缀为当前浏览器的域名，后缀为服务端生成的密钥。本文构建的用户交互实现流程如图 4-16 所示。

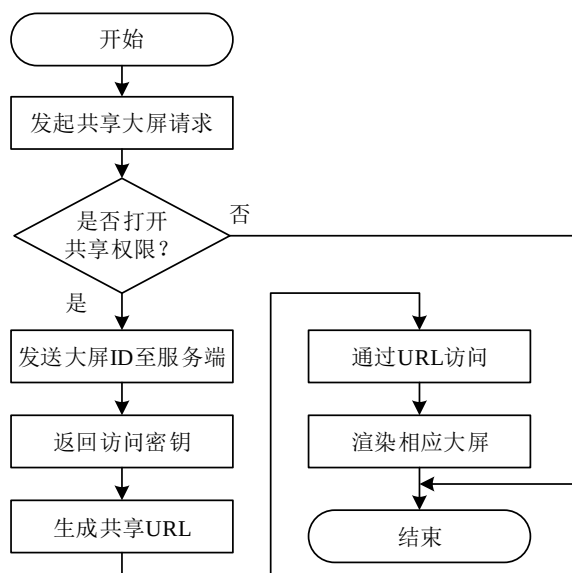


图 4-16 大屏共享解决方案

根据流程图可知，按照需求构建大屏后，可发起大屏共享交互的请求。如果共享权限为打开状态，则服务端根据当前大屏的 UUID（Universally Unique Identifier，通用唯一识别码）随机生成唯一的密钥。然后调用大屏发布的 API，传入该密钥作为参数，最后生成该可视化结果的专属 URL 链接。其他用户可直接在浏览器通过该 URL 对相应资源进行访问，无需登录。

4.3 实验及结果分析

随着可视化系统在实际应用中的实践，对相关系统平台进行有效测试也越来越重要。系统功能和性能的测试结果，为平台的应用实践提供了有力的支持。本节对构建的大屏系统从多方面进行完整的评测，测试构建的大屏系统在功能和性能的表现，以验证本文开发的大屏敏捷构建工具的实际应用性。

在测试本文构建的大屏系统中，所使用的测试机配置与第三章描述相同

4.3.1 功能测试

功能测试是对系统平台预期实现的功能进行验证。在测试中，对同一功能进行反复验证测试，并且测试点需覆盖系统所有的功能。

（1）大屏构建功能测试

大屏动态构建功能包含了编辑大屏和编辑组件的功能。使用者只需在可视化的编辑界面以鼠标点击和拖拽的方式完成可视化大屏的构建。

反复对大屏构建功能进行测试，如表 4-1 所示是对大屏构建功能的测试结果。

表 4-1 大屏构建功能测试结果

测试模块		测试功能	测试点	测试结果
大屏构建	新建大屏	输入大屏名称	设置大屏的名称是否生效	与预期相符
		设置大屏主题样式	设置大屏内的背景图片和样式是否生效	与预期相符
		保存大屏	是否在管理列表显示已有大屏	与预期相符
	编辑组件	添加组件	组件添加到大屏是否生效	与预期相符
		调整组件	鼠标拖拽改变组件的位置和大小是否生效	与预期相符
		移除组件	鼠标移除大屏内的组件是否生效	与预期相符
		编辑组件	鼠标点击是否可以进入指定组件的编辑界面	与预期相符

测试结果表明，可视化大屏的构建功能均达到预期。如图 4-17 所示为通过大屏构建功能模块构建的大屏效果图。

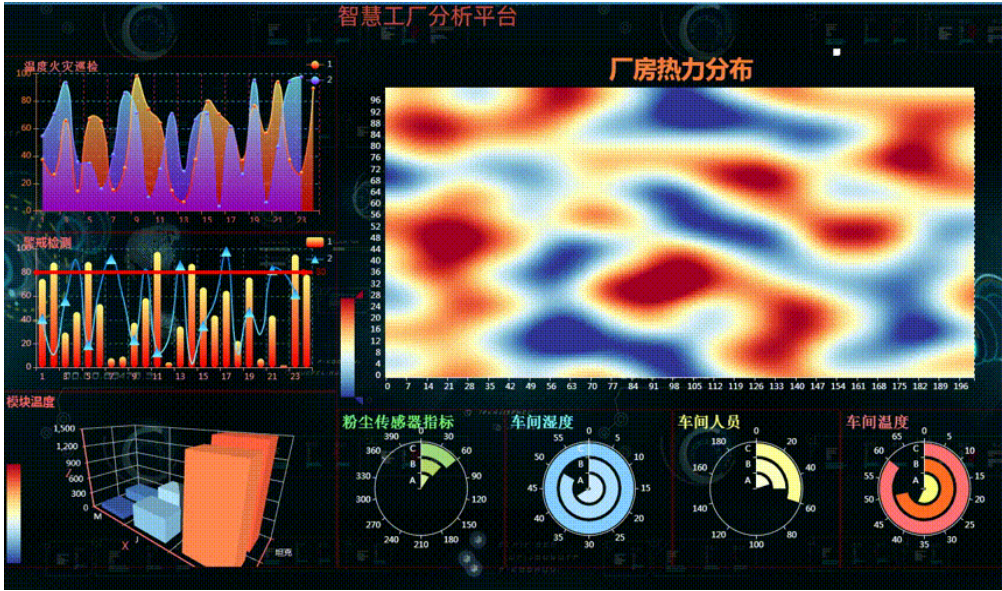


图 4-17 构建的完整大屏实现结果

(2) 大屏管理功能测试

大屏管理模块，提供在大屏管理界面中编辑可视化大屏的功能。反复对大屏管

理功能进行测试，如表 4-2 所示是对大屏管理功能的测试结果。

表 4-2 大屏管理功能测试结果

测试模块	测试功能	测试点	测试结果
大屏管理	大屏重命名	对大屏重命名是否生效	与预期相符
	大屏信息修改	修改大屏的描述信息是否生效	与预期相符
	编辑大屏	对创建的大屏再编辑修改，可视化视图是否生效	与预期相符
	删除大屏	删除可视化大屏是否生效	与预期相符
	预览大屏	预览生成的大屏是否生效	与预期相符
	发布大屏	是否可以生成大屏的 URL	与预期相符
	共享大屏	是否可以通过生成的 URL 访问大屏	与预期相符

测试结果表明，可视化大屏的管理功能均达到预期。如图 4-18 所示为测试在浏览器页面打开共享链接后的大屏共享结果。

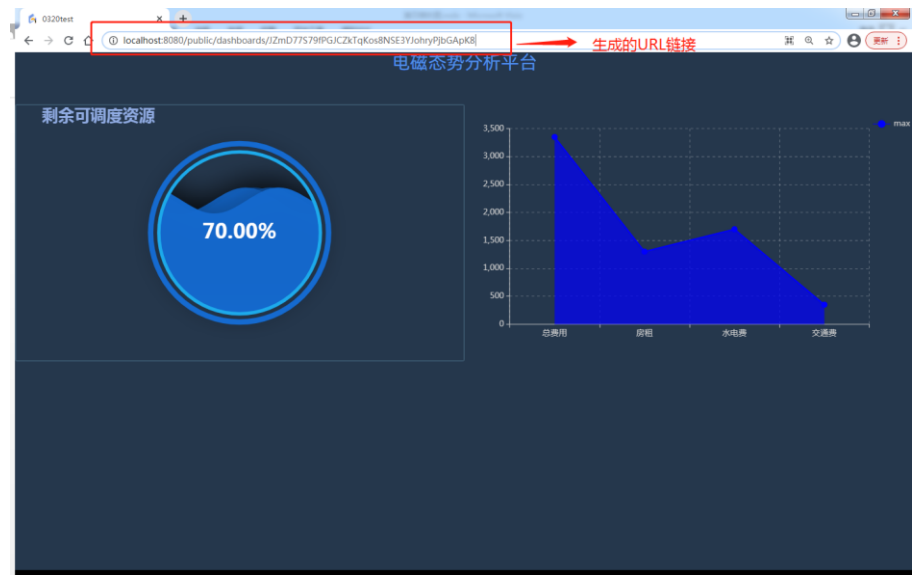


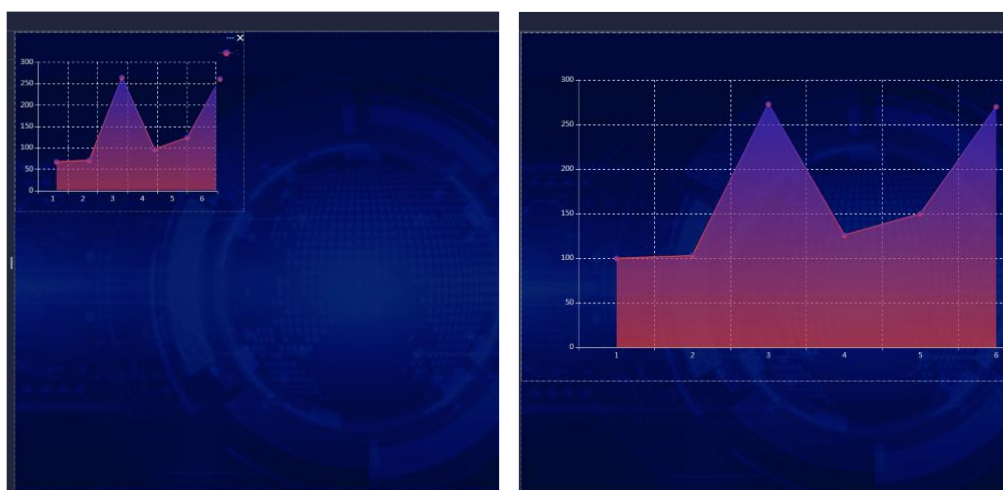
图 4-18 在浏览器打开 URL 后的大屏共享结果

(3) 大屏自适应功能测试

为了验证可视化大屏的自适应功能，按照设计的三种自适应策略进行测试。

测试一：在大屏画布页面，拖动组件边框。

如图 4-19 分别为在画布页面拖动组件前、后的组件大小变化情况。



(a) 拖拽前组件的大小

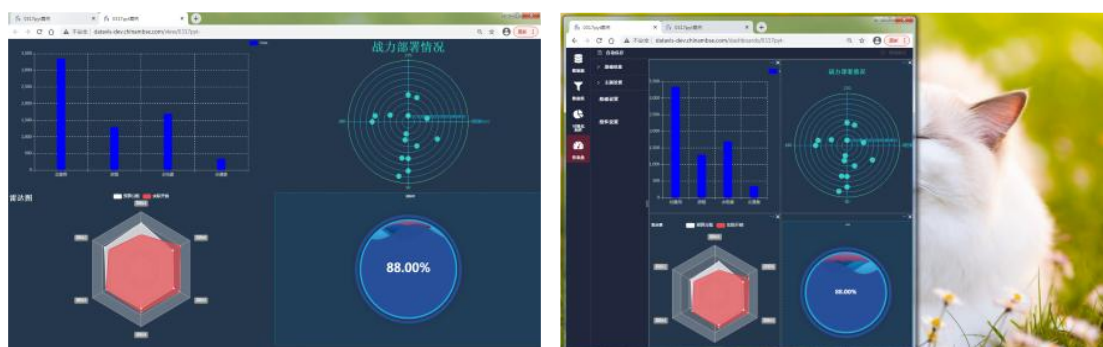
(b) 拖拽前组件的大小

图 4-19 组件拖拽前、后自适应测试结果

实验结果表明，鼠标拖动组件容器后，大屏页面容器内的组件可随着拖动自适应改变组件的宽高进行展示。

测试二：在全屏模式和窗口模式下预览大屏。

在屏幕分辨率为 1680x1050 的终端页面，对构建的大屏进行全屏预览和非全屏预览，测试结果如图 4-20 所示。



(a) 全屏模式

(b) 非全屏模式

图 4-20 全屏模式下和非全屏模式下自适应测试结果

实验结果表明，在全屏模式和非全屏模式下，可视化组件可以根据整体渲染窗口的大小自适应调整组件的布局和宽高，并填充整个窗口。

测试三：在不同分辨率的终端预览大屏。

最后考虑在不同屏幕分辨率的终端对大屏全屏预览，测试组件的自适应情况。在不同终端设备下设置屏幕的分辨率大小，分别在 Chrome 浏览器中打开同一个构

建好的大屏，测试画布页面的组件随分辨率变化的自适应情况。

如表 4-3 所示，为在不同终端显示器基于不同分辨率下的自适应测试结果。

表 4-3 在不同分辨率终端的自适应测试结果

终端显示器型号	屏幕大小	设置分辨率	自适应效果
华硕 VG27AQL1A	27 英寸	2560x1440	最好
		1920x1080	较好
戴尔 D2421DS	23.8 英寸	2560x1440	最好
		1920x1080	较好
戴尔 P2213	22 英寸	1680x1050	最好
		1440x900	较好
戴尔 E2010H	20 英寸	1600x900	最好
		1280x720	较好
Thinkvision L197wide	19 英寸	1440x900	最好
		1280x800	较好
戴尔 G3	15.6 英寸	1920×1080	最好
		1280×720	较好
联想 Z410	14 英寸	1366x768	最好
		1280×720	较好

根据实验结果可知，在不同终端的不同分辨率下大屏均有较好的自适应效果。所有自适应的测试结果总结如表 4-4 所示。

表 4-4 不同方案的自适应功能测试结果总结

测试方案	测试点	测试结果
缩放组件	在画布页面拖拽组件是否自适应	与预期相符
调整浏览器可视区域	调整浏览器窗口组件是否自适应	与预期相符
不同终端页面设置不同分辨率预览大屏	在不同终端下组件是否自适应	与预期相符

根据上述三种方案的自适应实验的测试结果，证明了本文构建的大屏具有较好的自适应展示效果，可满足不同终端分辨率下的可视化需求。

4.3.2 性能测试

一个系统的性能好坏不仅影响用户的操作体验，也决定着系统在未来应用实践的可行性。因此，在验证功能完备的基础上，还需对系统的性能进一步做评估实验。性能测试可以验证系统的鲁棒性，通过性能测试结果，可以分析系统中存在的潜藏缺陷并对系统性能优化改善。本文对比常用大屏的性能指标，主要从以下两个方面进行测试。

- 1) 更新组件视图，对浏览器内存消耗的影响
- 2) 频繁发送数据更新请求，对大屏实时响应速度的影响

本文围绕上述测试点，进行了性能实验测试。对测试结果进行讨论分析的基础上给出了优化方案，并对优化的改进结果再次进行了测试验证。

(1) 浏览器内存消耗测试

在实际对可视化大屏的操作中，可能存在对大屏内的组件进行重新编辑的情况，此时大屏预览界面的视图会进行更新。如果对组件的配置频繁操作，会导致浏览器内存消耗增大，系统响应时间延长，进而降低用户的体验。因此，此部分测试大屏内的组件视图更新时，浏览器的内存随时间的消耗情况。

测试过程如下：在 Chrome 浏览器中运行系统，并打开构建好的大屏预览页面。测试模拟每 2s 更新可视化组件的数据，并重新绘图。打开浏览器的开发者工具，记录浏览器的占用内存随时间的变化情况。如图 4-21 所示为实际测试结果。

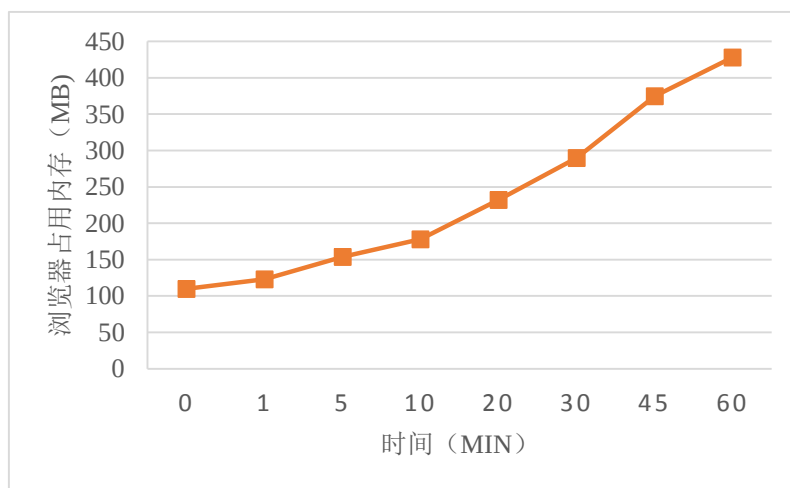


图 4-21 浏览器占用内存测试结果

分析测试结果，浏览器内存消耗随时间变化不断增多。在实际测试中，在 15 分钟后，浏览器内存消耗超过了 200M，此时大屏预览页面出现了略微卡顿。分析原因，这是因为系统为了提高页面的渲染速度，将生成的可视化结果的视图信息以

Dom 元素的形式存储在内存中而没有主动释放, 导致占用内存不断增加。

针对这个问题, 本文提出了内存清理的优化方法: 当系统监听到可视化组件的数据配置项发生变化时, 先清除上一个 Echarts 视图的缓存信息, 再加载最新配置项数据, 最后调用绘图方法进行渲染。希望通过清理缓存的方式降低浏览器内存的消耗, 减少页面卡顿的频率。实施优化方案后, 再次对系统测试, 如图 4-22 为优化前后的测试结果。

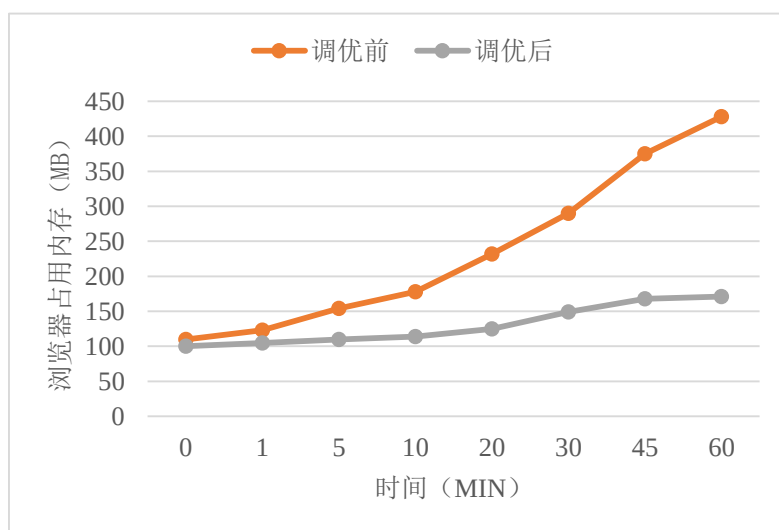


图 4-22 调优前后浏览器占用内存性能对比结果

分析测试结果, 优化后的方法释放了多余的缓存信息, 因此内存占用大大降少, 随着时间增长消耗情况趋于平稳。实验结果证明, 本文改进后的方法优化性能较好, 降低了视图更新对浏览器内存占用的影响, 提高了页面的流畅性。

(2) 响应时间及优化措施

在大屏交互中设置了数据刷新模块, 当数据指标发生变化时, 可视化视图会实时更新。由于数据源发生更新的实际时间间隔较长, 若频繁发送数据更新请求, 会导致系统资源的浪费。因此, 此部分测试数据频繁更新时, 大屏的实时响应情况。

测试过程如下: 模拟 1s 内发送 500 次数据更新的请求, 观察随着测试时间的增长, 视图更新的响应时间变化。其中, 响应时间以发送更新请求的时间戳为开始时间, 以视图调用绘图方法的时间戳为截止时间, 计算两者的差代表实际响应的时间。如图 4-23 为测试结果。

根据测试结果可知, 最初视图的平均响应时间约为 817ms。随时间变化, 请求次数增多, 响应时间显著增加。这不仅大大增加了用户的等待时间, 并且频繁发起更新请求也会消耗大量的系统资源。



图 4-23 实时大屏响应时延测试结果

针对数据频繁更新的问题，本文引入“节流”算法进行优化，节流的核心思想是当持续触发事件时，在固定周期内只执行一次动作。基于定时器的“节流”算法详细实现过程是：当用户触发事件时，首先判断当前是否存在定时器，如果不存在，则设置一个周期为 T 的定时器，并执行定时器内的事件，最后清空定时器。如果存在定时器，则不执行事件。本文在用户执行数据刷新操作和实际发起数据更新请求之间引入“节流”的思想，如图 4-24 为引入“节流”前后的数据更新流程对比。

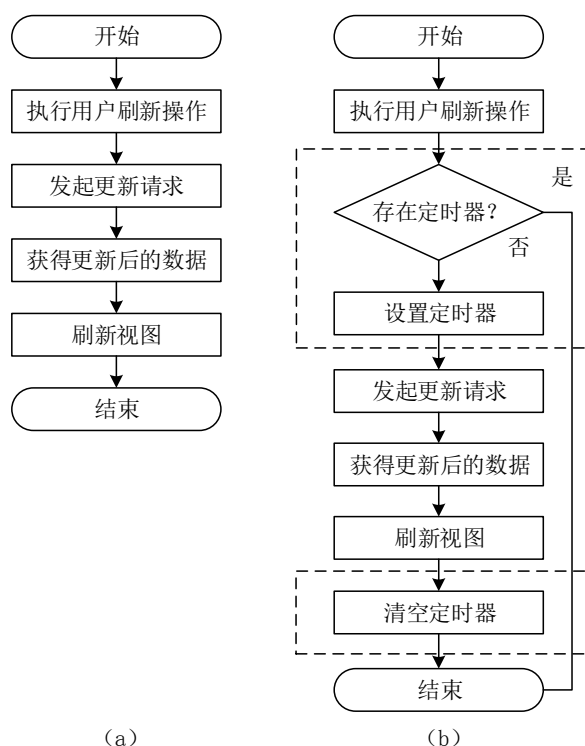


图 4-24 引入“节流”前后的数据更新流程对比。(a)节流前；(b)节流后

本文设置定时周期为 1s，如果在 1s 内用户多次发起数据刷新的操作，仅在 1s 后执行一次发起数据更新的请求。这样可以保证在一定时间内连续触发刷新事件时，只按照设定的频率 T 发起更新数据的请求，进而获取最新数据来更新相应的视图。

实施优化方案后，再次对系统测试，如图 4-25 为优化前后的测试结果对比。

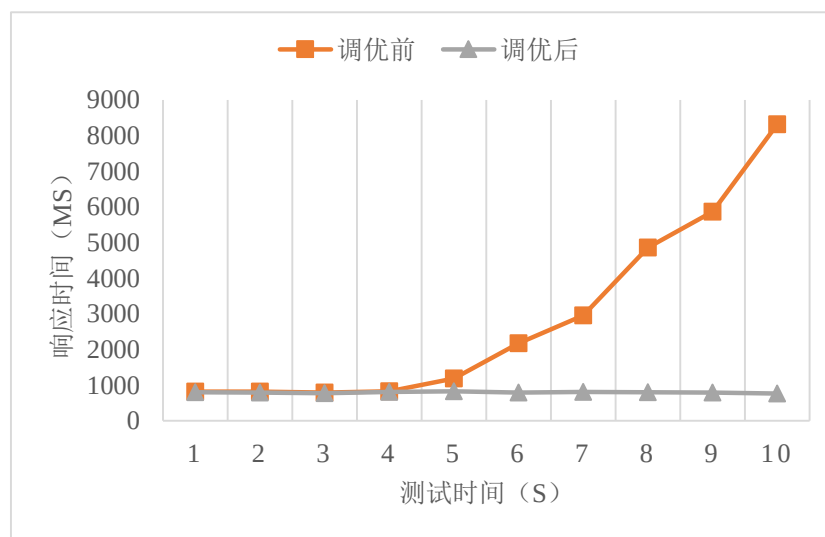


图 4-25 调优前后实时大屏响应时延对比结果

根据测试结果可知，引入“节流”的优化方法后，减少了大屏的响应时间。测试优化后的平均响应时间在 798ms 左右。根据页面响应的“2-5-8”原则，当页面的响应时间在 2 秒内，会让用户拥有最佳体验。因此优化后的方案不仅增加了大屏系统鲁棒性，也提升了用户的交互性。

4.4 本章小结

本章基于对可视化大屏构建方式的分析，提出一种基于页面自适应的大屏敏捷构建方法。首先，本文对比了主流的大屏构建方法的优缺点。接着，本文在研究大屏动态构建方案和大屏渲染机制的基础上，提出了一种大屏敏捷构建框架，并实现了面向组件的大屏构建系统。最后，设计了不同场景下的自适应策略，实现了可视化结果的自适应展现。

功能测试和性能测试的实验结果显示，基于本方法构建的自适应大屏，在功能和性能上均表现良好。此外，本工具提高了构建大屏的灵活性和用户的交互性，已经在实际项目中得到了实践应用。

第五章 面向多源异构数据的可视化案例展示

本章基于前文的研究成果,面向多源异构数据可视化的需求,使用构建的组件库和开发的大屏敏捷构建工具快速搭建联合作战电磁态势可视化分析场景,用于对复杂多源的电磁态势关键指标数据进行实时可视化展示与分析。

5.1 案例需求分析

电磁态势指电子对抗双方在作战空间区域内,根据电磁力量的分布和活动形成的态势^[57]。掌握电磁态势对克敌制胜有着至关重要的影响,通过可视化可以将复杂多源的空间电磁数据直观展示出来,而战场电磁态势可视化分析是辅助战场指挥人员感知态势、高效决策的重要依据。如今,面向作战仿真的电磁态势可视化研究已成为当前的研究热点。一种主流的观点把电磁态势的核心要素可划分为电磁设备、电磁设备的分布以及电磁活动^[59]。

5.1.1 电磁设备可视化

电磁设备作为战场空间中电磁态势的基本组成元素,电磁设备的可视化需求包括电磁设备类型和相关基本属性状态的显示。其中,电磁设备的种类分为五大类,分别是电子对抗、预警探测、指挥控制、导航识别及火控制导设备。基本属性包括名称、装备型号、敌我属性等。基本状态包开机状态、工作状态,坐标位置、信号频率等。

5.1.2 电磁行动可视化

作战行动是影响战场态势的关键要素,需要对敌对双方的电磁行动进行可视化展示。以下是部分常见的电磁行动:

(1) 对敌方干扰压制行动

可视化内容包括:任务的起始时间、执行任务的电子干扰站信息;目标预警机当前的信息,如雷达探测范围、通信畅通区的理论值;目标在行动执行前后的作战效能分析对比;行动达成的预期压制效果等。

(2) 远距离空地支援干扰行动

除了需要显示我方干扰任务的基本信息,如行动时间和行动实体等,还需要展示受干扰的敌方目标以及干扰前后目标的资源效能对比分析。

(3) 电子防空行动

需要显示任务的相关基本信息，防空保卫的目标区域、敌方的威胁目标及行动前后的目标效能变化情况。

5.1.3 电磁态势四域分布可视化

为了从宏观上把握战场电磁态势的环境变化，还应对态势要素从时、空、频、能进行分析显示。表 5-1 是部分电磁态势在各域应该显示的内容。

表 5-1 电磁态势四域显示内容

域类型	显示内容
时域	折线图：表示通联活跃程度随时间变化信息
空域	极坐标图：各电磁设备和行动的分布信息
频域	甘特图：各行动任务的频率使用分布信息
能域	饼图：电磁系统的作战资源效能占比信息， 雷达图：电磁信号场强度分布信息

5.2 电磁态势数据处理

5.2.1 电磁态势数据采集

实际电磁态势的相关数据，来自实时的传感器设备及系统数据，并且根据其保密性不便对外透露。因此本文通过对案例需求分析以及与项目合作研究所提供的脱密数据，按照电磁态势需要可视化的内容，创建电磁态势的数据集合。

5.2.2 电磁态势数据变换

在电磁分析系统中，一般需要采用数据融合方法，将多个传感器接收到的数据源信息进行联合处理，来获得精确的信息统计与完整的战场态势分析数据。

因此电磁信息数据融合面临将多个传感器数据的时间信息进行统一度量，以获取相关数据在时、空、频域的特征，数据处理包括以下几个步骤。

（1）时域坐标变换

将各传感器采集的数据信息以侦察点为导向，按照相同的时间基准排布，整合成时-频信息分布。

（2）空域坐标变换

将各传感器的目标数据信息变换到同一个空间坐标系显示。

（3）频域信息整理

将多点位的传感器信息进行统计分析，得出全频段的侦察信息。

5.2.3 电磁态势数据存储

将数据进行处理和整合后，需要存储至数据库保存，为模拟多源异构数据可视化需求，我们将所有的信息以下面三种形式存储。

(1) 文本形式

以 Excel 或 CSV 的形式存储数据信息。

(2) 关系型数据库

将数据信息存储在关系型数据库，如 MySQL、PostgreSQL 数据库中。

(3) 非关系型数据库

将数据信息存储在 Key-value 型数据库，如 Redis 数据库中。

5.3 电磁态势可视化分析场景实现

在本节中将详细描述电磁态势可视化分析场景的实现过程。首先查询存储在各数据库的相关电磁态势数据，建立相应的数据查询集合，这些数据集合有统一的数据预览视图。接着，基于案例需求分析，根据态势数据的特征，选择相应的组件模板，设计并生成相关的可视化组件。最后，基于生成的组件，构建可视化大屏对电磁态势可视化分析场景进行展示。

5.3.1 电磁数据查询集合

如图 5-1 所示是基于 Excel、MySQL 和 Redis 数据源形成的数据预览图。

电测设备类型	数量	区域	时间	活跃度	分组
探测预警	40	区域A	9:00:00	50	坦克连指挥网
指挥控制	215	区域A	9:10:00	60	坦克连指挥网
导航识别	35	区域A	9:20:00	90	坦克连指挥网
火控制导	63	区域A	9:30:00	100	坦克连指挥网
电子对抗	108	区域A	9:40:00	120	坦克连指挥网
探测预警	50	区域B	9:50:00	130	坦克连指挥网
指挥控制	200	区域B	10:00:00	140	坦克连指挥网
导航识别	30	区域B	10:10:00	150	坦克连指挥网
火控制导	75	区域B	10:20:00	180	坦克连指挥网

指标	指标最大	值	分类	总群	分群	指挥控制站序列号	控制站状态	雷达侦查站序列号	雷达侦查
雷达防空	40	30	我方	要地防空	雷达对抗	控制站kz10	雷达工作	侦查站zc11	雷达侦查
远距离支援	40	19	我方		通信对抗	控制站kz23	通信干扰	侦查站zc21	等待侦查
伴随掩护	40	17	我方		光电对抗	控制站kz33	雷达干扰	侦查站zc31	侦查结束
反辐射打击	40	27	我方	野战防空	雷达对抗	控制站kz12	通信干扰	侦查站zc13	等待侦查
精导防护	40	31	我方		通信对抗	控制站kz22	执行结束	侦查站zc21	雷达侦查
雷达防空	40	26	敌方		光电对抗	控制站kz35	正在执行	侦查站zc33	雷达侦查
远距离支援	40	25	敌方						
伴随掩护	40	13	敌方						
反辐射打击	40	30	敌方						

图 5-1 电磁数据查询集合的数据预览图

首先建立查询集合,使用数据查询接口分别查询和解析存储在数据库的信息以及文本格式的信息,并形成相应数据集合。根据以上结果可知,本系统可实现将多源异构数据进行统一整合,并转化为统一的数据预览图。

5.3.2 组件设计与生成

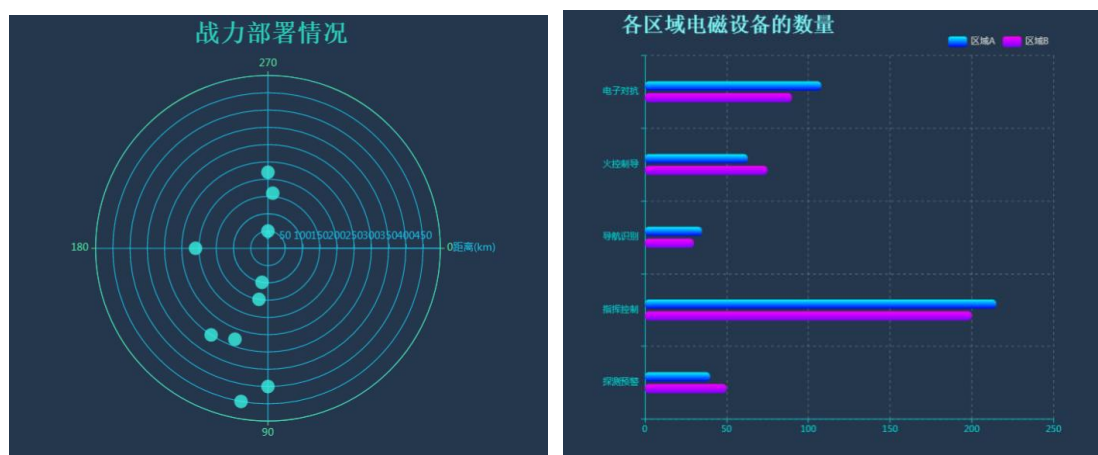
这一步根据需求分析,根据电磁态势的相关数据特征,选用合适的组件模板设计并生成组件。下面对生成的部分关键组件进行说明。

(1) 极坐标图

极坐标系属于二维坐标系统,该坐标系由一个角度轴和径向轴构成。可用一个夹角和一段相对极点的距离来表征任意点的位置。极坐标图则是将这些极坐标信息在环形区域表示出来的图形格式。因此可通过极坐标图直观展示我方战力部署分布情况,如图 5-2(a)所示。

(2) 柱状图

柱状图采用类目轴表征事物的类别,采用数目轴作衡量数值的大小,并使用水平或者垂直形状的柱形来比较多个对象在相同指标的不同。因此可用作战场区域内各种类电磁设备的数量对比,如图 5-2(b)所示。



(a) 极坐标图实现

(b) 柱状图实现

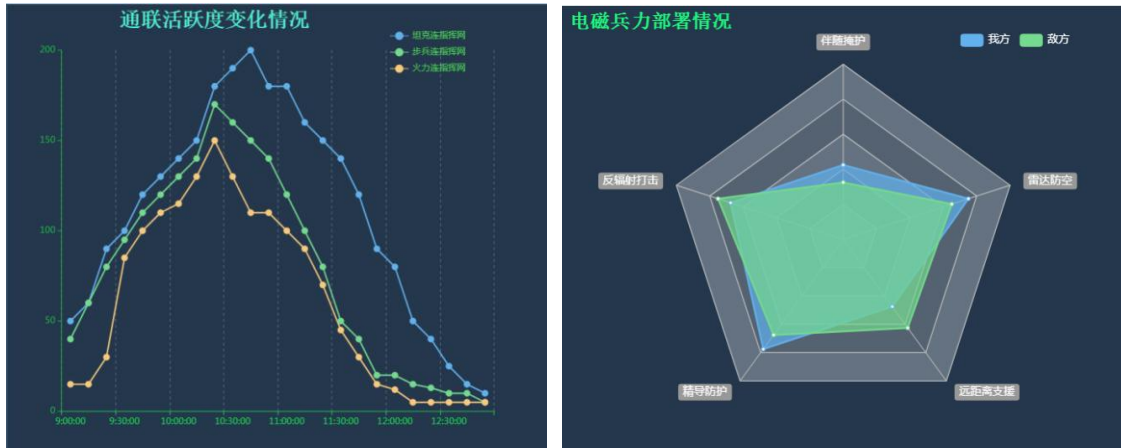
图 5-2 极坐标图和柱状图组件的实现效果

(3) 折线图

折线图构成与柱状图类似,区别是以折线的上升或下降来表示一定时间间隔的数值变化趋势。因此可用于同时显示多个对象的某种属性值在一定区域范围内的变化情况。如可以表示通联活跃度随时间的变化情况,如图 5-3(a)所示。

(4) 雷达图

雷达图显示的轴是根据变量的数目确定，从中心开始，每个变量都具有自己的轴并且以径向排列，所有变量的连线形成多边形区域，因此可用来显示一个对象在多个指标维度的性能分布情况，如可以显示电磁兵力部署在不同指标的能力分布情况，如图 5-3(b)所示。



(a) 折线图实现

(b) 雷达图实现

图 5-3 折线图和雷达图组件的实现效果

5.3.3 可视化分析场景实现

基于生成的组件，进一步的我们构建了可视化大屏，实现电磁态势可视化分析场景展示。如图 5-4 所示为基于本文提出的方法构建的电磁态势可视化分析平台。



图 5-4 电磁态势可视化分析平台实现

根据图 5-4 可知,通过快速搭建联合作战电磁态势数据可视化分析平台,可以对复杂多源的电磁态势的关键指标数据进行实时的可视化展示和分析。此外,由于大屏内的部分数据源来自实时的数据库,当数据库的数据发生变化时,大屏内的数据可根据设置的数据刷新周期实现动态更新。

5.3.4 结果分析

案例实现结果表明,根据本文提出的基于模型的组件可重构方法和基于页面自适应的大屏敏捷构建方法,可以根据需求,通过可视化配置界面快速建立面向作战场景系统下的电磁态势可视化效能分析平台。实现动态展示系统的关键性指标变化,辅助指挥人员实时掌握战场的综合态势,快速挖掘高价值情报。以此提高分析人员的状况感知能力,提升指挥决策效率。

5.4 本章小结

本章通过构建的组件库和开发的大屏敏捷构建工具,快速建立了电磁态势数据可视化分析场景。根据案例实现过程可知,本文的研究成果也适用于其他领域的多源异构数据可视化需求,可以建立相应的数据分析场景平台。

因此本文的研究成果对于不同领域的数据可视化分析具有较高的应用价值。此外,本文提出的大屏敏捷构建工具已在实际项目中获得了较好的反响。

第六章 总结与展望

6.1 总结

数据可视化需求的增多引起了各界对其应用实践的广泛关注。在大数据时代,数据具有海量性和异构性,因此如何基于可视化需求快速建立相应场景进行数据可视化分析是迫切需要解决的问题。本文依据现有可视化流程中存在的组件生成效率低和大展构建流程灵活性差的问题,在研究传统组件库的基础上,提出一种基于模型的组件可重构方法,并通过对比实验证明该方法的优越性。在组件研究基础上,本文还提出一种基于页面自适应的大屏敏捷构建方法,并通过实验对大屏系统的功能和性能进行了测试,以验证其效果。本文主要工作总结如下:

(1) 介绍了异构数据可视化的应用前景,并总结了国内外数据可视化及相关工具平台的研究成果。

(2) 对数据可视化的基本理论和分类做了介绍,阐明了多源异构数据可视化流程中涉及的关键技术及相应的实现途径。

(3) 研究了传统组件库生成组件的原理和流程,并在此基础上提出一种基于模型的组件可重构技术。基于本方法构建的组件库共集成了 24 种 Echarts 类型的图表组件,通过扩展实验验证了本方法的扩展性。此外,本文还对构建的组件库的功能与性能表现情况进行了完备的实验测试,并与其他组件库进行了对比分析,证明了构建的组件库的优越性。

(4) 研究了基于可视化组件的大屏构建机制,并提出一种基于页面自适应的敏捷大屏构建技术。通过功能测试实验验证了基于本工具构建的大屏的灵活性与可视化结果的自适应性。通过性能实验测试了构建的大屏系统在占用内存及响应时间的性能表现。此外,本文还对实验结果进一步分析并提出了优化方案。

(5) 最后,基于本文提出的组件快速生成方法和大屏敏捷构建方法,实现了对联合作战电磁态势可视化的案例研究,搭建的分析平台可用于对复杂多源的电磁态势的关键指标数据进行实时可视化展示与分析。案例实验证明了本文提出的工具在多源异构数据可视化分析有较好的应用前景。

6.2 未来展望

本文研究了可视化流程中的组件生成技术和大屏构建技术,其中仍然存在可提升和进一步研究的空间,包括以下几个方面:

(1) 本文提出的基于模型的组件可重构方法应用了前端框架中的 Angular，以实现双向数据绑定机制。由于目前主流的前端框架还有 VUE，且 VUE 也有双向绑定机制和完整的框架体系，因此可考虑将本文的方法与 VUE 结合实现，以提升该方法的应用性。

(2) 本文设计的大屏自适应策略，是根据最外层容器的变化来判定内部组件是否需要调整大小，此策略没有考虑在移动端的实现效果。基于此，可以考虑在未来的研究中引入对移动端的自适应适配方案，以覆盖更广泛的可视化需求。

(3) 本文提出的大屏动态构建技术，是在前端实现对大屏的渲染，这在数据量特别大的情况下可能会加重浏览器的负担，因此可以考虑在今后的研究中在后端实现大屏的渲染机制，以提升系统的整体性能。

(4) 本文实现的异构数据可视化案例，没有对可视化流程中的数据处理和数据整合技术进行详细的研究。因此，可以在本文的组件生成与大屏构建技术研究的基础上，对数据采集、数据处理与数据整合技术进行深入研究，融合数据分析流程的各个关键技术，以构建更加完整的数据可视化与分析机制。

致谢

时光荏苒，白驹过隙。回首这几年，我在各个方面获益匪浅，这得益于遇见的良师益友，感谢你们的出现，为我这段生涯挥洒了浓厚而鲜艳的笔墨。

感谢我的学校与学院为我提供了丰富的教学资源。

感谢项目指导老师王坚教授。王老师在科研上对我的严格要求，不仅有助我形成了严谨的科研逻辑，还培养我形成了独立分析与解决问题的能力。王老师对于学术问题的钻研精神，也让我深受鼓励。在王老师的指导下，我才能沉下心在学术上不断钻研，完成这篇论文的相关研究。

感谢我的导师杨鍊老师。杨老师是一位和蔼可亲的老师，她乐观的人生态度启迪着我。无论在科研上还是生活上遇到困难，她都能及时帮助我分析难处，鼓励我积极面对，给与了我很多关怀与帮助。同时还要感谢团队内的李桓老师，他在这三年内也给与了我诸多帮助。

还要感谢教研室的各位同学，同宿舍热情的杨文秀，文静的陈子昱，同届的朱继飞、徐博轩、师雨、于豪、李东升，还有认真负责的师兄师姐们，开朗活泼的师弟师妹们。谢谢你们带来的欢声笑语，是你们让这个大团队充满生机。

谢谢我的男朋友陈思捷，从大学到研究生，是你的支持与鼓励，让我不断朝着更好的自己迈进。

最要感谢的是我的家人，感恩于你们的无私付出，让我十几年来不断在学海的天空遨游，不断向艰难的巨峰攀登。

最后，诚挚的向拨冗垂阅本文并提出修改意见的各位老师们表示感谢！

参考文献

- [1] R. Pang et al. Dynamic Evaluation Technology of Bridge Based on Multi-source Heterogeneous Data Fusion[J]. IOP Conference Series: Earth and Environmental Science, 2021, 638(1): 012064
- [2] J. J. Hilda, C. Srimathi, B. Bonthu. A Review on the Development of Big Data Analytics and Effective Data Visualization Techniques in the Context of Massive and Multidimensional Data[J]. Indian Journal of Science & Technology, 2016, 9(27)
- [3] Y. C. Lin, A. Gro, T. Kirsten. Integration and visualization of spatial data in LIFE[J]. Information Technology, 2017, 59(4): 161-170
- [4] 李佳娱, 刘彦君, 刘如. 面向情报获取的信息表达路径研究[J]. 天津科技, 2017, 44(03): 25-28
- [5] J. Choo, H. Park. Customizing Computational Methods for Visual Analytics with Big Data[J]. IEEE Computer Graphics and Application, 2013, 33(4): 22-28
- [6] D. Keim, H. Qu, K. L. Ma. Big-Data Visualization[J]. IEEE Computer Graphics & Applications, 2013, 33(4): 20-21
- [7] X. M. Wang, T. Y. Zhang, Y. X. Ma, et al. A Survey of Visual Analytic Pipelines[J]. Journal of Computer Science&Technology, 2016, 31(4): 787-804
- [8] Y. F. Sheng, et al. Visualization Research and Application of Water Quality Monitoring Data Based on ECharts[J]. 2020, 2(1): 1-8
- [9] 潘玉春, 胡剑锋, 朱玉付. WEB 可视化技术在电网大数据场景下的应用研究[J]. 贵州电力技术, 2019, 022(003):8-12
- [10] 林胜华. 战场态势可视化技术的研究与实现[D]. 成都: 电子科技大学, 2015, 50-60
- [11] 李宏波,徐铁军.信息运行可视化平台的设计与实现[J]. 电力信息与通信技术, 2012, 10(3): 103-106
- [12] A. Abdelalim, W. O'Brien, Z.X. Shi . Data visualization and analysis of energy flow on a multi-zone building scale[J]. Automation in Construction, 2017, 84: 258-273
- [13] H. Mei, Y. Ma, Y. Wei, et al. The design space of construction tools for information visualization: A survey[J]. Journal of Visual Languages & Computing, 2018, 44: 120-132
- [14] L. Grammel, M. Tory, M. A. Storey. How Information Visualization Novices Construct Visualizations[J]. IEEE Transactions on Visualization and Computer Graphics, 2010, 16(6): 943-952

- [15] O. Sindiy, K. Litomisky, S. Davidoff, et al. Introduction to Information Visualization (InfoVis) Techniques for Model-Based Systems Engineering[J]. Procedia Computer Science, 2013, 16: 49-58
- [16] 张卓, 宣蕾, 郝树勇. 可视化技术研究 with 比较[J]. 现代电子技术, 2010, 17(45): 141-146
- [17] D. A. Keim. Information visualization and visual data mining[J]. IEEE Transactions on Visualization and Computer Graphics, 2010, 8(1): 1-8
- [18] 陈为, 沈则潜, 陶煜波. 数据可视化[M]. 北京: 电子工业出版社, 2019, 29-38
- [19] L. Ming. Message from the Editor-in-Chief. IEEE Visualization Conference and the IEEE Information Visualization Conference[J]. IEEE Transactions on Visualization & Computer Graphics, 2011, 17(12): ix
- [20] L. Zhang, A. Stoffel, M. Behrisch, et al. Visual analytics for the big data era-A comparative review of state-of-the-art commercial systems[J]. 2012 IEEE Conference on Visual Analytics Science and Technology, USA , 2012, 173-182
- [21] 王宁. 面向大屏的图表展示系统设计与实现[D]. 西安, 西安电子科技大学, 2019
- [22] Clark, Dan. Beginning Microsoft Power BI[M]. Apress, Berkeley, CA, 2020, 15-20
- [23] R. Wesley, M. Eldridge, P. T. Terlecki. An analytic data engine for visualization in tableau[C]. Acm Sigmod International Conference on Management of Data, Greece, 2011, 527-530
- [24] N. L. Leech, K. C. Barrett, G. A. Morgan. SPSS for intermediate statistics: Use and interpretation, 3rd ed.[J]. Fertility & Sterility, 2008, 74(3): S105
- [25] 何鑫. 基于 GeoServer 和 OpenLayers 的 WebGIS 地理信息服务系统的设计与实现[J]. 山西交通科技, 2017(4): 122-126
- [26] R. K. Mavi, M. Goh, N. K. Mavi, et al. Cross-Docking: A Systematic Literature Review[J]. Sustainability, 2020, 12(11): 4789-4789
- [27] 陈为. 数据可视化的基本原理和方法[M]. 北京: 科学出版社, 2013, 30-50
- [28] 袁晓如, 张昕, 肖何, 等. 可视化研究前沿及展望[J]. 科研信息化技术与应用, 2011, 2(4): 3-13.
- [29] C. F. Lai, Z. X. Lin, R. K. Jiang, et al. Automatic Annotation Synchronizing with Textual Description for Visualization[C]. CHI '20: CHI Conference on Human Factors in Computing Systems. 2020
- [30] G. Z. L, M. Tian, Q. M. Xu , et al. GoTree: A Grammar of Tree Visualizations[C]. CHI '20: CHI Conference on Human Factors in Computing Systems. 2020.

- [31] M Zhu , W Chen ,Y Hu, et al. DRGraph: An Efficient Graph Layout Algorithm for Large-scale Graphs by Dimensionality Reduction.[J]. IEEE transactions on visualization and computer graphics, 2020, 27(2): 1666-1676
- [32] J Li, S Chen, W Chen, et al. Semantics-Space-Time Cube: A Conceptual Framework for Systematic Analysis of Texts in Space and Time.[J]. IEEE transactions on visualization and computer graphics, 2020, 26(4) : 1789-1806
- [33] 潘卫国, 何宁, 薛健,等. 海量超声数据体可视化研究[J]. 电子学报, 2016, 044(002): 472-478
- [34] 张杰, 傅文博. 网络安全数据可视化融合的分析方法[J]. 微电子学与计算机, 2019, 036(006): 101-104
- [35] Q. Li, Z. M. Wu, L. L. Y, et al. WeSeer: Visual Analysis for Better Information Cascade Prediction of WeChat Articles[J]. IEEE transactions on visualization and computer graphics, 2020, 26(2) : 1399-1412.
- [36] M. Stacey, J. Salvatore, A. Jorgensen. Fundamentals of Visualization[M]. John Wiley & Sons, Ltd, 2014.
- [37] Mitchell. Data visualization: a guide to visual storytelling for libraries [J]. Technical Services Quarterly, 2017, 34(4): 439-440
- [38] R. Guo, J. M. Luo, W. Q. Gao. A New Method of Measuring the Age of Abalone Based on Data Visualization Analysis[J]. Journal of Physics: Conference Series, 2021, 1744(4): 042181
- [39] W. M, G. Gorges, Keim. Daniel. Interactive data visualization; foundations, techniques, and applications.[J]. Scitech Book News, 2010, 34(3)
- [40] 聂俊岚,陈欢欢,郭栋梁,张继凯.多维数字战场态势可视分析研究[J]. 小型微型计算机系统, 2014, 35(03): 626-629
- [41] 卢健, 张倩, 左世祥,等. Web 地图热力图原理及应用[J]. 科技创新与应用, 2019(35): 174-176
- [42] 肖永飞. 医学数据三维交互可视化方法的研究[D]. 哈尔滨, 哈尔滨工业大学.
- [43] 张杰, 傅文博. 网络安全数据可视化融合的分析方法[J]. 微电子学与计算机, 2019, 036(006): 101-104
- [44] 张雯政. 商业智能中基于 B/S 模式的数据可视化研究与应用[D]. 青岛,中国海洋大学,2012
- [45] 孙艳华, 冯妍, 李宏然. 基于 Python 平台的金融数据分析技术研究[J]. 信息与电脑(理论版),2020,32(15): 155-159
- [46] B. Haber. Visualization idioms: a conceptual model for scientific visualization systems[J]. Visualization in Scientific Computing IEEE, 1990: 74-93

- [47] S. C. Kim, K. K. Seo, I. K. Kim, et al. Readings in information visualization: using vision to think[J]. The Journal of Urology, 1999, 161(3): 964-969.
- [48] Ehrlich J, Marten J, et al. Functional dependency discovery: An experimental evaluation of seven algorithms[J]. Proceedings of the VLDB Endowment, 2015, 8(10): 1082-1093
- [49] S. X. Song, L. Chen. Differential dependencies: Reasoning and discovery[J]. ACM transactions on Database Systems, 2011, 36(3): 16
- [50] X. Chu, I. F. Ilyas, P. Papotti. Holistic data cleaning: Putting violations into context[C]. 2013 IEEE 29th International Conference on Data Engineering, 2013:458-469.
- [51] H. Shuang, T. Nan, G. Li, et al. A novel cost-based model for data repairing[C]. IEEE Transactions on Knowledge and data engineering 2016 29(4): 727-742
- [52] F. Chiang, R. J. Miller. A unified model for data and constraint repair[C]. 2011 IEEE 27th International Conference on Data Engineering, 2011: 446-457
- [53] A. A. Qahtan, A. K. Elmagarmid, R. C. Fernandez, et al. FAHES: A robust disguised missing values detector[C]. Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery Data Mining, 2018: 2100-2109.
- [54] 李翠, 李治洪, 周力杰. 基于 HTML5 的 WebGIS 前端要素绘制性能优化研究[J]. 计算机科学与应用, 2016, 6(1): 9-20.
- [55] 钟文广, 万强, 徐常福. Highcharts 图表库在 web 开发中的应用[J]. 科技广场, 2017(4): 55-58
- [56] 谢韵佳. 面向 ECharts 的高效学籍信息可视化研究[J]. 软件导刊, 2019, 18(12): 271-276
- [57] 王枭, 米贵山, 王峰辉. 联合作战电磁态势可视化[J]. 电子信息对抗技术, 2018, 33(05): 57-61
- [58] 曹轶. 多物理场耦合并行可视化方法研究[D]. 中国工程物理研究院, 2017
- [59] 刘庆国. 联合作战电磁态势分析[M]. 北京: 解放军出版社, 2012

攻读硕士学位期间研究成果

1. 参与项目

2019.10 至今 基于原生云架构的数据可视化设计与服务

2019.02-2019.08 智慧工厂危险态势检测系统

2. 获得奖励

2019.10 研究生奖学金一等

2018.10 研究生奖学金三等